

ANDROID · TEST TASK · SOURCE CODE

شرح كود المشروع ملفاً ملفاً

FakeStore Auth — Token Handling App

لكل ملف: الكود الكامل منسّقاً مع أرقام الأسطر، ثم شرح مفصّل لما يفعله ولماذا
كُتِبَ بهذه الطريقة. وفي الجزء العاشر: المنطق كقصة، ودليل الاختبار على الهاتف.

MVVM

Retrofit + OkHttp

4

جلسات دراسة

21

اختباراً ناجحاً

56

ملفاً مشروحاً

الفهرس

0. قبل البدء: كيف تنقل المشروع إلى Android Studio

الجزء التمهيدي ★: دليل الدراسة

الجزء الأول: إعدادات المشروع (Gradle)

- 1. gradle/libs.versions.toml
- 2. settings.gradle.kts
- 3. build.gradle.kts (الجزر)
- 4. app/build.gradle.kts
- 5. gradle.properties
- 6. app/src/main/AndroidManifest.xml

الجزء الثاني: نماذج البيانات وال APIs

- 7. data/model/Product.kt
- 8. data/remote/ApiConfig.kt
- 9. data/remote/dto/AuthDtos.kt
- 10. data/remote/dto/ProductDto.kt
- 11. data/remote/AuthApi.kt
- 12. data/remote/ProductApi.kt

الجزء الثالث ★: منطق ال Token (قلب المهمة)

- 13. data/auth/Clock.kt
- 14. data/auth/AuthSession.kt
- 15. data/auth/TokenStorage.kt
- 16. data/auth/EncryptedTokenStorage.kt
- 17. data/auth/AuthExceptions.kt
- 18. data/auth/AuthEvent.kt
- 19. data/auth/AuthHeaders.kt
- 20. data/auth/AuthManager.kt ★★
- 21. data/auth/AuthInterceptor.kt
- 22. data/auth/TokenAuthenticator.kt

الجزء الرابع: ال Repository وحقن الاعتماديات (Hilt)

- 23. FakeStoreApp.kt
- 24. data/repository/ProductRepository.kt
- 25. di/Qualifiers.kt
- 26. di/NetworkModule.kt ★
- 27. di/DataModule.kt

الجزء الخامس: أدوات الواجهة المشتركة

- 28. ui/common/UiState.kt

ui/common/ErrorMessage.kt .29

ui/common/FlowExt.kt .30

ui/common/Formatters.kt .31

الجزء السادس: الشاشات (View + ViewModel)

- 32. res/values/strings.xml
- 33. res/values/themes.xml
- 34. res/layout/activity_main.xml
- 35. res/navigation/nav_graph.xml
- 36. MainActivity.kt
- 37. ui/login/LoginViewModel.kt
- 38. res/layout/fragment_login.xml
- 39. ui/login/LoginFragment.kt
- 40. res/layout/view_error.xml
- 41. ui/products/ProductsViewModel.kt
- 42. res/layout/item_product.xml
- 43. ui/products/ProductAdapter.kt ★
- 44. res/layout/fragment_products.xml
- 45. res/menu/menu_products.xml
- 46. ui/products/ProductsFragment.kt
- 47. ui/detail/ProductDetailViewModel.kt
- 48. res/layout/fragment_product_detail.xml
- 49. ui/detail/ProductDetailFragment.kt

الجزء السابع: الاختبارات (Unit Tests)

- 50. test/.../TestDoubles.kt
- 51. test/.../AuthManagerTest.kt
- 52. test/.../AuthInterceptorTest.kt
- 53. test/.../TokenAuthenticatorTest.kt

الجزء الثامن: الصورة الكاملة

- 54. رحلة طلب كاملة عبر الملفات
- 55. كيف تعرض المشروع للمدير (Demo)
- 56. تقابل المتطلبات مع الكود

الجزء التاسع: الوضع التجريبي (Mock Backend)

- 57. data/mock/MockProducts.kt
- 58. data/mock/MockBackendInterceptor.kt ★
- 59. test/.../MockBackendInterceptorTest.kt
- 60. كيف تعرض الوضع التجريبي للمدير

الجزء العاشر ★: فهم المنطق بالسيناريوهات

61. المنطق كله في سبع جمل

62. السيناريوهات خطوة بخطوة

63. اختبار فهمك (أسئلة متوقعة في المقابلة)

64. دليل اختبار التطبيق على الهاتف

65. نتائج المراجعة الشاملة

0. قبل البدء: كيف تنقل المشروع إلى Android Studio

الطريقة الأسهل (موصى بها)

1. حمّل الريبو من GitHub كملف ZIP، أو استخدم `git clone`.
2. افتح Android Studio، ثم اختر **Open** ← **File** وحدد مجلد المشروع.
3. انتظر انتهاء **Gradle Sync**، فهو يحمّل المكتبات تلقائياً.
4. اضغط **Run** ▶.

إذا أردت نسخ الملفات يدوياً إلى مشروع جديد

1. أنشئ مشروعاً جديداً من قالب **Empty Views Activity**.
2. اجعل اسم الحزمة **Package name** يساوي `com.example.fakestore`، واللغة **Kotlin**، و **Minimum SDK** يساوي 24.
3. انسخ الملفات بنفس المسارات المكتوبة في عنوان كل قسم من هذا الملف.
4. استبدل ملفات Gradle الموجودة بالملفات الواردة هنا، ثم اضغط **Sync Now**.

بنية المشروع

```
app/src/main/java/com/example/fakestore/
├─ FakeStoreApp.kt           Application + Hilt
├─ MainActivity.kt          the only Activity
├─ data/
│   ├─ model/Product.kt     domain model
│   ├─ remote/              Retrofit APIs + DTOs
│   ├─ auth/                ★ token logic (core of the task)
│   ├─ mock/                dev-only mock server (FakeStore outage)
│   └─ repository/          ProductRepository
├─ di/                      Hilt modules
├─ ui/
│   ├─ common/              UiState, errors, helpers
│   ├─ login/               Login screen
│   ├─ products/            list (RecyclerView)
│   └─ detail/              product details
app/src/main/res/           layouts, navigation, menu, strings
app/src/test/.../data/auth/ unit tests for the token logic
app/src/test/.../data/mock/ tests on the mock server
```

ترتيب القراءة المقترح

اقرأ الأقسام بالترتيب، فكل قسم يعتمد على ما قبله:

الجزء	المحتوى
التمهيدي ★	دليل الدراسة: ماذا تقرأ أولاً، وإلى أي عمق
الأول	إعدادات Gradle وال Manifest

الجزء	المحتوى
الثاني	نماذج البيانات وال APIs
الثالث ★	منطق ال Token، وهو قلب المهمة
الرابع	ال Repository وحقق الاعتماديات (Hilt)
الخامس	أدوات الواجهة المشتركة
السادس	الشاشات
السابع	الاختبارات
الثامن	الرحلة الكاملة للطلب، وكيف تعرض المشروع للمدير
التاسع	الوضع التجريبي (السيرفر الوهمي) أثناء تعطل FakeStore
العاشر ★	فهم المنطق بالسيناريوهات، ودليل اختبار التطبيق على الهاتف، ونتائج المراجعة

قبل أن تبدأ: اقرأ "الجزء التمهيدي: دليل الدراسة" التالي مباشرة. فيه خطة من أربع جلسات تقول لك ماذا تقرأ أولاً، وما الذي يمكن تحظّيه.

ملاحظة: أرقام الأسطر الظاهرة بجانب الكود تساعدك على متابعة الشرح.

سجل التعديلات

المشروع مرّ بخمس مراحل، وكلها مشروحة في هذا الملف:

المرحلة	ماذا أُضيف	لماذا	الأقسام
1. التنفيذ الأساسي	كل متطلبات المهمة: الدخول، والتخزين المشفّر، وال <code>Interceptor</code> ، والانتهاء بعد 60 ثانية، والتجديد الواحد، والخروج الإجباري، والقائمة، والتفاصيل، والخروج + 15 اختباراً	ملف المهمة	1 إلى 56
2. رسالة "السيرفر غير متاح"	نوع خطأ جديد <code>ServerUnavailableException</code> ، ورسالة دقيقة عند أعطال السيرفر (5xx)، واختباران	FakeStore تعطل، والتطبيق كان يقول "لا يوجد إنترنت"، وهذا غير دقيق	17، 20، 29، 51
3. الوضع التجريبي	سيرفر وهمي لنسخة التطوير فقط، يُفعّل من <code>gradle.properties</code> ، وشريط تنبيه في شاشة الدخول، و4 اختبارات	FakeStore بقي متعطّلاً للجميع، فلم يكن ممكناً تشغيل التطبيق أو عرضه	4، 5، 26، 38، 39، الجزء التاسع
4. المراجعة الشاملة	قفل <code>@Synchronized</code> في <code>EncryptedTokenStorage</code> ، وحقل اسم مستخدم بلا تكبير تلقائي، وتوضيح حدود السيرفر الوهمي، والجزء العاشر كاملاً	مراجعة كل الملفات قبل التسليم	16، 38، 55، الجزء التاسع، الجزء العاشر

المرحلة	ماذا أُضيف	لماذا	الأقسام
5. دليل الدراسة	الجزء التمهيدي: مستويات الأهمية، وخطة أربع جلسات، وقاموس Flutter ← Android، وقاعدة "لو حذفنا هذا السطر"	تسهيل الدراسة قبل المقابلة	الجزء التمهيدي

المجموع الحالي: 56 ملفاً مشروحاً، و21 اختباراً ناجحاً.

الجزء التمهيدي★: دليل الدراسة

هذا الجزء لا يشرح كوداً. يقول لك **ماذا تقرأ، وبأي ترتيب، وإلى أي عمق**، حتى تفهم المشروع بسرعة ودون أن تضيع في التفاصيل.

أ) ثلاثة مستويات للأهمية

ليس كل ملف بنفس الأهمية. وزّع وقتك هكذا:

المستوى	المعنى	الأقسام
● افهمه سطراً سطراً	قلب المهمة، والمدير سيسأل عنه حتماً	17، 20، 21، 22، 25، 26، 61، 62
● افهم الفكرة فقط	اعرف ماذا يفعل و لماذا نحتاجه، دون كل التفاصيل	14، 16، 24، 27، 29، 35، 36، 37، 41، 43، 46، 58
● مرّ عليه بسرعة	إعدادات وتصميم، لا يُسأل عنها عادة	1 إلى 12، 23، 28، 30 إلى 34، 38 إلى 40، 42، 44، 45، 47 إلى 49، 57

الفرق المهم:

- "لا أعرف **كيف** تعمل مكتبة Retrofit من الداخل": طبيعي، ولا أحد يسألك عنه.
- "لا أعرف **لماذا** كتبنا هذا السطر في `AuthManager`": خطر، لأن المدير يفترض أنك أنت من كتبه.

ب) خطة الدراسة: أربع جلسات

كل جلسة ساعة تقريباً. لا تنتقل للجلسة التالية حتى تجيب عن أسئلة نهاية الجلسة **دون النظر للملف**.

الجلسة	ماذا تقرأ	الهدف
1. الصورة الكاملة	61، ثم 62، ثم 54	تفهم القصة دون كود
2. قلب المهمة ●	17، 20 (ببطء)، 21، 22، ثم 25 و26	تفهم الكود الذي ينفّذ القصة
3. الواجهة وHilt ●	مقدمة الجزء السادس، ثم 36، 37، 41، 43، ثم 24 و27	تفهم كيف تصل البيانات للشاشة
4. الإثبات والعرض	51، 52، 53، ثم 55، ثم 63	تعرف كيف تثبت أن الكود صحيح وكيف تعرضه

أسئلة نهاية كل جلسة:

الجلسة	أجب بصوت عالٍ
1	ماذا يحدث عندما ترسل 5 طلبات معاً وال token منتهٍ؟ ومتى نُخرج المستخدم إجبارياً، ومتى لا؟
2	لماذا يوجد تحقق ثانٍ داخل القفل؟ ولماذا عميلاً OkHttp؟ ولماذا تراث الأخطاء من <code>IOException</code> ؟
3	من أين يأتي <code>AuthManager</code> إلى <code>LoginViewModel</code> ؟ وكيف تعرف الشاشة أن البيانات وصلت؟

ج) قاعدة "لو حذفنا هذا السطر"

أفضل طريقة لتعرف أنك فهمت سطرًا: اسأل نفسك **ماذا سيحدث لو حذفناه؟** هذه إجابات أهم الأسطر في المشروع:

لو حذفنا...	الملف	ماذا سيحدث
التحقق الثاني داخل القفل (السطر 78)	AuthManager	5 طلبات متزامنة تنتج 5 طلبات login بدل واحد
synchronized(refreshLock)	AuthManager	الطلبات تجدد كلها في نفس اللحظة، دون أي انتظار
@Singleton	AuthManager	كل كلاس يحصل على نسخة خاصة به، بقفل خاص. فيفشل منع التجديد المكرر، ولا تصل أحداث الخروج إلى MainActivity
فحص رقم الجيل في saveIfStillCurrent	AuthManager	تجديد بدأ قبل Logout يعيد المستخدم مسجلًا للدخول بعد أن خرج
@PlainClient العميل المنفصل	NetworkModule	طلب login يمر بـ AuthInterceptor ، فيطلب تجديداً، والتجديد هو login: حلقة لا نهائية
extends IOException	AuthExceptions	رمي الخطأ من داخل OkHttp يُسقط التطبيق (crash) بدل عرض رسالة
priorResponse فحص	TokenAuthenticator	سيرفر يرد 401 دائماً يسبب حتى 20 محاولة متتالية
@Synchronized	EncryptedTokenStorage	قد يُقرأ token قديم مع وقت حفظ جديد، فيُعامل ال token المنتهي كأنه صالح
popUpTo في action_login_to_products	nav_graph.xml	زر الرجوع من المنتجات يعيد المستخدم لشاشة Login وهو مسجل الدخول
throw e في catch (CancellationException)	ViewModels ال	تتعطل آلية إلغاء ال coroutines عند إغلاق الشاشة
_binding = null في onDestroyView	Fragments ال	تسريب ذاكرة (memory leak): واجهة ميتة تبقى في الذاكرة

د) قاموس سريع: من Flutter إلى Android

كل مفهوم في هذا المشروع له مقابل تعرفه من Flutter:

Flutter في	في هذا المشروع	القسم
pubspec.yaml	build.gradle.kts + libs.versions.toml	4, 1
Widget لشاشة كاملة	XML + Fragment	39, 38

Flutter في	في هذا المشروع	القسم
ChangeNotifier أو Bloc أو Cubit	ViewModel	37
setState أو emit	_state.update { ... }	37
Stream	StateFlow / Flow	30
BlocBuilder أو StreamBuilder	collectWhenStarted { render(it) }	30
Future و async/await	دوال suspend و coroutines	20 (د)
ListView.builder	Adapter + RecyclerView	43
Navigator أو go_router	nav_graph.xml + Navigation Component	35
Interceptors مع dio	Interceptor مع OkHttp	21
حزمة retrofit في Dart	Retrofit	12 ، 11
json_serializable	DTO + Gson	10
get_it أو injectable أو Provider	Hilt	27 ، 26 ، 23
flutter_secure_storage	EncryptedSharedPreferences	16
main() و runApp	MainActivity + FakeStoreApp	36 ، 23

هـ) خمسة مصطلحات تتكرر في كل الملف

المصطلح	المعنى باختصار
Thread	مسار تنفيذ مستقل. OkHttp يرسل كل طلب على thread خاص، لذلك قد تصل عدة طلبات إلى AuthManager في نفس اللحظة
Lock (قفل)	يضمن أن thread واحداً فقط ينفذ جزءاً من الكود في المرة الواحدة، والبقية ينتظرون
Interceptor	كود يمر عليه كل طلب قبل إرساله، فيستطيع تعديله، مثل إضافة token
Dependency Injection	الكلاس لا ينشئ ما يحتاجه بنفسه، بل يستلمه جاهزاً من الخارج (من Hilt)
Blocking	الدالة تُوَقِف الـ thread حتى تنتهي، مثل .execute() الذي ينتظر رد السيرفر

لمصطلحات أكثر: القاموس الكامل (+70 مصطلحاً) في الملف [Android_Test_Task_Guide.pdf](#).

الجزء الأول: إعدادات المشروع (Gradle)

Gradle هو نظام البناء في Android. يقرأ هذه الملفات ليعرف:

- ما المكتبات التي يحملها.
- كيف يبني التطبيق.

يقابل ملف `pubspec.yaml` في Flutter.

1. `gradle/libs.versions.toml`

الدور: كتالوج الإصدارات (Version Catalog). كل أسماء المكتبات وإصداراتها في مكان واحد.

gradle/libs.versions.toml

51 lines

```
1  [versions]
2  agp = "8.5.2"
3  kotlin = "2.0.20"
4  ksp = "2.0.20-1.0.25"
5  hilt = "2.52"
6  coreKtx = "1.13.1"
7  appcompat = "1.7.0"
8  material = "1.12.0"
9  constraintlayout = "2.1.4"
10 recyclerview = "1.3.2"
11 fragmentKtx = "1.8.3"
12 lifecycle = "2.8.5"
13 navigation = "2.8.0"
14 securityCrypto = "1.1.0-alpha06"
15 retrofit = "2.11.0"
16 okhttp = "4.12.0"
17 coil = "2.7.0"
18 coroutines = "1.8.1"
19 junit = "4.13.2"
20
21 [libraries]
22 androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }
23 androidx-appcompat = { group = "androidx.appcompat", name = "appcompat", version.ref =
"appcompat" }
24 material = { group = "com.google.android.material", name = "material", version.ref = "material"
}
25 androidx-constraintlayout = { group = "androidx.constraintlayout", name = "constraintlayout",
version.ref = "constraintlayout" }
26 androidx-recyclerview = { group = "androidx.recyclerview", name = "recyclerview", version.ref =
"recyclerview" }
27 androidx-fragment-ktx = { group = "androidx.fragment", name = "fragment-ktx", version.ref =
"fragmentKtx" }
28 androidx-lifecycle-viewmodel-ktx = { group = "androidx.lifecycle", name = "lifecycle-viewmodel-
ktx", version.ref = "lifecycle" }
29 androidx-lifecycle-runtime-ktx = { group = "androidx.lifecycle", name = "lifecycle-runtime-ktx",
version.ref = "lifecycle" }
30 androidx-navigation-fragment-ktx = { group = "androidx.navigation", name = "navigation-fragment-
ktx", version.ref = "navigation" }
```

```

31  androidx-navigation-ui-ktx = { group = "androidx.navigation", name = "navigation-ui-ktx",
version.ref = "navigation" }
32  androidx-security-crypto = { group = "androidx.security", name = "security-crypto", version.ref
= "securityCrypto" }
33
34  hilt-android = { group = "com.google.dagger", name = "hilt-android", version.ref = "hilt" }
35  hilt-compiler = { group = "com.google.dagger", name = "hilt-android-compiler", version.ref =
"hilt" }
36
37  retrofit = { group = "com.squareup.retrofit2", name = "retrofit", version.ref = "retrofit" }
38  retrofit-converter-gson = { group = "com.squareup.retrofit2", name = "converter-gson",
version.ref = "retrofit" }
39  okhttp = { group = "com.squareup.okhttp3", name = "okhttp", version.ref = "okhttp" }
40  okhttp-logging = { group = "com.squareup.okhttp3", name = "logging-interceptor", version.ref =
"okhttp" }
41  okhttp-mockwebserver = { group = "com.squareup.okhttp3", name = "mockwebserver", version.ref =
"okhttp" }
42  coil = { group = "io.coil-kt", name = "coil", version.ref = "coil" }
43  kotlin-coroutines-android = { group = "org.jetbrains.kotlin", name = "kotlin-coroutines-
android", version.ref = "coroutines" }
44  kotlin-coroutines-test = { group = "org.jetbrains.kotlin", name = "kotlin-coroutines-test",
version.ref = "coroutines" }
45  junit = { group = "junit", name = "junit", version.ref = "junit" }
46
47  [plugins]
48  android-application = { id = "com.android.application", version.ref = "agp" }
49  kotlin-android = { id = "org.jetbrains.kotlin.android", version.ref = "kotlin" }
50  ksp = { id = "com.google.devtools.ksp", version.ref = "ksp" }
51  hilt = { id = "com.google.dagger.hilt.android", version.ref = "hilt" }

```

الشرح

الملف مقسوم إلى ثلاثة أقسام:

القسم	ماذا يحتوي
[versions]	أرقام الإصدارات فقط، مثل <code>retrofit = "2.11.0"</code>
[libraries]	المكتبات. كل مكتبة لها <code>group</code> و <code>name</code> ، وتشير إلى رقم إصدارها عبر <code>version.ref</code>
[plugins]	الإضافات (Plugins) التي تغيّر طريقة البناء نفسها

لماذا هذا الأسلوب؟ عند تحديث مكتبة تغيّر رقماً واحداً فقط. وفي ملفات Gradle تكتب `libs.retrofit` بدل النص الطويل `com.squareup.retrofit2:retrofit:2.11.0`.

أهم المكتبات ولماذا اخترناها:

المكتبة	لماذا
<code>converter-gson</code> + <code>retrofit</code>	تعريف طلبات الـ API وتحويل JSON إلى كائنات Kotlin. مطلوبة في المهمة
<code>logging-interceptor</code> + <code>okhttp</code>	المحرك الذي ينفذ الطلبات، وفيه الـ Interceptors. مطلوبة في المهمة
<code>security-crypto</code>	التخزين المشفّر للـ token

المكتبة	لماذا
hilt	حقن الاعتماديات (Dependency Injection)
navigation-ui- / navigation-fragment-ktx ktx	التنقل بين الشاشات
recyclerview	القائمة. مطلوبة في المهمة
coil	تحميل صور المنتجات من الإنترنت
mockwebserver	سيرفر وهمي محلي للاختبارات

2. settings.gradle.kts

الدور: يحدد أين يبحث Gradle عن المكتبات، وما الوحدات (**modules**) الموجودة في المشروع.

settings.gradle.kts	18 lines
<pre> 1 pluginManagement { 2 repositories { 3 google() 4 mavenCentral() 5 gradlePluginPortal() 6 } 7 } 8 9 dependencyResolutionManagement { 10 repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS) 11 repositories { 12 google() 13 mavenCentral() 14 } 15 } 16 17 rootProject.name = "FakeStoreAuth" 18 include(":app") </pre>	

الشرح

- `pluginManagement.repositories`: المستودعات التي تُحمّل منها الإضافات (Plugins).
- `dependencyResolutionManagement`: المستودعات التي تُحمّل منها المكتبات:
 - `google()`: مكتبات Android.
 - `mavenCentral()`: باقي المكتبات مثل Retrofit و OkHttp.
- `FAIL_ON_PROJECT_REPOS`: يمنع أي وحدة من إضافة مستودعات خاصة بها، فتبقى كل المصادر في مكان واحد.
- `include(":app")`: المشروع فيه وحدة واحدة اسمها `app`.

3. build.gradle.kts (الجزر)

الدور: يعلن الإضافات على مستوى المشروع دون أن يطبقها.

build.gradle.kts

6 lines

```
1 plugins {
2     alias(libs.plugins.android.application) apply false
3     alias(libs.plugins.kotlin.android) apply false
4     alias(libs.plugins.ksp) apply false
5     alias(libs.plugins.hilt) apply false
6 }
```

الشرح

تعني: "حمل هذه الإضافة لكن لا تطبقها هنا". كل وحدة تطبق ما تحتاجه منها، وفي مشروعنا الوحدة الوحيدة هي `app`.

4. app/build.gradle.kts

الدور: ملف بناء التطبيق نفسه، وهو أهم ملفات Gradle.

app/build.gradle.kts

91 lines

```
1 plugins {
2     alias(libs.plugins.android.application)
3     alias(libs.plugins.kotlin.android)
4     alias(libs.plugins.ksp)
5     alias(libs.plugins.hilt)
6 }
7
8 // Dev switch (gradle.properties): answer API calls with a built-in mock server.
9 // Only debug builds can use it; release builds always talk to the real FakeStore.
10 val useMockBackend = providers.gradleProperty("fakestore.useMockBackend").orNull.toBoolean()
11
12 android {
13     namespace = "com.example.fakestore"
14     compileSdk = 34
15
16     defaultConfig {
17         applicationId = "com.example.fakestore"
18         minSdk = 24
19         targetSdk = 34
20         versionCode = 1
21         versionName = "1.0"
22     }
23
24     buildTypes {
25         debug {
26             buildConfigField("boolean", "USE MOCK BACKEND", useMockBackend.toString())
27         }
28         release {
29             buildConfigField("boolean", "USE MOCK BACKEND", "false")
30             isMinifyEnabled = false
31         }
32     }
33 }
```

```

31         proguardFiles(
32             getDefaultProguardFile("proguard-android-optimize.txt"),
33             "proguard-rules.pro",
34         )
35     }
36 }
37
38 compileOptions {
39     sourceCompatibility = JavaVersion.VERSION_17
40     targetCompatibility = JavaVersion.VERSION_17
41 }
42
43 kotlinOptions {
44     jvmTarget = "17"
45 }
46
47 buildFeatures {
48     viewBinding = true
49     buildConfig = true
50 }
51 }
52
53 dependencies {
54     // AndroidX + Material UI
55     implementation(libs.androidx.core.ktx)
56     implementation(libs.androidx.appcompat)
57     implementation(libs.material)
58     implementation(libs.androidx.constraintlayout)
59     implementation(libs.androidx.recyclerview)
60     implementation(libs.androidx.fragment.ktx)
61
62     // MVVM
63     implementation(libs.androidx.lifecycle.viewmodel.ktx)
64     implementation(libs.androidx.lifecycle.runtime.ktx)
65     implementation(libs.kotlinx.coroutines.android)
66
67     // Navigation between fragments
68     implementation(libs.androidx.navigation.fragment.ktx)
69     implementation(libs.androidx.navigation.ui.ktx)
70
71     // Secure token storage
72     implementation(libs.androidx.security.crypto)
73
74     // Networking
75     implementation(libs.retrofit)
76     implementation(libs.retrofit.converter.gson)
77     implementation(libs.okhttp)
78     implementation(libs.okhttp.logging)
79
80     // Image loading
81     implementation(libs.coil)
82
83     // Dependency injection
84     implementation(libs.hilt.android)
85     ksp(libs.hilt.compiler)
86
87     // Unit tests (JVM)
88     testImplementation(libs.junit)
89     testImplementation(libs.kotlinx.coroutines.test)
90     testImplementation(libs.okhttp.mockwebserver)
91 }

```

الشرح

الإضافات (plugins):

الإضافة	ماذا تفعل
<code>android.application</code>	تجعل الوحدة تطبيق Android قابلاً للتثبيت
<code>kotlin.android</code>	دعم لغة Kotlin
<code>ksp</code>	Kotlin Symbol Processing : يولّد كوداً وقت البناء، ويحتاجه Hilt
<code>hilt</code>	يفعّل Hilt

إعدادات `android { }`:

الإعداد	المعنى
<code>applicationId</code> / <code>namespace</code>	المعرّف الفريد للتطبيق
<code>compileSdk = 34</code>	تُترجم الكود مقابل Android 14
<code>minSdk = 24</code>	أقل نسخة مدعومة: Android 7.0
<code>targetSdk = 34</code>	النسخة التي اختُبر عليها التطبيق
<code>kotlinOptions</code> / <code>compileOptions</code>	استخدام Java 17

الميزات (`buildFeatures`):

- `viewBinding = true`: يولّد لكل ملف XML كلاساً يحمل مراجع عناصره، مثلاً `fragment_login.xml` ← `FragmentLoginBinding`. هذا يعني عن `findViewById` ويمنع أخطاء الأنواع والقيم الفارغة.
- `buildConfig = true`: يولّد كلاس `BuildConfig`. نستخدم منه `BuildConfig.DEBUG` لتفعيل سجل الشبكة في نسخة التطوير فقط.

مفتاح السيرفر الوهمي (`useMockBackend`):

- السطر `providers.gradleProperty("fakestore.useMockBackend")` يقرأ القيمة من `gradle.properties`.
- `buildConfigField(...)` يحوّلها إلى ثابت في الكود اسمه `BuildConfig.USE MOCK BACKEND`.
- في `debug` يأخذ قيمة المفتاح. وفي `release` يكون دائماً `false`، فلا يمكن أن تصل نسخة الإنتاج إلى السيرفر الوهمي بالخطأ.

أنواع الاعتماديات (`dependencies`):

النوع	متى يُستخدم
<code>implementation</code>	مكتبة يحتاجها التطبيق
<code>ksp(...)</code>	مولّد كود يعمل وقت البناء فقط
<code>testImplementation</code>	مكتبة للاختبارات فقط، ولا تدخل في التطبيق النهائي

5. gradle.properties

الدور: إعدادات عامة لـ Gradle.

gradle.properties

8 lines

```
1 org.gradle.jvmargs=-Xmx2048m -Dfile.encoding=UTF-8
2 android.useAndroidX=true
3 android.nonTransitiveRClass=true
4 kotlin.code.style=official
5
6 # Development only: answer API calls with a built-in mock server instead of fakestoreapi.com
7 # (useful while the real server is down). Release builds ignore it. Set to false to use the real
8 fakestore.useMockBackend=true
```

الشرح

الإعداد	المعنى
org.gradle.jvmargs	ذاكرة Gradle أثناء البناء
android.useAndroidX	استخدام مكتبات AndroidX الحديثة
android.nonTransitiveRClass	كل وحدة ترى موارد R الخاصة بها فقط، فيكون البناء أسرع
fakestore.useMockBackend	مفتاح الوضع التجريبي. true يعني أن نسخة التطوير تستخدم السيرفر الوهمي بدل FakeStore. اجعله false عندما يعود FakeStore للعمل. انظر الجزء التاسع

6. app/src/main/AndroidManifest.xml

الدور: بطاقة هوية التطبيق: الصلاحيات، والشاشات، والإعدادات العامة.

app/src/main/AndroidManifest.xml

28 lines

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <manifest xmlns:android="http://schemas.android.com/apk/res/android">
3
4     <uses-permission android:name="android.permission.INTERNET" />
5
6     <!--
7         allowBackup="false": the encrypted token must not be copied to backups. The
8         encryption key stays in this device's Keystore, so a restored copy could not
9         be decrypted anyway.
10    -->
11     <application
12         android:name=".FakeStoreApp"
13         android:allowBackup="false"
14         android:label="@string/app_name"
15         android:supportRtl="true"
16         android:theme="@style/Theme.FakeStore">
17
18         <activity
```



```

19         android:name=".MainActivity"
20         android:exported="true"
21         android:windowSoftInputMode="adjustResize">
22         <intent-filter>
23             <action android:name="android.intent.action.MAIN" />
24             <category android:name="android.intent.category.LAUNCHER" />
25         </intent-filter>
26     </activity>
27 </application>
28 </manifest>

```

الشرح

الصلاحيات والإعدادات:

- `INTERNET` : صلاحية الإنترنت. بدونها يفشل أي طلب شبكة.
- `android:name=".FakeStoreApp"` : كلاس الـ Application الذي يشغل Hilt.
- `android:allowBackup="false"` (**قرار أمني**): يمنع نسخ ملفات التطبيق، ومنها الـ token المشفر، إلى نسخ Google الاحتياطية. ومفتاح التشفير موجود في **Keystore** هذا الجهاز فقط، فالنسخة المستعادة على جهاز آخر لن تُفك أصلاً.
- `android:supportsRtl="true"` : دعم اللغات التي تُكتب من اليمين لليساار.

الشاشة:

- `MainActivity` : الشاشة الوحيدة (**Activity**).
- `exported="true"` مع `intent-filter` (MAIN + LAUNCHER): هذه الشاشة تظهر في قائمة التطبيقات وتفتح أولاً.
- `windowSoftInputMode="adjustResize"` : عند ظهور لوحة المفاتيح تقلص الشاشة، فلا تغطي حقول تسجيل الدخول.

الجزء الثاني: نماذج البيانات والـ APIs

7. data/model/Product.kt

الدور: شكل المنتج كما تراه الواجهة.

app/src/main/java/com/example/fakestore/data/model/Product.kt

15 lines

```
1 package com.example.fakestore.data.model
2
3 /**
4  * Product as the UI layer sees it: non-null, already cleaned up from the network DTO.
5  */
6 data class Product(
7     val id: Int,
8     val title: String,
9     val price: Double,
10    val description: String,
11    val category: String,
12    val imageUrl: String,
13    val rating: Double,
14    val ratingCount: Int,
15 )
```

الشرح

- `data class`: كلاس لحمل البيانات. يولد Kotlin له تلقائياً `equals` و `hashCode` و `toString` و `copy`.
- `equals` مهمة لاحقاً: يستخدمها `DiffUtil` في الـ `RecyclerView` ليعرف هل تغير المنتج.
- **كل الحقول غير قابلة لأن تكون null**: الواجهة لا تتعامل أبداً مع قيم فارغة. التنظيف يحدث في الطبقة التي تحت الواجهة (القسم 10).

8. data/remote/ApiConfig.kt

الدور: عنوان السيرفر الأساسي في مكان واحد.

app/src/main/java/com/example/fakestore/data/remote/ApiConfig.kt

5 lines

```
1 package com.example.fakestore.data.remote
2
3 object ApiConfig {
4     const val BASE_URL = "https://fakestoreapi.com/"
5 }
```

الشرح

- `object`: كائن وحيد (`Singleton`) لا يُنشأ منه نسخ.

- `const val`: ثابت معروف وقت الترجمة.
- **مهم:** العنوان ينتهي بـ `/`، وهذا شرط عند Retrofit لدمجه مع مسارات مثل `products`.

9. `data/remote/dto/AuthDtos.kt`

الدور: شكل JSON المرسل والمستقبل في تسجيل الدخول.

```
app/src/main/java/com/example/fakestore/data/remote/dto/AuthDtos.kt 12 lines

1 package com.example.fakestore.data.remote.dto
2
3 /** Body of `POST /auth/login`. */
4 data class LoginRequest(
5     val username: String,
6     val password: String,
7 )
8
9 /** Response of `POST /auth/login`. Nullable because Gson can leave fields null. */
10 data class LoginResponse(
11     val token: String?,
12 )
```

الشرح

DTO اختصار **Data Transfer Object**: كلاس يطابق شكل JSON حرفياً.

الكلاس	يتحول إلى
LoginRequest	<code>{"username": "...", "password": "..."}"</code>
LoginResponse	<code>{"token": "..."}"</code> يُقرأ منه

لماذا `token: String?` قابل لأن يكون null؟ مكتبة **Gson** لا تحترم قواعد Kotlin. إذا غاب الحقل من JSON تضع فيه `null` حتى لو كان النوع غير قابل لذلك، فينهار التطبيق لاحقاً في مكان غير متوقع. لذلك نعلنه `String?` ونتحقق منه بأنفسنا في `AuthManager`.

10. `data/remote/dto/ProductDto.kt`

الدور: شكل المنتج كما يأتي من السيرفر، ودالة تحويله إلى `Product`.

```
app/src/main/java/com/example/fakestore/data/remote/dto/ProductDto.kt 33 lines

1 package com.example.fakestore.data.remote.dto
2
3 import com.example.fakestore.data.model.Product
4
5 /**
6  * Product exactly as returned by the API. Every field is nullable because the server
7  * is outside our control; [toDomain] turns it into a safe [Product].
8  */
```

```

9  data class ProductDto(
10      val id: Int,
11      val title: String?,
12      val price: Double?,
13      val description: String?,
14      val category: String?,
15      val image: String?,
16      val rating: RatingDto?,
17  )
18
19  data class RatingDto(
20      val rate: Double?,
21      val count: Int?,
22  )
23
24  fun ProductDto.toDomain(): Product = Product(
25      id = id,
26      title = title.orEmpty(),
27      price = price ?: 0.0,
28      description = description.orEmpty(),
29      category = category.orEmpty(),
30      imageUrl = image.orEmpty(),
31      rating = rating?.rate ?: 0.0,
32      ratingCount = rating?.count ?: 0,
33  )

```

الشرح

- كل الحقول قابلة لأن تكون null، لأن السيرفر خارج سيطرتنا.
- دالة امتداد (**Extension Function**): تضيف دالة لكلاس موجود دون تعديله. تحوّل الـ DTO إلى `Product` نظيف:
 - `orEmpty()`: يحوّل `null` إلى نص فارغ.
 - `?:` (عامل Elvis): "إن كانت القيمة null فاستخدم البديل".
- اسم الحقل في JSON هو `image`، وعندنا `imageUrl`. هذا الفصل يحمي الواجهة من تغييرات السيرفر.

لماذا كلاسان (Model و DTO)؟ إذا غيّر السيرفر شكل الرد، نعدّل `ProductDto` و `toDomain()` فقط، والواجهة لا تتأثر.

11. data/remote/AuthApi.kt

الدور: تعريف طلب تسجيل الدخول.

app/src/main/java/com/example/fakestore/data/remote/AuthApi.kt

22 lines

```

1  package com.example.fakestore.data.remote
2
3  import com.example.fakestore.data.remote.dto.LoginRequest
4  import com.example.fakestore.data.remote.dto.LoginResponse
5  import retrofit2.Call
6  import retrofit2.http.Body
7  import retrofit2.http.POST
8
9  /**

```

```

10  * Authentication endpoint.
11  *
12  * Returns [Call] (not a suspend function) because the token refresh runs inside
13  * an OkHttp interceptor, which is synchronous and must block until the new token arrives.
14  *
15  * IMPORTANT: this API is built on an OkHttp client WITHOUT the AuthInterceptor,
16  * otherwise a refresh would trigger another refresh forever.
17  */
18  interface AuthApi {
19
20      @POST("auth/login")
21      fun login(@Body body: LoginRequest): Call<LoginResponse>
22  }

```

الشرح

- `@POST("auth/login")` : طلب POST إلى `https://fakestoreapi.com/auth/login`
- `@Body` : يحوّل `LoginRequest` إلى JSON ويضعه في جسم الطلب.

لماذا ترجع `Call<LoginResponse>` وليس دالة `suspend` ؟ هذا قرار مقصود:

- التجديد يحدث داخل الـ `Interceptor`.
- الـ `Interceptor` دالة عادية متزامنة (**blocking**) تعمل على thread خاص بـ `OkHttp`، وليست `coroutine`.
- لذلك نحتاج طلباً ننفّذه وننتظر نتيجته مباشرة بـ `call.execute()`.

تحذير مكتوب في الكود: هذا الـ API يُبنى على عميل `OkHttp` **بدون** `AuthInterceptor`. لو مرّ طلب الـ `login` بالـ `Interceptor`، فسيطلب الـ `token` `Interceptor` صالحاً، فيبدأ تجديداً، والتجديد يستدعي `login` من جديد... **حلقة لا نهائية**.

12. `data/remote/ProductApi.kt`

الدور: طلبات المنتجات "المحمية".

app/src/main/java/com/example/fakestore/data/remote/ProductApi.kt

18 lines

```

1  package com.example.fakestore.data.remote
2
3  import com.example.fakestore.data.remote.dto.ProductDto
4  import retrofit2.http.GET
5  import retrofit2.http.Path
6
7  /**
8   * Protected product endpoints. Built on the authenticated OkHttp client, so every call
9   * automatically carries `Authorization: Bearer <token>`.
10  */
11  interface ProductApi {
12
13      @GET("products")
14      suspend fun getProducts(): List<ProductDto>
15
16      @GET("products/{id}")
17      suspend fun getProduct(@Path("id") id: Int): ProductDto
18  }

```

الشرح

- `@GET("products")`: يجلب قائمة المنتجات.
 - `@GET("products/{id}")` مع `@Path("id")`: يضع رقم المنتج داخل المسار، مثل `products/5`.
 - `suspend`: هنا مناسبة، لأن الـ `ViewModels` تستدعي هذه الدوال من `coroutines`.
 - **لا يوجد أي ذكر للـ token هنا.** الـ `header` يُضاف تلقائياً بواسطة `AuthInterceptor`، لأن هذا الـ `API` مبني على العميل المحمي (القسم 26). هذا بالضبط معنى طلب المهمة: `"Attach Authorization: Bearer to all product requests"`.
-

الجزء الثالث ★: منطق ال Token (قلب المهمة)

كل ملفات هذا الجزء في المجلد `data/auth/` ، وتعمل معاً هكذا:

الدور	الملف
يعطي الوقت الحالي	<code>Clock</code>
ما نحفظه: ال token. و <code>tokenSavedAt</code> ، وبيانات الدخول	<code>AuthSession</code>
واجهة التخزين، وتنفيذها <code>EncryptedTokenStorage</code> المشفّر	<code>TokenStorage</code>
العقل: الدخول، والانتها، والتجديد، والقفل، والخروج	★ <code>AuthManager</code>
يضيف <code>Bearer</code> لكل طلب، ويطلب token صالحاً من <code>AuthManager</code>	<code>AuthInterceptor</code>
شبكة أمان عند رد 401	<code>TokenAuthenticator</code>
أدوات مساعدة	<code>AuthHeaders</code> / <code>AuthEvent</code> / <code>AuthExceptions</code>

13. `data/auth/Clock.kt`

الدور: مصدر الوقت الحالي.

app/src/main/java/com/example/fakestore/data/auth/Clock.kt

15 lines

```
1 package com.example.fakestore.data.auth
2
3 import javax.inject.Inject
4
5 /**
6  * Source of the current time. Injected so tests can move time forward instantly
7  * instead of waiting 60 real seconds for a token to expire.
8  */
9 fun interface Clock {
10     fun nowMillis(): Long
11 }
12
13 class SystemTimeClock @Inject constructor() : Clock {
14     override fun nowMillis(): Long = System.currentTimeMillis()
15 }
```

الشرح

- `fun interface` : واجهة فيها دالة واحدة فقط، تعيد الوقت بالميلي ثانية.
- `SystemTimeClock` : التنفيذ الحقيقي، يستخدم `System.currentTimeMillis()`.
- `@Inject constructor()` : يخبر Hilt أنه يستطيع إنشاء هذا الكلاس بنفسه.

لماذا لا نستخدم `System.currentTimeMillis()` مباشرة؟ حتى نختبر الكود. في الاختبارات نستبدل الساعة بـ `FakeClock` ونقدّم الوقت 61 ثانية فوراً، بدل الانتظار دقيقة كاملة في كل اختبار. هذا مثال عملي على فائدة حقن الاعتماديات (**Dependency Injection**).

لماذا `currentTimeMillis` وليس `SystemClock.elapsedRealtime` ؟ لأن `tokenSavedAt` يُحفظ على القرص ويبقى بعد إعادة تشغيل الجهاز، و `elapsedRealtime` يعود إلى الصفر عند إعادة التشغيل.

14. `data/auth/AuthSession.kt`

الدور: البيانات التي نحفظها بعد تسجيل الدخول.

app/src/main/java/com/example/fakestore/data/auth/AuthSession.kt

26 lines

```
1 package com.example.fakestore.data.auth
2
3 /**
4  * Username and password, kept only to simulate a token refresh (FakeStore has no
5  * refresh endpoint). A real backend would issue a refresh token instead.
6  */
7 data class Credentials(
8     val username: String,
9     val password: String,
10 ) {
11     override fun toString(): String = "Credentials(username=$username, password=***)"
12 }
13
14 /**
15  * Everything we persist after a successful login.
16  *
17  * @property savedAtMillis the `tokenSavedAt` timestamp used to compute expiry.
18  */
19 data class AuthSession(
20     val token: String,
21     val savedAtMillis: Long,
22     val credentials: Credentials,
23 ) {
24     override fun toString(): String =
25         "AuthSession(token=***, savedAtMillis=$savedAtMillis, credentials=$credentials)"
26 }
```

الشرح

Credentials : اسم المستخدم وكلمة المرور.

- نحتاجهما لأن **التجديد = تسجيل دخول جديد**، ف FakeStore لا يملك endpoint للتجديد.

AuthSession :

- **token** : الرمز نفسه.
- **savedAtMillis** : هو **tokenSavedAt** المطلوب في المهمة.
- **credentials** : بيانات الدخول.

تفصيل أمني مهم: `override fun toString()`

- `data class` يطبع كل حقوله تلقائياً عند الطباعة.
- لو كتب أحد `Log.d("x", session.toString())` لظهرت كلمة المرور وال token في Logcat.
- أعدنا تعريف `toString` لتطبع `**بدلاً منهما`.

15. data/auth/TokenStorage.kt

الدور: واجهة (`interface`) للتخزين: اقرأ، احفظ، امسح.

app/src/main/java/com/example/fakestore/data/auth/TokenStorage.kt

11 lines

```
1 package com.example.fakestore.data.auth
2
3 /**
4  * Persistence for the current [AuthSession]. Implementations must be thread-safe:
5  * several OkHttp threads may read while a refresh writes.
6  */
7 interface TokenStorage {
8     fun read(): AuthSession?
9     fun save(session: AuthSession)
10    fun clear()
11 }
```

الشرح

لماذا interface؟ يوجد تنفيذان:

التنفيذ	أين يُستخدم
EncryptedTokenStorage	في التطبيق: مشفر وعلى القرص
InMemoryTokenStorage	في الاختبارات: في الذاكرة فقط

و `AuthManager` لا يعرف ولا يهتم أيهما يستخدم.

شرط أمان الخيوط (Thread-safety): التطبيق يجب أن يكون آمناً عند وصول عدة threads في نفس اللحظة. OkHttp يقرأ من عدة threads، والتجديد يكتب في نفس الوقت.

16. data/auth/EncryptedTokenStorage.kt

الدور: تحقيق متطلب "Securely store the token".

app/src/main/java/com/example/fakestore/data/auth/EncryptedTokenStorage.kt

69 lines

```
1 package com.example.fakestore.data.auth
2
3 import android.content.Context
4 import android.content.SharedPreferences
5 import androidx.security.crypto.EncryptedSharedPreferences
6 import androidx.security.crypto.MasterKey
7 import dagger.hilt.android.qualifiers.ApplicationContext
```

```

8 import javax.inject.Inject
9 import javax.inject.Singleton
10
11 /**
12  * [TokenStorage] backed by EncryptedSharedPreferences: keys and values are encrypted
13  * with AES-256, and the master key lives in the Android Keystore, so it never leaves the
device.
14  *
15  * read/save/clear are @Synchronized: a session is four separate keys, and without the
16  * lock a reader could see a half-written session (e.g. the old token with the new
tokenSavedAt).
17  */
18 @Singleton
19 class EncryptedTokenStorage @Inject constructor(
20     @ApplicationContext private val context: Context,
21 ) : TokenStorage {
22
23     private val prefs: SharedPreferences by lazy {
24         val masterKey = MasterKey.Builder(context)
25             .setKeyScheme(MasterKey.KeyScheme.AES256_GCM)
26             .build()
27
28         EncryptedSharedPreferences.create(
29             context,
30             FILE_NAME,
31             masterKey,
32             EncryptedSharedPreferences.PrefKeyEncryptionScheme.AES256_SIV,
33             EncryptedSharedPreferences.PrefValueEncryptionScheme.AES256_GCM,
34         )
35     }
36
37     @Synchronized
38     override fun read(): AuthSession? {
39         val token = prefs.getString(KEY_TOKEN, null) ?: return null
40         val username = prefs.getString(KEY_USERNAME, null) ?: return null
41         val password = prefs.getString(KEY_PASSWORD, null) ?: return null
42         val savedAt = prefs.getLong(KEY_SAVED_AT, 0L)
43         return AuthSession(token, savedAt, Credentials(username, password))
44     }
45
46     @Synchronized
47     override fun save(session: AuthSession) {
48         // commit() writes synchronously; we are always on a background thread here.
49         prefs.edit()
50             .putString(KEY_TOKEN, session.token)
51             .putLong(KEY_SAVED_AT, session.savedAtMillis)
52             .putString(KEY_USERNAME, session.credentials.username)
53             .putString(KEY_PASSWORD, session.credentials.password)
54             .commit()
55     }
56
57     @Synchronized
58     override fun clear() {
59         prefs.edit().clear().commit()
60     }
61
62     private companion object {
63         const val FILE_NAME = "secure_auth_prefs"
64         const val KEY_TOKEN = "token"
65         const val KEY_SAVED_AT = "token_saved_at"
66         const val KEY_USERNAME = "username"
67         const val KEY_PASSWORD = "password"

```

```
68 }
69 }
```

الشرح

MasterKey: المفتاح الرئيسي للتشفير.

- AES256_GCM**: خوارزمية تشفير قوية.

- المفتاح يُنشأ ويُحفظ داخل **Android Keystore**، وهي خزانة في نظام Android لا يمكن إخراج المفاتيح منها، ولا حتى للتطبيق نفسه. التطبيق يطلب منها "شفر" و"فك" فقط.

EncryptedSharedPreferences.create(...): نسخة مشفرة من **SharedPreferences**:

- AES256_SIV**: يشفر أسماء المفاتيح، فلا يُعرف حتى أن هناك حقلاً اسمه "token".

- AES256_GCM**: يشفر القيم.

by lazy: يُنشأ الكائن عند أول استخدام فقط، وليس عند إنشاء الكلاس، لأن إنشاءه يتعامل مع الـ **Keystore** وهو مكلف نسبياً.

read():

- يقرأ الحقول الأربعة.

- إذا غاب أي منها يُرجع **null**، أي لا توجد جلسة.

- return null**: إن كانت القيمة **null** فخرج من الدالة فوراً.

save():

- يكتب الحقول الأربعة دفعة واحدة.

- يستخدم **commit()**: يكتب على القرص **وينتظر** حتى ينتهي.

- البديل **apply()** يكتب في الخلفية. اخترنا **commit** لأننا دائماً على **thread** خلفي، فلا مشكلة في الانتظار، ونريد ضمان الحفظ.

clear(): يسمح كل شيء، ويُستخدم عند تسجيل الخروج.

@Synchronized على الدوال الثلاث (أضيف في المراجعة):

- المشكلة**: الجلسة مخزنة في أربعة مفاتيح منفصلة، و **read()** يقرأها واحداً تلو الآخر.

- لو كتب **thread** التجديد جلسة جديدة **بين** قراءة **token** وقراءة **tokenSavedAt**، لحصل القارئ على خليط: **ال token القديم مع وقت الحفظ الجديد**. فيظن أن **ال token** القديم المنتهي صالح لدقيقة كاملة.

- الحل**: **@Synchronized** يجعل القراءة والكتابة والمسح **لا تتداخل أبداً**. كل قارئ يرى الجلسة كاملة قبل الكتابة أو بعدها، وليس في منتصفها.

- هذا قفل ثالث، منفصل عن أقفال **AuthManager**، ويُمسك لأجزاء من الثانية فقط، فلا يبطئ شيئاً.

- في المقابلة**: هذا مثال على "عملية مركبة" (**compound operation**). كل قراءة وحدها آمنة، لكن مجموعها ليس آمناً بلا قفل.

ملاحظة للمقابلة: Google أوقفت تطوير مكتبة **security-crypto** (أصبحت **deprecated**). البديل الحديث هو **DataStore** مع تشفير بمفتاح من **Keystore** (مثلاً بمكتبة **Tink**). وبفضل **TokenStorage** كـ **interface**، يكفي كتابة

17. `data/auth/AuthExceptions.kt`

الدور: أنواع الأخطاء الخاصة بالمصادقة.

app/src/main/java/com/example/fakestore/data/auth/AuthExceptions.kt

24 lines

```
1 package com.example.fakestore.data.auth
2
3 import java.io.IOException
4
5 /**
6  * Both exceptions extend IOException on purpose: they can be thrown from inside an
7  * OkHttp interceptor, and OkHttp only reports IOExceptions as normal call failures.
8  * Any other exception type thrown there would crash the app.
9  */
10
11 /** The user is not logged in anymore (no session, or the refresh was rejected). */
12 class SessionExpiredException(cause: Throwable? = null) :
13     IOException("Session expired. Please log in again.", cause)
14
15 /** The server rejected the username/password (HTTP 4xx from /auth/login). */
16 class InvalidCredentialsException(val httpCode: Int) :
17     IOException("Invalid username or password (HTTP $httpCode).")
18
19 /**
20  * The server answered with an error that is not the user's fault (HTTP 5xx, e.g. the
21  * Cloudflare 52x pages shown when FakeStore is down). The session is kept: retry later.
22  */
23 class ServerUnavailableException(val httpCode: Int) :
24     IOException("Server is unavailable (HTTP $httpCode).")
```

الشرح

SessionExpiredException : لا توجد جلسة، أو السيرفر رفض التجديد. النتيجة: يجب تسجيل الدخول من جديد.

InvalidCredentialsException : السيرفر رفض اسم المستخدم أو كلمة المرور، أي رد بكود **4xx** مثل 401. نحتفظ بالكود في `httpCode`.

ServerUnavailableException : السيرفر نفسه متعطل، أي رد بكود **5xx**. مثاله صفحات Cloudflare من نوع 52x التي تظهر عندما يتوقف FakeStore. هذا ليس خطأ المستخدم، **فلا نُخرجه**، ونعرض له رسالة "السيرفر غير متاح حالياً، حاول لاحقاً".

⚠ **لماذا يرثان من `IOException` ؟** هذه نقطة تقنية دقيقة ومهمة:

- هذه الأخطاء قد تُرمى من داخل الـ **Interceptor**.
- `OkHttp` يتعامل مع `IOException` كفشل طبيعي للطلب، فيمرره لـ `Retrofit` ثم لـ `ViewModel`، فيظهر للمستخدم كرسالة خطأ.
- **أي نوع آخر (مثل `IllegalStateException`)** يُعتبر خطأً برمجياً، فينهار التطبيق كاملاً (**crash**) لأنه يُرمى على `thread` خاص بـ `OkHttp`.

18. data/auth/AuthEvent.kt

الدور: أحداث لمرة واحدة يرسلها AuthManager للواجهة.

app/src/main/java/com/example/fakestore/data/auth/AuthEvent.kt

7 lines

```
1 package com.example.fakestore.data.auth
2
3 /** One-time events emitted by [AuthManager] and observed by the UI. */
4 sealed interface AuthEvent {
5     /** Refresh failed and the session was wiped: the UI must return to the login screen. */
6     data object SessionExpired : AuthEvent
7 }
```

الشرح

- `sealed interface`: مجموعة مغلقة من الأنواع. حالياً نوع واحد هو `SessionExpired` ، ويمكن إضافة أنواع لاحقاً.
- `data object`: كائن وحيد لا يحمل بيانات.

لماذا "حدث" (Event) وليس "حالة" (State)؟

- تسجيل الخروج الإجباري شيء يحدث مرة واحدة: نعرض رسالة ثم نتقل لشاشة Login.
- لو كان "حالة" لبقى محفوظاً، ولتكرر الانتقال عند كل تدوير للشاشة.

19. data/auth/AuthHeaders.kt

الدور: ثوابت ودوال لبناء header الـ Authorization وقراءته.

app/src/main/java/com/example/fakestore/data/auth/AuthHeaders.kt

12 lines

```
1 package com.example.fakestore.data.auth
2
3 object AuthHeaders {
4     const val AUTHORIZATION = "Authorization"
5     private const val BEARER_PREFIX = "Bearer "
6
7     fun bearer(token: String): String = BEARER_PREFIX + token
8
9     /** Extracts the token from `Bearer <token>`, or null if the header is missing. */
10    fun tokenFrom(headerValue: String?): String? =
11        headerValue?.takeIf { it.startsWith(BEARER_PREFIX) }?.removePrefix(BEARER_PREFIX)
12 }
```

الشرح

- `bearer(token)`: يبني النص `Bearer abc123`.
- `tokenFrom(header)`: العكس، يستخرج `abc123` من `Bearer abc123`. يستخدمه `TokenAuthenticator` ليعرف أي token رفضه السيرفر.
- `takeIf { ... }`: يُرجع القيمة إذا تحقق الشرط، وإلا يُرجع `null`.

لماذا ملف خاص لهذه الثوابت؟ حتى لا يُكتب النص "Authorization" و "Bearer" في عدة أماكن. خطأ إملائي واحد، مثل نسيان المسافة بعد Bearer، يكسر المصادقة كلها.

20. ★★ data/auth/AuthManager.kt

الدور: أهم ملف في المهمة. المسؤول الوحيد عن حالة المصادقة.

app/src/main/java/com/example/fakestore/data/auth/AuthManager.kt

163 lines

```
1 package com.example.fakestore.data.auth
2
3 import com.example.fakestore.data.remote.AuthApi
4 import com.example.fakestore.data.remote.dto.LoginRequest
5 import kotlinx.coroutines.Dispatchers
6 import kotlinx.coroutines.flow.MutableSharedFlow
7 import kotlinx.coroutines.flow.SharedFlow
8 import kotlinx.coroutines.flow.asSharedFlow
9 import kotlinx.coroutines.withContext
10 import java.io.IOException
11 import javax.inject.Inject
12 import javax.inject.Singleton
13
14 /**
15  * Single owner of the authentication state: login, token expiry, refresh and logout.
16  *
17  * Token rules (simulated, because FakeStore tokens never really expire):
18  * - A token is valid for [TOKEN_LIFETIME_MS] after it was saved (`tokenSavedAt`).
19  * - An expired token is never handed out: it is refreshed first.
20  * - Refresh = calling `/auth/login` again with the stored credentials.
21  * - Concurrent callers share ONE refresh: the first one refreshes while the others
22  *   wait on [refreshLock], then reuse the new token (double-checked locking).
23  * - If the server rejects the refresh, the user is logged out ([AuthEvent.SessionExpired]).
24  */
25 @Singleton
26 class AuthManager @Inject constructor(
27     private val authApi: AuthApi,
28     private val storage: TokenStorage,
29     private val clock: Clock,
30 ) {
31
32     /** Serializes refreshes: only one thread at a time may request a new token. */
33     private val refreshLock = Any()
34
35     /**
36      * Guards writes to [storage] together with [sessionGeneration]. Held only briefly
37      * (never during network calls), so logout() on the main thread never waits for a refresh.
38      */
39     private val storageLock = Any()
40
41     /** Incremented on every logout, so a refresh that started before it cannot revive the
42      session. */
43     private var sessionGeneration = 0
44
45     private val _events = MutableSharedFlow<AuthEvent>(extraBufferCapacity = 1)
46     val events: SharedFlow<AuthEvent> = _events.asSharedFlow()
47
48     fun isLoggedIn(): Boolean = storage.read() != null
```

```

48
49  /**
50   * Logs in from the UI and stores the new session.
51   *
52   * @throws InvalidCredentialsException if the server rejects the username/password.
53   * @throws IOException on network or server errors.
54   */
55  suspend fun login(username: String, password: String) {
56      withContext(Dispatchers.IO) {
57          val generation = currentGeneration()
58          val session = requestNewSession(Credentials(username, password))
59          saveIfStillCurrent(generation, session)
60      }
61  }
62
63  /**
64   * Returns a token that is safe to send right now, refreshing it first if it expired.
65   *
66   * Blocking: called by [AuthInterceptor] on an OkHttp background thread.
67   *
68   * @throws SessionExpiredException if there is no session or the refresh was rejected.
69   * @throws IOException if the refresh failed because of the network (session is kept).
70   */
71  fun getValidToken(): String {
72      // Fast path: no locking when the token is still valid (the common case).
73      storage.read()?.takeUnless { it.isExpired() }?.let { return it.token }
74
75      synchronized(refreshLock) {
76          // Second check: while we waited for the lock, another thread may have refreshed.
77          val session = storage.read() ?: throw SessionExpiredException()
78          if (!session.isExpired()) return session.token
79          return refresh(session.credentials)
80      }
81  }
82
83  /**
84   * Called by [TokenAuthenticator] when the server answered 401 to [rejectedToken].
85   *
86   * @return a token to retry with, or null to give up.
87   */
88  fun refreshAfterUnauthorized(rejectedToken: String?): String? {
89      synchronized(refreshLock) {
90          val session = storage.read() ?: return null
91          // Another request already replaced the rejected token: just retry with the new one.
92          if (session.token != rejectedToken) return session.token
93          return try {
94              refresh(session.credentials)
95          } catch (e: IOException) {
96              null
97          }
98      }
99  }
100
101  /** User-initiated logout. */
102  fun logout() {
103      synchronized(storageLock) {
104          sessionGeneration++
105          storage.clear()
106      }
107  }
108
109  /** Logout caused by a failed refresh: clears the session and tells the UI. */

```

```

110 fun forceLogout() {
111     logout()
112     _events.tryEmit(AuthEvent.SessionExpired)
113 }
114
115 /** Must be called while holding [refreshLock]. */
116 private fun refresh(credentials: Credentials): String {
117     val generation = currentGeneration()
118     val newSession = try {
119         requestNewSession(credentials)
120     } catch (e: InvalidCredentialsException) {
121         // The server refused our credentials: retrying would fail forever, so log out.
122         forceLogout()
123         throw SessionExpiredException(e)
124     }
125     saveIfStillCurrent(generation, newSession)
126     return newSession.token
127 }
128
129 /** Blocking network call to `/auth/login`. */
130 private fun requestNewSession(credentials: Credentials): AuthSession {
131     val response = authApi
132         .login(LoginRequest(credentials.username, credentials.password))
133         .execute()
134
135     if (response.code() in 400..499) throw InvalidCredentialsException(response.code())
136     if (!response.isSuccessful) throw ServerUnavailableException(response.code())
137
138     val token = response.body()?.token
139     if (token.isNullOrEmpty()) throw IOException("Login response did not contain a token")
140
141     return AuthSession(token, clock.nowMillis(), credentials)
142 }
143
144 private fun currentGeneration(): Int = synchronized(storageLock) { sessionGeneration }
145
146 /** Saves [session] unless a logout happened since [generation] was read. */
147 private fun saveIfStillCurrent(generation: Int, session: AuthSession) {
148     synchronized(storageLock) {
149         if (generation != sessionGeneration) throw SessionExpiredException()
150         storage.save(session)
151     }
152 }
153
154 private fun AuthSession.isExpired(): Boolean {
155     val age = clock.nowMillis() - savedAtMillis
156     // A negative age means the device clock moved backwards: don't trust the token.
157     return age < 0 || age ≥ TOKEN_LIFETIME_MS
158 }
159
160 companion object {
161     const val TOKEN_LIFETIME_MS = 60_000L
162 }
163 }

```


الشرح التفصيلي

أ) الإعلان والاعتماديات (الأسطر 25-30)

- `@Singleton` : نسخة واحدة فقط في التطبيق كله. ضروري، لأن القفل يجب أن يكون واحداً لكل الطلبات. لو وُجدت نسختان لكان لكل منهما قفلها، ولفشل منع التجديد المكرر.
- يحصل على 3 أشياء من الخارج (حقن الاعتماديات):
 - `loginApi` : لإرسال `login`.
 - `storage` : للحفظ.
 - `clock` : للوقت.

ب) الأقفال والحالة (الأسطر 33-45)

- `refreshLock` : القفل الأساسي.
- يضمن أن `thread` واحد فقط يجدد `token` الـ مرة الواحدة.
- `Any()` كائن فارغ نستخدمه كمفتاح للقفل.
- `storageLock` و `sessionGeneration` : لحل مشكلة دقيقة:
- السيناريو: بدأ تجديد، والمستخدم ضغط Logout أثناء انتظار رد السيرفر، ثم وصل الرد بـ `token` جديد.
- بدون حماية: يُحفظ `token` الجديد بعد الخروج، فيعود المستخدم مسجلاً للدخول رغم أنه خرج!
- الحل: رقم "جيل" (`sessionGeneration`) يزيد بواحد عند كل Logout.
- 1. قبل التجديد نحفظ رقم الجيل الحالي.
- 2. بعد الرد نقارنه بالرقم الحالي.
- 3. إذا تغيّر فهذا يعني أن خروجاً حدث، فلا نحفظ.
- `storageLock` يُمسك لجزء صغير جداً من الثانية فقط (بلا أي طلب شبكة)، فلا تنتظر الواجهة عند Logout.
- `_events` و `events` :
- `MutableSharedFlow` : قناة بث للأحداث. خاصة وقابلة للإرسال.
- `events` : النسخة العامة، للقراءة فقط. `MainActivity` تستمع لها.
- `extraBufferCapacity = 1` : مكان لحدث واحد ينتظر، فلا يضيع إذا أرسل والمستمع مشغول.

ج) `isLoggedIn()` (السطر 47)

هل توجد جلسة محفوظة؟ نستخدمها `MainActivity` لتقرر أي شاشة تبدأ بها.

د) `login()` : تسجيل الدخول من الشاشة (الأسطر 55-61)

- دالة `suspend` ، تستدعيها `LoginViewModel` من `coroutine`.
- `withContext(Dispatchers.IO)` : ينقل التنفيذ إلى `thread` مخصص لعمليات الشبكة والقرص، لأن `requestNewSession` تنتظر الرد (blocking). لو نُفذت على `Main thread` لتجمّد التطبيق، ورمى Android الخطأ `NetworkOnMainThreadException`.
- الخطوات: نأخذ رقم الجيل، ثم نطلب جلسة جديدة، ثم نحفظها إن لم يحدث خروج في الأثناء.

★ (هـ) `getValidToken()` : الدالة الأهم (الأسطر 71-81)

تُستدعى من `AuthInterceptor` قبل كل طلب منتجات. تمر بثلاث مراحل:

المرحلة 1: المسار السريع (السطر 73)، بلا قفل

```
storage.read()?.takeUnless { it.isExpired() }?.let { return it.token }
```

اقرأ الجلسة. إذا لم تكن منتهية فأرجع الـ token فوراً.

- `{ شرط } takeUnless` : يُرجع القيمة إلا إذا تحقق الشرط.
- هذه الحالة الأكثر شيوعاً (الـ token صالح)، فلا نريد أن تنتظر الطلبات في طابور القفل بلا داع.

المرحلة 2: دخول القفل (السطر 75) `{ ... } synchronized(refreshLock)` : هنا يقف الطابور. أول thread يدخل، والبقية ينتظرون خارج القفل حتى يخرج.

المرحلة 3: التحقق الثاني داخل القفل (الأسطر 77-79)، سر الحل

- السطر 77: نقرأ الجلسة **من جديد**. إذا لم توجد (خرج المستخدم) نرمي `SessionExpiredException`.
- السطر 78: إذا أصبح الـ token صالحاً الآن نرجعه دون تجديد. هذا يعني أن thread آخر دخل القفل قبلنا وجّده بينما كنا ننتظر.
- السطر 79: ما زال منتهياً، فنحن أول من وصل، فنجدّد.

مثال بثلاثة طلبات A و B و C والـ token منتهٍ:

اللحظة	A	B	C
1	المسار السريع: منتهٍ	المسار السريع: منتهٍ	المسار السريع: منتهٍ
2	يدخل القفل 🚫	ينتظر ⌚	ينتظر ⌚
3	التحقق الثاني: منتهٍ، فيجدّد (login)	ينتظر ⌚	ينتظر ⌚
4	يحفظ الجديد ويخرج 🚪	يدخل، والتحقق الثاني: صالح!	ينتظر ⌚
5	يرسل طلبه ✅	يخرج ويرسل ✅	يدخل، صالح! فيرسل ✅

النتيجة: **login واحد فقط**. أثبتنا ذلك في الاختبارات (القسم 51): بدون السطر 78، أنتجت 10 طلبات متزامنة 10 طلبات login.

لماذا `synchronized` وليس `Mutex` من `coroutines`؟ الـ `Interceptor` ليس `coroutine`، بل دالة عادية تعمل على thread حقيقي من `OkHttp`، و `synchronized` هو القفل المناسب للـ threads الحقيقية. `Mutex` يحتاج دالة `suspend`، فكنا سنضطر إلى `runBlocking`، وهو أقل وضوحاً.

و) `refreshAfterUnauthorized()` (الأسطر 88-99)

يستدعيها `TokenAuthenticator` عندما يرد السيرفر 401:

- السطر 90: لا توجد جلسة، فنرجع `null`، أي استسلم.
- السطر 92: إذا كان الـ token المحفوظ **يختلف** عن الذي رُفض، فطلب آخر جدّده قبلنا. نرجع الجديد دون تجديد إضافي.
- غير ذلك: نجدّد. وإذا فشل التجديد نرجع `null`.

ز) logout() و forceLogout() (الأسطر 102-113)

logout() : خروج عادي بطلب المستخدم.

- يزيد رقم الجيل بواحد، ثم يمسح التخزين.
- كلاهما داخل storageLock ليحدثا معاً كخطوة واحدة.

forceLogout() : خروج إجباري.

- يفعل نفس الشيء، ثم يرسل حدث SessionExpired للواجهة.
- tryEmit : يرسل دون انتظار، لأننا قد نكون على thread خاص بـ OkHttp.

ح) refresh() (الأسطر 116-127)

- تُستدعى فقط وهي ممسكة بـ refreshLock .
- تطلب جلسة جديدة بنفس بيانات الدخول المحفوظة.
- إذا رفض السيرفر بيانات الدخول (InvalidCredentialsException):

1. تسجيل خروج إجباري (forceLogout).
 2. نرمي SessionExpiredException .
 3. لا نعيد المحاولة، فالمحاولة الثانية ستفشل أيضاً، وإعادتها تعني حلقة لا نهائية. هذا تنفيذ لتحذير المهمة: "Avoid infinite retry loops" و "If refresh fails → force logout".
- إذا كان الفشل بسبب الشبكة (IOException): الخطأ يمر للطلب كما هو، والجلسة تبقى. انقطاع الإنترنت لحظياً لا يجب أن يُخرج المستخدم، والمحاولة التالية ستجدد عندما يعود الإنترنت.

ط) requestNewSession() (الأسطر 130-142)

الطلب الفعلي إلى /auth/login :

- execute() : ينفذ الطلب وينتظر الرد (blocking).
- السطر 135: كود من 400 إلى 499، أي أن السيرفر رفض البيانات، فنرمي InvalidCredentialsException .
- السطر 136: أي فشل آخر (مثل 500 أو 521، أي عطل في السيرفر) نرمي ServerUnavailableException . ليس ذنب المستخدم، فلا نُخرجه، وتعرض له الشاشة رسالة "السيرفر غير متاح حالياً".
- السطر 139: نجاح لكن بلا token (رد غريب) نرمي IOException أيضاً.
- السطر 141: ننشئ الجلسة مع clock.nowMillis() ، وهذا هو tokenSavedAt .

ي) isExpired() (الأسطر 154-158)

```
age = now - savedAtMillis
expired = age < 0 || age ≥ 60_000
```

- age ≥ 60_000 : مرت 60 ثانية أو أكثر، فال token منتهٍ. هذا متطلب "Treat token as expired after 60 seconds".
- age < 0 : الوقت الحالي قبل وقت الحفظ. يحدث هذا إذا غيّر المستخدم ساعة الجهاز للخلف. لا نثق بال token، فنعتبره منتهياً ونجّده. حالة حدية (edge case) نادرة، لكن معالجتها تُظهر الاحترافية.

مدة الصلاحية = 60_000L ميلي ثانية = 60 ثانية. الشرطة السفلية _ لتسهيل قراءة الأرقام فقط.

21. data/auth/AuthInterceptor.kt

الدور: متطلب "Authorization Interceptor".

app/src/main/java/com/example/fakestore/data/auth/AuthInterceptor.kt

27 lines

```

1 package com.example.fakestore.data.auth
2
3 import okhttp3.Interceptor
4 import okhttp3.Response
5 import javax.inject.Inject
6
7 /**
8  * Adds `Authorization: Bearer <token>` to every request of the authenticated client.
9  *
10 * [AuthManager.getValidToken] guarantees the token is not expired: if it is, the call
11 * blocks here until the (single, shared) refresh finishes. A request is therefore never
12 * sent with an expired token.
13 */
14 class AuthInterceptor @Inject constructor(
15     private val authManager: AuthManager,
16 ) : Interceptor {
17
18     override fun intercept(chain: Interceptor.Chain): Response {
19         val token = authManager.getValidToken()
20
21         val authorizedRequest = chain.request().newBuilder()
22             .header(AuthHeaders.AUTHORIZATION, AuthHeaders.bearer(token))
23             .build()
24
25         return chain.proceed(authorizedRequest)
26     }
27 }

```

الشرح

يرث Interceptor من OkHttp، وفيه دالة واحدة هي intercept(chain)، يستدعيها OkHttp لكل طلب يمر بالعميل المحمي:

1. authManager.getValidToken(): اطلب token صالحاً. - إذا كان صالحاً يعود فوراً. - إذا كان منتهياً ينتظر هنا حتى ينتهي التجديد (واحد مشترك بين كل الطلبات). - النتيجة: لا يُرسل أي طلب بـ token منتهٍ أبداً. هذا متطلب "Do not send requests with an expired token".
2. chain.request().newBuilder(): الطلبات في OkHttp غير قابلة للتعديل (immutable)، فننشئ نسخة جديدة منها.
3. header("Authorization", "Bearer ..."): نضيف الـ header. header تستبدل أي قيمة سابقة بنفس الاسم، بخلاف addHeader التي تضيف نسخة ثانية.
4. chain.proceed(request): نمرر الطلب للحلقة التالية في السلسلة (Interceptor آخر أو الشبكة)، ونرجع الرد.

تشبيه: موظف عند باب الشركة يختم كل رسالة خارجة. إذا كان الختم منتهياً يطلب ختماً جديداً أولاً، وإذا جاء معه زملاء في نفس اللحظة ينتظرون الختم الجديد ولا يطلب كل منهم ختماً.

22. data/auth/TokenAuthenticator.kt

الدور: شبكة أمان إضافية عند رد 401 Unauthorized.

app/src/main/java/com/example/fakestore/data/auth/TokenAuthenticator.kt

35 lines

```
1 package com.example.fakestore.data.auth
2
3 import okhttp3.Authenticator
4 import okhttp3.Request
5 import okhttp3.Response
6 import okhttp3.Route
7 import javax.inject.Inject
8
9 /**
10  * Safety net for a real backend: OkHttp calls this when the server answers 401.
11  * (FakeStore never does, because it does not check tokens; expiry is handled up front
12  * by [AuthInterceptor].)
13  *
14  * Retries at most ONCE per request, so a server that keeps rejecting us can never
15  * cause an infinite refresh loop.
16  */
17 class TokenAuthenticator @Inject constructor(
18     private val authManager: AuthManager,
19 ) : Authenticator {
20
21     override fun authenticate(route: Route?, response: Response): Request? {
22         // priorResponse != null means this request was already retried once and failed again.
23         if (response.priorResponse != null) {
24             authManager.forceLogout()
25             return null
26         }
27
28         val rejectedToken =
29             AuthHeaders.tokenFrom(response.request.header(AuthHeaders.AUTHORIZATION))
30         val newToken = authManager.refreshAfterUnauthorized(rejectedToken) ?: return null
31
32         return response.request.newBuilder()
33             .header(AuthHeaders.AUTHORIZATION, AuthHeaders.bearer(newToken))
34             .build()
35     }
36 }
```

الشرح

ما هو Authenticator في OkHttp؟ كلاس يستدعيه OkHttp تلقائياً عندما يرد السيرفر بالكود 401، أي "غير مصادَق".

- إذا أُرِج طلباً جديداً، يعيد OkHttp المحاولة به.
- إذا أُرِج `null`، يتوقف ويُرجع رد الـ 401 كما هو.

لماذا نحتاجه إذا كان FakeStore لا يرد بـ 401 أبداً؟ في السيرفرات الحقيقية قد يرفض السيرفر token نعتبره صالحاً، مثلاً إذا ألغاه المسؤول أو اختلفت الساعات. هذا الكلاس يُظهر للمراجع أنك تعرف كيف تُبنى المصادقة في التطبيقات الحقيقية.

متى يعمل	الأسلوب
قبل الإرسال	استباقي (Proactive) AuthInterceptor
بعد رد 401	تفاعلي (Reactive) TokenAuthenticator

المنطق خطوة بخطوة:

1. `response.priorResponse != null` : هذا الطلب أُعيدت محاولته مرة من قبل ورفض مرة ثانية. نسجل خروجاً إجبارياً ونرجع `null`. هكذا لا تتكرر المحاولة أكثر من مرة، فلا توجد حلقة لا نهائية.
2. نستخرج الـ token المرفوض من الطلب.
3. نطلب من `AuthManager` تجديداً، وهو يتحقق أولاً هل جدّد طلب آخر قبلنا.
4. نعيد بناء الطلب بالـ token الجديد، فيرسله `OkHttp` مرة ثانية.

الجزء الرابع: ال Repository وحقن الاعتماديات (Hilt)

Dependency Injection (حقن الاعتماديات): بدل أن ينشئ كل كلاس ما يحتاجه بنفسه، يُعطى له من الخارج.

تشبيه: الطاهي لا يزرع الخضار بنفسه، بل يستلمها جاهزة من المورد. **Hilt** هو "المورد": يعرف كيف يصنع كل شيء، ويوصله لمن يحتاجه.

23. FakeStoreApp.kt

الدور: كلاس التطبيق، وأول ما يُنشأ عند التشغيل.

app/src/main/java/com/example/fakestore/FakeStoreApp.kt

10 lines

```
1 package com.example.fakestore
2
3 import android.app.Application
4 import dagger.hilt.android.HiltAndroidApp
5
6 /**
7  * Application entry point. [HiltAndroidApp] generates the app-wide dependency container.
8  */
9 @HiltAndroidApp
10 class FakeStoreApp : Application()
```

الشرح

- `@HiltAndroidApp`: يجعل Hilt يولد "الحاوية" الرئيسية للاعتماديات، التي تعيش طوال عمر التطبيق.
- بدونها لا يعمل أي `@Inject` أو `@AndroidEntryPoint` في المشروع.
- مسجّل في ال Manifest عبر `android:name`.

24. data/repository/ProductRepository.kt

الدور: المصدر الوحيد لبيانات المنتجات بالنسبة لل ViewModels.

app/src/main/java/com/example/fakestore/data/repository/ProductRepository.kt

21 lines

```
1 package com.example.fakestore.data.repository
2
3 import com.example.fakestore.data.model.Product
4 import com.example.fakestore.data.remote.ProductApi
5 import com.example.fakestore.data.remote.dto.toDomain
6 import javax.inject.Inject
7
8 /** Single source of product data for the ViewModels. */
9 interface ProductRepository {
10     suspend fun getProducts(): List<Product>
11     suspend fun getProduct(id: Int): Product
12 }
```

```

13
14 class ProductRepositoryImpl @Inject constructor(
15     private val api: ProductApi,
16 ) : ProductRepository {
17
18     override suspend fun getProducts(): List<Product> = api.getProducts().map { it.toDomain() }
19
20     override suspend fun getProduct(id: Int): Product = api.getProduct(id).toDomain()
21 }

```

الشرح

- **interface** للفصل: ال ViewModel يعرف `ProductRepository` فقط، ولا يعرف Retrofit.
 - `ProductRepositoryImpl`: يستدعي ال API ويحوّل كل DTO إلى `Product` عبر `toDomain()`.
 - `map { it.toDomain() }`: يحوّل كل عنصر في القائمة.
 - `@Inject constructor(private val api: ProductApi)`: Hilt يعطيه `ProductApi` جاهزاً، ومع ال العميل المحمي وال `Interceptor`.
- هذه طبقة ال Model في MVVM. لو أضفنا لاحقاً تخزيناً محلياً (Room) للعمل بلا إنترنت، نعدّل هذا الكلاس فقط.

25. di/Qualifiers.kt

الدور: تسميتان لتمييز عمليّي OkHttpClient.

app/src/main/java/com/example/fakestore/di/Qualifiers.kt

13 lines

```

1 package com.example.fakestore.di
2
3 import javax.inject.Qualifier
4
5 /** OkHttpClient WITHOUT auth: used only for `/auth/login`. */
6 @Qualifier
7 @Retention(AnnotationRetention.BINARY)
8 annotation class PlainClient
9
10 /** OkHttpClient WITH AuthInterceptor + TokenAuthenticator: used for protected APIs. */
11 @Qualifier
12 @Retention(AnnotationRetention.BINARY)
13 annotation class AuthClient

```

الشرح

المشكلة: عندنا كائنان من نفس النوع `OkHttpClient`، و Hilt يميّز الأشياء بنوعها، فلن يعرف أيهما يعطي.

الحل: `@Qualifier`، أي annotation نصنعها لتكون "ملصقاً" يميّز بينهما:

العميل	الملصق
بلا مصادقة، لطلب login فقط	<code>@PlainClient</code>
مع <code>AuthInterceptor</code> و <code>TokenAuthenticator</code> ، للمنتجات	<code>@AuthClient</code>

الدور: "وصفة" بناء كل ما يخص الشبكة، وهنا يتحقق فصل العميلين.

app/src/main/java/com/example/fakestore/di/NetworkModule.kt

99 lines

```
1 package com.example.fakestore.di
2
3 import com.example.fakestore.BuildConfig
4 import com.example.fakestore.data.auth.AuthHeaders
5 import com.example.fakestore.data.auth.AuthInterceptor
6 import com.example.fakestore.data.auth.TokenAuthenticator
7 import com.example.fakestore.data.mock.MockBackendInterceptor
8 import com.example.fakestore.data.remote.ApiConfig
9 import com.example.fakestore.data.remote.AuthApi
10 import com.example.fakestore.data.remote.ProductApi
11 import dagger.Module
12 import dagger.Provides
13 import dagger.hilt.InstallIn
14 import dagger.hilt.components.SingletonComponent
15 import okhttp3.OkHttpClient
16 import okhttp3.logging.HttpLoggingInterceptor
17 import retrofit2.Retrofit
18 import retrofit2.converter.gson.GsonConverterFactory
19 import java.util.concurrent.TimeUnit
20 import javax.inject.Singleton
21
22 @Module
23 @InstallIn(SingletonComponent::class)
24 object NetworkModule {
25
26     private const val TIMEOUT_SECONDS = 20L
27
28     /**
29      * Dev-only fake server, shared by both clients. Null in normal builds: requests then
30      * go to the real fakestoreapi.com. (Kept out of the Hilt graph on purpose: it is a
31      * build-time switch, not a dependency anyone else needs.)
32      */
33     private val mockBackend: MockBackendInterceptor? =
34         if (BuildConfig.USE MOCK_BACKEND) MockBackendInterceptor() else null
35
36     @Provides
37     @Singleton
38     fun provideLoggingInterceptor(): HttpLoggingInterceptor =
39         HttpLoggingInterceptor().apply {
40             // BASIC prints one line per request (method + URL + status): enough to see
41             // in Logcat that only one /auth/login runs. Never log bodies: they hold the
42             password.
43             level = if (BuildConfig.DEBUG) {
44                 HttpLoggingInterceptor.Level.BASIC
45             } else {
46                 HttpLoggingInterceptor.Level.NONE
47             }
48             redactHeader(AuthHeaders.AUTHORIZATION)
49         }
```

```

50 @Provides
51 @Singleton
52 @PlainClient
53 fun providePlainClient(logging: HttpLoggingInterceptor): OkHttpClient =
54     baseClientBuilder()
55         .addInterceptor(logging)
56         .addMockBackend(mockBackend)
57         .build()
58
59 @Provides
60 @Singleton
61 @AuthClient
62 fun provideAuthClient(
63     authInterceptor: AuthInterceptor,
64     tokenAuthenticator: TokenAuthenticator,
65     logging: HttpLoggingInterceptor,
66 ): OkHttpClient =
67     baseClientBuilder()
68         .addInterceptor(authInterceptor) // first: attach a valid token
69         .addInterceptor(logging)        // then: log the final request
70         .addMockBackend(mockBackend)    // last (dev only): answer instead of the server
71         .authenticator(tokenAuthenticator)
72         .build()
73
74 @Provides
75 @Singleton
76 fun provideAuthApi(@PlainClient client: OkHttpClient): AuthApi =
77     buildRetrofit(client).create(AuthApi::class.java)
78
79 @Provides
80 @Singleton
81 fun provideProductApi(@AuthClient client: OkHttpClient): ProductApi =
82     buildRetrofit(client).create(ProductApi::class.java)
83
84 private fun baseClientBuilder(): OkHttpClient.Builder =
85     OkHttpClient.Builder()
86         .connectTimeout(TIMEOUT_SECONDS, TimeUnit.SECONDS)
87         .readTimeout(TIMEOUT_SECONDS, TimeUnit.SECONDS)
88
89 /** The mock must stay last: it answers the request instead of passing it on. */
90 private fun OkHttpClient.Builder.addMockBackend(mock: MockBackendInterceptor?) =
91     apply { if (mock != null) addInterceptor(mock) }
92
93 private fun buildRetrofit(client: OkHttpClient): Retrofit =
94     Retrofit.Builder()
95         .baseUrl(ApiConfig.BASE_URL)
96         .client(client)
97         .addConverterFactory(GsonConverterFactory.create())
98         .build()
99 }

```

الشرح

مصطلحات Hilt في هذا الملف:

المصطلح	المعنى
+ @Module	وحدة وصفات تعيش طوال عمر التطبيق
@InstallIn(SingletonComponent::class)	

المصطلح	المعنى
@Provides	"هكذا تصنع هذا الشيء". نحتاجه لكلاسات من مكتبات خارجية لا نستطيع وضع @Inject عليها
@Singleton	اصنعه مرة واحدة فقط وأعد استخدامه

provideLoggingInterceptor : سجل الطلبات في Logcat.

- مستوى BASIC : سطر واحد لكل طلب (الطريقة، والعنوان، والنتيجة). هذا كافٍ لإثبات أن login حدث مرة واحدة.
- يعمل في نسخة التطوير (DEBUG) فقط، ومعتّل في النسخة النهائية.
- redactHeader("Authorization") : يخفي الـ token من السجل.
- لا نسجّل جسم الطلبات أبداً، لأن جسم طلب login يحتوي كلمة المرور.

providePlainClient : العميل العادي. فيه سجل الطلبات فقط، بلا AuthInterceptor.

provideAuthClient : العميل المحمي:

- الترتيب مهم: authInterceptor أولاً ليضيف الـ token، ثم logging ليسجّل الطلب النهائي.
- authenticator(tokenAuthenticator) : لمعالجة رد 401.

provideProductApi و **provideAuthApi** : كل API مع عميله:

- AuthApi ← @PlainClient
- ProductApi ← @AuthClient

الدوال المساعدة:

- baseClientBuilder : الإعدادات المشتركة، أي مهلة 20 ثانية (timeout) للاتصال والقراءة.
- buildRetrofit : ينشئ Retrofit بالعنوان الأساسي ومحوّل Gson.

mockBackend و **addMockBackend** (للتطوير فقط):

- إذا كان BuildConfig.USE MOCK_BACKEND يساوي true، يُنشأ MockBackendInterceptor واحد يشترك فيه العميلان.
- addMockBackend يضيفه في آخر السلسلة. بعد AuthInterceptor وسجل الطلبات، فكل شيء قبله يعمل كما في الإنتاج.
- إذا كان false يكون null، ولا يُضاف شيء، فتذهب الطلبات إلى FakeStore الحقيقي.
- لم نضعه في Hilt عن قصد، لأنه مفتاح وقت البناء، وليس اعتمادية يحتاجها كلاس آخر.

خلاصة هذا الملف: هنا تحديداً نمنع الحلقة اللانهائية. طلب login لا يمر أبداً بالـ AuthInterceptor.

27. di/DataModule.kt

الدور: ربط كل interface بتنفيذه الحقيقي.

app/src/main/java/com/example/fakestore/di/DataModule.kt

27 lines

```

1 package com.example.fakestore.di
2
3 import com.example.fakestore.data.auth.Clock
4 import com.example.fakestore.data.auth.EncryptedTokenStorage
5 import com.example.fakestore.data.auth.SystemTimeClock
6 import com.example.fakestore.data.auth.TokenStorage
7 import com.example.fakestore.data.repository.ProductRepository
8 import com.example.fakestore.data.repository.ProductRepositoryImpl
9 import dagger.Binds
10 import dagger.Module
11 import dagger.hilt.InstallIn
12 import dagger.hilt.components.SingletonComponent
13
14 /** Binds each interface to its production implementation. */
15 @Module
16 @InstallIn(SingletonComponent::class)
17 abstract class DataModule {
18
19     @Binds
20     abstract fun bindTokenStorage(impl: EncryptedTokenStorage): TokenStorage
21
22     @Binds
23     abstract fun bindClock(impl: SystemTimeClock): Clock
24
25     @Binds
26     abstract fun bindProductRepository(impl: ProductRepositoryImpl): ProductRepository
27 }

```

الشرح

`@Binds` معناها: "عندما يطلب أحد X أعطه Y". وهي أخف من `@Provides` لأنها لا تحتاج كتابة كود إنشاء.

يُطلب	يُعطى
TokenStorage	EncryptedTokenStorage
Clock	SystemTimeClock
ProductRepository	ProductRepositoryImpl

لتغيير طريقة التخزين لاحقاً (مثلاً إلى DataStore): غيّر سطرًا واحداً هنا.

الجزء الخامس: أدوات الواجهة المشتركة

28. ui/common/UiState.kt

الدور: حالات أي شاشة تحمّل بيانات.

app/src/main/java/com/example/fakestore/ui/common/UiState.kt

10 lines

```
1 package com.example.fakestore.ui.common
2
3 import androidx.annotation.StringRes
4
5 /** State of a screen that loads data: exactly one of these at a time. */
6 sealed interface UiState<out T> {
7     data object Loading : UiState<Nothing>
8     data class Success<T>(val data: T) : UiState<T>
9     data class Error(@StringRes val messageRes: Int) : UiState<Nothing>
10 }
```

الشرح

- sealed interface بثلاث حالات فقط، والشاشة في واحدة منها دائماً:

الحالة	ماذا تعرض الشاشة
Loading	دائرة تحميل
Success(data)	البيانات
Error(messageRes)	رسالة خطأ

- <out T> : يسمح باستخدام Loading (النوع Nothing) مكان أي <أي شيء> UiState.
- @StringRes Int : رقم مورد نصي مثل R.string.error_network ، وليس نصاً مكتوباً. هكذا تبقى النصوص في strings.xml ، فيسهل ترجمتها لاحقاً.

29. ui/common/ErrorMessage.kt

الدور: تحويل أي خطأ تقني إلى رسالة مفهومة للمستخدم.

app/src/main/java/com/example/fakestore/ui/common/ErrorMessage.kt

20 lines

```
1 package com.example.fakestore.ui.common
2
3 import androidx.annotation.StringRes
4 import com.example.fakestore.R
5 import com.example.fakestore.data.auth.InvalidCredentialsException
6 import com.example.fakestore.data.auth.ServerUnavailableException
```

```

7 import com.example.fakestore.data.auth.SessionExpiredException
8 import retrofit2.HttpException
9 import java.io.IOException
10
11 /** Maps any failure to a message the user can understand. Order matters: most specific first.
*/
12 @StringRes
13 fun Throwable.toMessageRes(): Int = when (this) {
14     is InvalidCredentialsException → R.string.error_invalid_credentials
15     is SessionExpiredException → R.string.error_session_expired
16     is ServerUnavailableException → R.string.error_server_unavailable
17     is HttpException → if (code() ≥ 500) R.string.error_server_unavailable else
R.string.error_server
18     is IOException → R.string.error_network
19     else → R.string.error_unknown
20 }

```

الشرح

الترتيب مهم: من الأكثر تحديداً إلى الأعم. `SessionExpiredException` و `InvalidCredentialsException` و `ServerUnavailableException` كلها ترث من `IOException`. لو وضعنا `is IOException` أولاً، لظهرت لهما رسالة "لا يوجد إنترنت" الخاطئة.

الخطأ	الرسالة
<code>InvalidCredentialsException</code>	اسم المستخدم أو كلمة المرور خاطئة
<code>SessionExpiredException</code>	انتهت الجلسة
<code>ServerUnavailableException</code>	السيرفر غير متاح حالياً، حاول لاحقاً
<code>HttpException</code> بـ 5xx (من Retrofit)	السيرفر غير متاح حالياً، حاول لاحقاً
<code>HttpException</code> بـ كود آخر	خطأ في السيرفر
<code>IOException</code>	لا يوجد إنترنت
أي شيء آخر	حدث خطأ ما

30. `ui/common/FlowExt.kt`

الدور: دالة مساعدة لمراقبة الـ `Flow` بأمان داخل `Fragment`.

app/src/main/java/com/example/fakestore/ui/common/FlowExt.kt

20 lines

```

1 package com.example.fakestore.ui.common
2
3 import androidx.fragment.app.Fragment
4 import androidx.lifecycle.Lifecycle
5 import androidx.lifecycle.LifecycleScope
6 import androidx.lifecycle.repeatOnLifecycle
7 import kotlinx.coroutines.flow.Flow
8 import kotlinx.coroutines.launch
9

```

```

10  /**
11   * Collects [flow] only while the fragment's view is visible (STARTED), and stops
12   * automatically when the view is destroyed. Avoids leaks and updates to dead views.
13   */
14  fun <T> Fragment.collectWhenStarted(flow: Flow<T>, action: (T) → Unit) {
15      viewLifecycleOwner.lifecycleScope.launch {
16          viewLifecycleOwner.repeatOnLifecycle(Lifecycle.State.STARTED) {
17              flow.collect { action(it) }
18          }
19      }
20  }

```

الشرح

المشكلة: الـ Fragment له دورة حياة، وواجهته (**View**) تُدمّر وتُنشأ من جديد، مثلاً عند الانتقال لشاشة أخرى ثم العودة. لو بقينا نحدّث واجهة مدمّرة لحدث crash أو تسريب ذاكرة.

الحل:

- `viewLifecycleOwner`: دورة حياة **الواجهة** فقط، وليس الـ Fragment كله.
 - `: repeatOnLifecycle(STARTED)`
 - يبدأ المراقبة عندما تصبح الشاشة ظاهرة.
 - يوقفها عندما تختفي، مثلاً عند الذهاب للخلفية.
 - يعيد تشغيلها عند العودة.
 - عند تدمير الواجهة يتوقف كل شيء تلقائياً.
- الاستخدام:** في كل Fragment نكتب سطرًا واحدًا:

```
collectWhenStarted(viewModel.state) { render(it) }
```

31. ui/common/Formatters.kt

الدور: تنسيق السعر.

```
app/src/main/java/com/example/fakestore/ui/common/Formatters.kt
```

7 lines

```

1  package com.example.fakestore.ui.common
2
3  import java.text.NumberFormat
4  import java.util.Locale
5
6  /** FakeStore prices are in US dollars. */
7  fun Double.formatPrice(): String = NumberFormat.getCurrencyInstance(Locale.US).format(this)

```

الشرح

- `NumberFormat.getCurrencyInstance(Locale.US)`: يحوّل 109.95 إلى \$109.95.
- `Locale.US` ثابت لأن أسعار FakeStore بالدولار، مهما كانت لغة الجهاز.

الجزء السادس: الشاشات (View + ViewModel)

نمط MVVM في كل شاشة:

الطبقة	الملفات	الدور
View	XML ملف + Fragment	تعرض الحالة، وتُبلغ ال ViewModel بضغوطات المستخدم
ViewModel	...ViewModel.kt	يطلب البيانات، ويحوّلها إلى UiState ، ويبقى حياً عند تدوير الشاشة
Model	AuthManager / Repository	البيانات

32. res/values/strings.xml

الدور: كل نصوص التطبيق.

```
app/src/main/res/values/strings.xml 35 lines

1 <resources>
2   <string name="app_name">FakeStore</string>
3
4   <!-- Screen titles -->
5   <string name="title_login">Login</string>
6   <string name="title_products">Products</string>
7   <string name="title_product_detail">Product details</string>
8
9   <!-- Login -->
10  <string name="login_heading">Welcome back</string>
11  <string name="login_subtitle">Sign in to browse products</string>
12  <string name="hint_username">Username</string>
13  <string name="hint_password">Password</string>
14  <string name="action_login">Log in</string>
15  <string name="mock_backend_banner">Demo mode: using a built-in mock server instead of
fakestoreapi.com</string>
16  <string name="test_account_hint">Test account: mor_2314 / 83r5^</string>
17
18  <!-- Products -->
19  <string name="menu_refresh">Refresh</string>
20  <string name="menu_concurrency_test">Run concurrency test</string>
21  <string name="menu_logout">Log out</string>
22  <string name="concurrency_test_done">%1$d parallel requests finished. Check Logcat: only one
/auth/login.</string>
23  <string name="product_rating">★ %1$.1f (%2$d reviews)</string>
24  <string name="product_image">Product image</string>
25
26  <!-- Errors -->
27  <string name="retry">Retry</string>
28  <string name="error_empty_fields">Please enter your username and password.</string>
29  <string name="error_invalid_credentials">Invalid username or password.</string>
30  <string name="error_session_expired">Your session has expired. Please log in again.</string>
31  <string name="error_server">The server returned an error. Please try again.</string>
32  <string name="error_server_unavailable">The server is unavailable right now. Please try
again later.</string>
```



```

33     <string name="error_network">No internet connection. Check your network and try again.
</string>
34     <string name="error_unknown">Something went wrong. Please try again.</string>
35 </resources>

```

الشرح

- لا نكتب نصوصاً داخل الكود أو XML مباشرة، فكل نص هنا. هذا يسهّل الترجمة لاحقاً بملف `values-ar/strings.xml`
- `%1$d` و `%1$.1f` : أماكن لقيم تُملأ وقت التشغيل:
 - `%1$d` : رقم صحيح (الأول).
 - `%1$.1f` : رقم عشري بخانة واحدة بعد الفاصلة.
- `test_account_hint` : يعرض بيانات حساب الاختبار على شاشة الدخول، لتسهيل التجربة على المراجع.

33. `res/values/themes.xml`

الدور: ثيم التطبيق.

```

app/src/main/res/values/themes.xml 3 lines
1 <resources>
2     <style name="Theme.FakeStore" parent="Theme.Material3.DayNight.NoActionBar" />
3 </resources>

```

الشرح

- `Theme.Material3.DayNight` : تصميم Material 3 مع دعم الوضع الليلي تلقائياً.
- `NoActionBar` : بلا شريط علوي افتراضي، لأننا نضع `MaterialToolbar` بأنفسنا في `activity_main.xml`.

34. `res/layout/activity_main.xml`

الدور: هيكل الشاشة الرئيسية: شريط علوي، ومساحة تتبدل فيها الشاشات.

```

app/src/main/res/layout/activity_main.xml 26 lines
1 <?xml version="1.0" encoding="utf-8"?>
2 <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3     xmlns:app="http://schemas.android.com/apk/res-auto"
4     android:layout_width="match_parent"
5     android:layout_height="match_parent"
6     android:orientation="vertical">
7
8     <com.google.android.material.appbar.AppBarLayout
9         android:layout_width="match_parent"
10        android:layout_height="wrap_content">
11
12        <com.google.android.material.appbar.MaterialToolbar
13            android:id="@+id/toolbar"
14            android:layout_width="match_parent"
15            android:layout_height="?attr/actionBarSize" />

```

```

16 </com.google.android.material.appbar.AppBarLayout>
17
18 <!-- The graph is set from code (MainActivity) to pick the start screen. -->
19 <androidx.fragment.app.FragmentContainerView
20     android:id="@+id/nav_host"
21     android:name="androidx.navigation.fragment.NavHostFragment"
22     android:layout_width="match_parent"
23     android:layout_height="0dp"
24     android:layout_weight="1"
25     app:defaultNavHost="true" />
26 </LinearLayout>

```

الشرح

- `MaterialToolbar` داخل `AppBarLayout`: الشريط العلوي. يعرض عنوان الشاشة الحالية، وسهم الرجوع، والقائمة (:).
- `FragmentContainerView` مع `NavHostFragment`: المساحة التي تُعرض فيها الشاشات (Fragments)، ومكوّن `Navigation` يبدّلها.
- `app:defaultNavHost="true"`: زر الرجوع في الجهاز يتحكم بالتنقل داخل هذه المساحة.
- `layout_height="0dp"` مع `layout_weight="1"`: تأخذ كل المساحة المتبقية تحت الشريط.
- `app:navGraph` هنا عن قصد. نحدد خريطة التنقل من الكود في `MainActivity`، لنختار شاشة البداية.

35. res/navigation/nav_graph.xml

الدور: خريطة التنقل: ما الشاشات، وكيف نتنقل بينها.

app/src/main/res/navigation/nav_graph.xml

47 lines

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <!--
3     Screens and the transitions between them. The real start destination is chosen in
4     MainActivity (login or products, depending on whether a session exists).
5 -->
6 <navigation xmlns:android="http://schemas.android.com/apk/res/android"
7     xmlns:app="http://schemas.android.com/apk/res-auto"
8     android:id="@+id/nav_graph"
9     app:startDestination="@id/loginFragment">
10
11     <fragment
12         android:id="@+id/loginFragment"
13         android:name="com.example.fakestore.ui.login.LoginFragment"
14         android:label="@string/title_login">
15         <!-- popUpTo login (inclusive): Back from Products must not return to Login. -->
16         <action
17             android:id="@+id/action_login_to_products"
18             app:destination="@id/productsFragment"
19             app:popUpTo="@id/loginFragment"
20             app:popUpToInclusive="true" />
21     </fragment>
22
23     <fragment
24         android:id="@+id/productsFragment"
25         android:name="com.example.fakestore.ui.products.ProductsFragment"
26         android:label="@string/title_products">
27         <action

```

```

28         android:id="@+id/action_products_to_detail"
29         app:destination="@id/productDetailFragment" />
30     </fragment>
31
32     <fragment
33         android:id="@+id/productDetailFragment"
34         android:name="com.example.fakestore.ui.detail.ProductDetailFragment"
35         android:label="@string/title_product_detail">
36         <argument
37             android:name="productId"
38             app:argType="integer" />
39     </fragment>
40
41     <!-- Logout from anywhere: clear the whole back stack, then show Login. -->
42     <action
43         android:id="@+id/action_global_login"
44         app:destination="@id/loginFragment"
45         app:popUpTo="@id/nav_graph"
46         app:popUpToInclusive="true" />
47 </navigation>

```

الشرح

- `<fragment>` : كل شاشة لها `id` واسم كلاسها (`android:name`) وعنوانها (`label`) ، والعنوان يظهر تلقائياً في الشريط العلوي.
- `<action>` : انتقال مسموح من شاشة إلى أخرى.
- `action_login_to_products`
- `popUpTo="@id/loginFragment" + popUpToInclusive="true"` : عند الانتقال للمنتجات تُحذف شاشة Login من سجل الرجوع (`back stack`).
- لولا ذلك، لعاد المستخدم لشاشة Login بزر الرجوع وهو مسجل الدخول.
- `<argument android:name="productId">`
- شاشة التفاصيل تحتاج رقم المنتج.
- `argType="integer"` : نوعه رقم صحيح.
- `action_global_login` : انتقال عام (`global action`)، يمكن استخدامه من أي شاشة:
- `inclusive + popUpTo="@id/nav_graph"` : **يمسح كل الشاشات** ثم يفتح Login.
- يُستخدم في Logout، وفي الخروج الإجباري. هذا تحقيق لمتطلب `"Redirect user to Login screen"`. بعده، زر الرجوع يُغلق التطبيق بدل العودة للمنتجات.

36. MainActivity.kt

الدور: ال Activity الوحيدة. تختار شاشة البداية، وتستمع للخروج الإجباري.

app/src/main/java/com/example/fakestore/MainActivity.kt

85 lines

```

1 package com.example.fakestore
2
3 import android.os.Bundle

```

```

4 import android.widget.Toast
5 import androidx.appcompat.app.AppCompatActivity
6 import androidx.lifecycle.Lifecycle
7 import androidx.lifecycle.LifecycleScope
8 import androidx.lifecycle.repeatOnLifecycle
9 import androidx.navigation.NavController
10 import androidx.navigation.fragment.NavHostFragment
11 import androidx.navigation.ui.AppBarConfiguration
12 import androidx.navigation.ui.navigateUp
13 import androidx.navigation.ui.setupActionBarWithNavController
14 import com.example.fakestore.data.auth.AuthEvent
15 import com.example.fakestore.data.auth.AuthManager
16 import com.example.fakestore.databinding.ActivityMainBinding
17 import dagger.hilt.android.AndroidEntryPoint
18 import kotlinx.coroutines.launch
19 import javax.inject.Inject
20
21 /**
22  * Single activity hosting all screens (fragments). It also listens for forced logouts,
23  * because they can happen on any screen.
24  */
25 @AndroidEntryPoint
26 class MainActivity : AppCompatActivity() {
27
28     @Inject
29     lateinit var authManager: AuthManager
30
31     private lateinit var binding: ActivityMainBinding
32     private lateinit var navController: NavController
33     private lateinit var appBarConfiguration: AppBarConfiguration
34
35     override fun onCreate(savedInstanceState: Bundle?) {
36         super.onCreate(savedInstanceState)
37         binding = ActivityMainBinding.inflate(layoutInflater)
38         setContentView(binding.root)
39         setSupportActionBar(binding.toolbar)
40
41         val navHost = supportFragmentManager.findFragmentById(R.id.nav_host) as NavHostFragment
42         navController = navHost.navController
43
44         // Start on the product list if a session exists (an expired token is refreshed
45         // automatically on the first request), otherwise on the login screen.
46         val graph = navController.navInflater.inflate(R.navigation.nav_graph).apply {
47             setStartDestination(
48                 if (authManager.isLoggedIn()) R.id.productsFragment else R.id.loginFragment,
49             )
50         }
51         navController.setGraph(graph, null)
52
53         // Login and Products are top-level: no "back" arrow on them.
54         appBarConfiguration = AppBarConfiguration(setOf(R.id.loginFragment,
55 R.id.productsFragment))
56         setupActionBarWithNavController(navController, appBarConfiguration)
57
58         observeSession()
59     }
60
61     override fun onSupportNavigateUp(): Boolean =
62         navController.navigateUp(appBarConfiguration) || super.onSupportNavigateUp()
63
64     private fun observeSession() {
65         lifecycleScope.launch {

```

```

65         repeatOnLifecycle(Lifecycle.State.STARTED) {
66             // The session may have been cleared while the app was in the background.
67             if (!authManager.isLoggedIn()) goToLogin(showMessage = false)
68
69             authManager.events.collect { event →
70                 when (event) {
71                     AuthEvent.SessionExpired → goToLogin(showMessage = true)
72                 }
73             }
74         }
75     }
76 }
77
78 private fun goToLogin(showMessage: Boolean) {
79     if (navController.currentDestination?.id == R.id.loginFragment) return
80     if (showMessage) {
81         Toast.makeText(this, R.string.error_session_expired, Toast.LENGTH_LONG).show()
82     }
83     navController.navigate(R.id.action_global_login)
84 }
85 }

```

الشرح

@AndroidEntryPoint : يسمح لـ Hilt بحقن الاعتماديات هنا وفي الـ Fragments داخلها.

@Inject lateinit var authManager : حقن في حقل (**field injection**). Hilt يملؤه قبل **onCreate** . نستخدم هذا الأسلوب لأن Android هو من ينشئ الـ Activity، وليس نحن، فلا نستطيع تمرير شيء لـ **constructor**.

onCreate :

1. **ActivityMainBinding.inflate** : ننشئ الواجهة عبر **ViewBinding**.

2. **setSupportActionBar(binding.toolbar)** : نجعل الـ **Toolbar** هو الشريط العلوي.

3. نحصل على **navController** من **NavHostFragment**.

4. **اختيار شاشة البداية** : - نحمل خريطة التنقل من **XML** - نحدد البداية: إذا كانت هناك جلسة محفوظة نبدأ بـ **المنتجات**، وإلا بـ **Login** . - حتى لو كان الـ **token** منتهياً، سيُجدد تلقائياً مع أول طلب.

5. **AppBarConfiguration(setOf(login, products))** : هاتان شاشتان رئيسيتان (**top-level**)، فلا يظهر عليهما سهم الرجوع. أما التفاصيل فيظهر عليها.

observeSession() : مراقبة الجلسة، تعمل فقط والتطبيق ظاهر.

1. **أولاً**: إذا لم تكن هناك جلسة نذهب لشاشة **Login**. قد تكون الجلسة مُسحت والتطبيق في الخلفية، حين لم يكن أحد يستمع للحدث.

2. **ثانياً**: نستمع لـ **authManager.events** . عند وصول **SessionExpired** ننتقل لـ **Login** مع رسالة.

لماذا هنا في MainActivity وليس في كل Fragment؟ الخروج الإجباري قد يحدث في أي شاشة، أثناء القائمة أو التفاصيل. الـ Activity موجودة دائماً، فهي المكان المناسب لاستقبال الحدث مرة واحدة.

goToLogin : إذا كنا أصلاً في **Login** لا نفعل شيئاً، فهذا يمنع فتح **Login** مرتين.

app/src/main/java/com/example/fakestore/ui/login/LoginViewModel.kt

53 lines

```

1  package com.example.fakestore.ui.login
2
3  import androidx.annotation.StringRes
4  import androidx.lifecycle.ViewModel
5  import androidx.lifecycle.viewModelScope
6  import com.example.fakestore.R
7  import com.example.fakestore.data.auth.AuthManager
8  import com.example.fakestore.ui.common.toMessageRes
9  import dagger.hilt.android.lifecycle.HiltViewModel
10 import kotlinx.coroutines.CancellationException
11 import kotlinx.coroutines.flow.MutableStateFlow
12 import kotlinx.coroutines.flow.StateFlow
13 import kotlinx.coroutines.flow.asStateFlow
14 import kotlinx.coroutines.flow.update
15 import kotlinx.coroutines.launch
16 import javax.inject.Inject
17
18 data class LoginUiState(
19     val isLoading: Boolean = false,
20     @StringRes val errorRes: Int? = null,
21     val isLoggedIn: Boolean = false,
22 )
23
24 @HiltViewModel
25 class LoginViewModel @Inject constructor(
26     private val authManager: AuthManager,
27 ) : ViewModel() {
28
29     private val _state = MutableStateFlow(LoginUiState())
30     val state: StateFlow<LoginUiState> = _state.asStateFlow()
31
32     fun login(username: String, password: String) {
33         if (_state.value.isLoading) return // ignore double taps
34
35         val trimmedUsername = username.trim()
36         if (trimmedUsername.isEmpty() || password.isEmpty()) {
37             _state.update { it.copy(errorRes = R.string.error_empty_fields) }
38             return
39         }
40
41         viewModelScope.launch {
42             _state.update { it.copy(isLoading = true, errorRes = null) }
43             try {
44                 authManager.login(trimmedUsername, password)
45                 _state.update { it.copy(isLoading = false, isLoggedIn = true) }
46             } catch (e: CancellationException) {
47                 throw e
48             } catch (e: Exception) {
49                 _state.update { it.copy(isLoading = false, errorRes = e.toMessageRes()) }
50             }
51         }
52     }
53 }

```

`LoginUiState` : كل ما تحتاجه الشاشة في كائن واحد:

الحقل	المعنى
<code>isLoading</code>	يجري تسجيل الدخول؟ (دائرة تحميل، وزر معطل)
<code>errorRes</code>	رسالة خطأ، أو <code>null</code> إن لم يكن هناك خطأ
<code>isLoggedIn</code>	نجح الدخول؟ (إشارة للانتقال)

`: StateFlow / MutableStateFlow`

- `_state` خاص وقابل للتعديل.
- `state` عام وللقراءة فقط: **الشاشة تقرأ ولا تعدّل.**
- `_state.update { it.copy(...) }` : ينسخ الحالة الحالية مع تغيير حقول معينة، بشكل آمن مع الـ `threads`.

login() خطوة بخطوة:

1. إذا كان التحميل جارياً نتجاهل الضغطة، فهذا يمنع **الضغط المزدوج** على الزر.
2. التحقق المحلي: إذا كانت الحقول فارغة نعرض خطأ **دون** إرسال طلب.
3. `viewModelScope.launch` : نبدأ `coroutine` تُلغى تلقائياً إذا أُغلقت الشاشة نهائياً.
4. نجعل الحالة `Loading`.
5. نستدعي `authManager.login()` : - **نجاح**: `isLoggedIn = true` . - **فشل**: نحوّل الخطأ إلى رسالة عبر `.toMessageRes()`

`: catch (e: CancellationException) { throw e }`

- إلغاء الـ `coroutine` ليس خطأً.
- إذا التقطناه ضمن `catch (e: Exception)` ولم نمرره، تتعطل آلية الإلغاء.
- لذلك نعيد رميه دائماً، وهذه ممارسة احترافية مهمة.

38. `res/layout/fragment_login.xml`

الدور: تصميم شاشة تسجيل الدخول.

app/src/main/res/layout/fragment_login.xml	113 lines
<pre> 1 <?xml version="1.0" encoding="utf-8"?> 2 <ScrollView xmlns:android="http://schemas.android.com/apk/res/android" 3 xmlns:app="http://schemas.android.com/apk/res-auto" 4 android:layout_width="match_parent" 5 android:layout_height="match_parent" 6 android:fillViewport="true"> 7 8 <LinearLayout 9 android:layout_width="match_parent" 10 android:layout_height="wrap_content" 11 android:layout_gravity="center_vertical" 12 android:orientation="vertical" </pre>	

```

13     android:padding="24dp">
14
15     ←!— Shown only when the dev mock server is enabled (see gradle.properties). →
16     <TextView
17         android:id="@+id/mock_backend_banner"
18         android:layout_width="match_parent"
19         android:layout_height="wrap_content"
20         android:layout_marginBottom="16dp"
21         android:background="?attr/colorTertiaryContainer"
22         android:padding="12dp"
23         android:text="@string/mock_backend_banner"
24         android:textColor="?attr/colorOnTertiaryContainer"
25         android:visibility="gone" />
26
27     <TextView
28         android:layout_width="wrap_content"
29         android:layout_height="wrap_content"
30         android:text="@string/login_heading"
31         android:textAppearance="?attr/textAppearanceHeadlineMedium" />
32
33     <TextView
34         android:layout_width="wrap_content"
35         android:layout_height="wrap_content"
36         android:layout_marginTop="4dp"
37         android:layout_marginBottom="24dp"
38         android:text="@string/login_subtitle"
39         android:textAppearance="?attr/textAppearanceBodyMedium" />
40
41     ←!—
42         Usernames are case-sensitive: textVisiblePassword stops the keyboard from
43         capitalizing the first letter or auto-correcting it (mor_2314 → Mor_2314).
44     →
45     <com.google.android.material.textfield.TextInputLayout
46         android:id="@+id/username_layout"
47         style="@style/Widget.Material3.TextInputLayout.OutlinedBox"
48         android:layout_width="match_parent"
49         android:layout_height="wrap_content"
50         android:hint="@string/hint_username">
51
52         <com.google.android.material.textfield.TextInputEditText
53             android:id="@+id/username_input"
54             android:layout_width="match_parent"
55             android:layout_height="wrap_content"
56             android:autofillHints="username"
57             android:imeOptions="actionNext"
58             android:inputType="textVisiblePassword|textNoSuggestions"
59             android:maxLines="1" />
60     </com.google.android.material.textfield.TextInputLayout>
61
62     <com.google.android.material.textfield.TextInputLayout
63         android:id="@+id/password_layout"
64         style="@style/Widget.Material3.TextInputLayout.OutlinedBox"
65         android:layout_width="match_parent"
66         android:layout_height="wrap_content"
67         android:layout_marginTop="12dp"
68         android:hint="@string/hint_password"
69         app:endIconMode="password_toggle">
70
71         <com.google.android.material.textfield.TextInputEditText
72             android:id="@+id/password_input"
73             android:layout_width="match_parent"
74             android:layout_height="wrap_content"

```



```

75         android:autofillHints="password"
76         android:imeOptions="actionDone"
77         android:inputType="textPassword"
78         android:maxLines="1" />
79     </com.google.android.material.textfield.TextInputLayout>
80
81     <TextView
82         android:id="@+id/error_text"
83         android:layout_width="match_parent"
84         android:layout_height="wrap_content"
85         android:layout_marginTop="12dp"
86         android:textColor="?attr/colorError"
87         android:visibility="gone" />
88
89     <com.google.android.material.button.MaterialButton
90         android:id="@+id/login_button"
91         android:layout_width="match_parent"
92         android:layout_height="wrap_content"
93         android:layout_marginTop="20dp"
94         android:text="@string/action_login" />
95
96     <com.google.android.material.progressindicator.CircularProgressIndicator
97         android:id="@+id/progress"
98         android:layout_width="wrap_content"
99         android:layout_height="wrap_content"
100        android:layout_gravity="center_horizontal"
101        android:layout_marginTop="16dp"
102        android:indeterminate="true"
103        android:visibility="gone" />
104
105    <TextView
106        android:layout_width="wrap_content"
107        android:layout_height="wrap_content"
108        android:layout_gravity="center_horizontal"
109        android:layout_marginTop="24dp"
110        android:text="@string/test_account_hint"
111        android:textAppearance="?attr/textAppearanceBodySmall" />
112    </LinearLayout>
113 </ScrollView>

```

الشرح

- مع `ScrollView` `fillViewport`: الشاشة قابلة للتمرير إذا غطتها لوحة المفاتيح.
- `TextInputLayout` + `TextInputEditText`: حقل إدخال من Material، فيه عنوان عائم وإطار.
- `inputType="textPassword"`: يُخفي كلمة المرور بنقاط.
- `endIconMode="password_toggle"`: أيقونة عين لإظهار كلمة المرور أو إخفائها.
- `imeOptions`: زر لوحة المفاتيح:
 - `actionNext` في حقل الاسم: ينتقل لحقل كلمة المرور.
 - `actionDone` في حقل كلمة المرور: ينفذ الدخول.
- `autofillHints`: يسمح لمدير كلمات المرور بملء الحقول.
- `inputType="textVisiblePassword|textNoSuggestions"` في **حقل اسم المستخدم** (أُضيف في المراجعة):
 - **المشكلة التي واجهناها:** لوحة مفاتيح Infinix كبرت أول حرف، فأصبح `mor_2314` هو `Mor_2314`، ورفض السيرفر الدخول.

- `textVisiblePassword` : يجعل لوحة المفاتيح تعامل الحقل كنص حرفي، **فلا تكبر ولا تصحّ**.
- `textNoSuggestions` : يلغي شريط الاقتراحات.
- النتيجة: ما تكتبه يصل للسيرفر **حرفاً بحرف**، وهذا مهم لأن أسماء المستخدمين حساسة لحالة الأحرف.
- `error_text` : رسالة الخطأ، مخفية (gone) حتى يحدث خطأ. `?attr/colorError` يجعل لونها أحمر من الثيم.
- `CircularProgressIndicator` : دائرة التحميل، مخفية في البداية.
- `mock_backend_banner` : شريط ملوّن في أعلى الشاشة يقول إن التطبيق في **الوضع التجريبي**. مخفي إلا إذا كان السيرفر الوهمي مفعّلاً، حتى لا يظن أحد أن البيانات حقيقية.

39. `ui/login/LoginFragment.kt`

الدور: شاشة تسجيل الدخول (ال View).

app/src/main/java/com/example/fakestore/ui/login/LoginFragment.kt

68 lines

```

1 package com.example.fakestore.ui.login
2
3 import android.os.Bundle
4 import android.view.View
5 import android.view.inputmethod.EditorInfo
6 import androidx.core.view.isVisible
7 import androidx.fragment.app.Fragment
8 import androidx.fragment.app.viewModels
9 import androidx.navigation.fragment.findNavController
10 import com.example.fakestore.BuildConfig
11 import com.example.fakestore.R
12 import com.example.fakestore.databinding.FragmentLoginBinding
13 import com.example.fakestore.ui.common.collectWhenStarted
14 import dagger.hilt.android.AndroidEntryPoint
15
16 @AndroidEntryPoint
17 class LoginFragment : Fragment(R.layout.fragment_login) {
18
19     private val viewModel: LoginViewModel by viewModels()
20
21     private var _binding: FragmentLoginBinding? = null
22     private val binding get() = _binding!!
23
24     override fun onCreateView(view: View, savedInstanceState: Bundle?) {
25         super.onCreateView(view, savedInstanceState)
26         _binding = FragmentLoginBinding.bind(view)
27         binding.mockBackendBanner.isVisible = BuildConfig.USE MOCK_BACKEND
28
29         binding.loginButton.setOnClickListener { submit() }
30         binding.passwordInput.setOnEditorActionListener { _, actionId, _ →
31             if (actionId == EditorInfo.IME_ACTION_DONE) {
32                 submit()
33                 true
34             } else {
35                 false
36             }
37         }
38
39         collectWhenStarted(viewModel.state) { render(it) }
40     }

```

```

41
42     private fun submit() {
43         viewModel.login(
44             username = binding.usernameInput.text?.toString().orEmpty(),
45             password = binding.passwordInput.text?.toString().orEmpty(),
46         )
47     }
48
49     private fun render(state: LoginUiState) {
50         binding.progress.isVisible = state.isLoading
51         binding.loginButton.isEnabled = !state.isLoading
52         binding.usernameLayout.isEnabled = !state.isLoading
53         binding.passwordLayout.isEnabled = !state.isLoading
54
55         binding.errorText.isVisible = state.errorRes != null
56         state.errorRes?.let { binding.errorText.setText(it) }
57
58         val navController = findNavController()
59         if (state.isLoggedIn && navController.currentDestination?.id == R.id.loginFragment) {
60             navController.navigate(R.id.action_login_to_products)
61         }
62     }
63
64     override fun onDestroyView() {
65         super.onDestroyView()
66         _binding = null
67     }
68 }

```

الشرح

- الـ `Fragment(R.layout.fragment_login)` ينشئ واجهته من هذا الملف تلقائياً.
- `by viewModel()`: يحصل على الـ `ViewModel`، و `Hilt` يوفر له `AuthManager`. الـ `ViewModel` نفسه يعود بعد تدوير الشاشة.

نمط `binding` / `_binding`:

- واجهة الـ `Fragment` تُدَمَّر قبل الـ `Fragment` نفسه.
- لذلك نجعل `_binding = null` في `onDestroyView`، حتى لا نحفظ بواجهة ميتة في الذاكرة (**memory leak**).
- `binding` (مع `!!`) للاستخدام المريح بينهما.

`onViewCreated`:

- `mockBackendBanner.isVisible = BuildConfig.USE MOCK_BACKEND`: يُظهر شريط الوضع التجريبي فقط عند تفعيله.
- ربط الزر `Done` في لوحة المفاتيح بالدالة `submit()`.
- `collectWhenStarted(viewModel.state) { render(it) }`: كلما تغيّرت الحالة تُرسم الشاشة من جديد.

`render(state)`: الشاشة **تعكس** الحالة فقط:

- التحميل: تظهر الدائرة، ويُعطّل الزر والحقول.
- الخطأ: تظهر الرسالة.
- `isLoggedIn`: تنتقل للمنتجات.
- الشرط `currentDestination == loginFragment` يمنع تكرار الانتقال، وهو ما كان سيسبب `crash`.

الدور: واجهة خطأ قابلة لإعادة الاستخدام: رسالة + زر إعادة المحاولة.

app/src/main/res/layout/view_error.xml

24 lines

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <!-- Reusable error state: message + Retry button. Included in the products and detail screens.
→
3  <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
4      android:layout_width="match_parent"
5      android:layout_height="wrap_content"
6      android:layout_gravity="center"
7      android:gravity="center"
8      android:orientation="vertical"
9      android:padding="24dp">
10
11      <TextView
12          android:id="@+id/error_message"
13          android:layout_width="wrap_content"
14          android:layout_height="wrap_content"
15          android:gravity="center"
16          android:textAppearance="?attr/textAppearanceBodyLarge" />
17
18      <com.google.android.material.button.MaterialButton
19          android:id="@+id/retry_button"
20          android:layout_width="wrap_content"
21          android:layout_height="wrap_content"
22          android:layout_marginTop="16dp"
23          android:text="@string/retry" />
24  </LinearLayout>

```

الشرح

- تُضمَّن في شاشتي المنتجات والتفاصيل بالوسم `<include>` ، بدل تكرار نفس التصميم.
- `ViewBinding` يولّد لها `ViewErrorBinding` ، فنصل لعناصرها مثلاً عبر `binding.errorView.retryButton`.

الدور: منطق شاشة المنتجات.

app/src/main/java/com/example/fakestore/ui/products/ProductsViewModel.kt

92 lines

```

1  package com.example.fakestore.ui.products
2
3  import androidx.annotation.StringRes
4  import androidx.lifecycle.ViewModel
5  import androidx.lifecycle.viewModelScope
6  import com.example.fakestore.data.auth.AuthManager
7  import com.example.fakestore.data.model.Product
8  import com.example.fakestore.data.repository.ProductRepository
9  import com.example.fakestore.ui.common.UiState
10 import com.example.fakestore.ui.common.toMessageRes
11 import dagger.hilt.android.lifecycle.HiltViewModel

```

```

12 import kotlinx.coroutines.CancellationException
13 import kotlinx.coroutines.async
14 import kotlinx.coroutines.awaitAll
15 import kotlinx.coroutines.channels.Channel
16 import kotlinx.coroutines.coroutineScope
17 import kotlinx.coroutines.flow.Flow
18 import kotlinx.coroutines.flow.MutableStateFlow
19 import kotlinx.coroutines.flow.StateFlow
20 import kotlinx.coroutines.flow.asStateFlow
21 import kotlinx.coroutines.flow.receiveAsFlow
22 import kotlinx.coroutines.launch
23 import javax.inject.Inject
24
25 /** One-time events for the products screen (navigation, snackbars). */
26 sealed interface ProductsEvent {
27     data object LoggedOut : ProductsEvent
28     data class ConcurrencyTestFinished(val requestCount: Int) : ProductsEvent
29     data class ShowError(@StringRes val messageRes: Int) : ProductsEvent
30 }
31
32 @HiltViewModel
33 class ProductsViewModel @Inject constructor(
34     private val repository: ProductRepository,
35     private val authManager: AuthManager,
36 ) : ViewModel() {
37
38     private val _state = MutableStateFlow<UiState<List<Product>>>>(UiState.Loading)
39     val state: StateFlow<UiState<List<Product>>>> = _state.asStateFlow()
40
41     // Channel: each event is delivered once, even if the screen is rotated.
42     private val _events = Channel<ProductsEvent>(Channel.BUFFERED)
43     val events: Flow<ProductsEvent> = _events.receiveAsFlow()
44
45     init {
46         loadProducts()
47     }
48
49     fun loadProducts() {
50         viewModelScope.launch {
51             _state.value = UiState.Loading
52             _state.value = try {
53                 UiState.Success(repository.getProducts())
54             } catch (e: CancellationException) {
55                 throw e
56             } catch (e: Exception) {
57                 UiState.Error(e.toMessageRes())
58             }
59         }
60     }
61
62     /**
63      * Demo for the reviewer: fires [CONCURRENT_REQUESTS] product requests at the same time.
64      * Wait > 60 s after login, run it, and Logcat shows a single `/auth/login`.
65      */
66     fun runConcurrencyTest() {
67         viewModelScope.launch {
68             val event = try {
69                 coroutineScope {
70                     (1..CONCURRENT_REQUESTS)
71                         .map { id → async { repository.getProduct(id) } }
72                         .awaitAll()
73

```

```

74         ProductsEvent.ConcurrencyTestFinished(CONCURRENT_REQUESTS)
75     } catch (e: CancellationException) {
76         throw e
77     } catch (e: Exception) {
78         ProductsEvent.ShowError(e.toMessageRes())
79     }
80     _events.send(event)
81 }
82 }
83
84 fun logout() {
85     authManager.logout()
86     viewModelScope.launch { _events.send(ProductsEvent.LoggedOut) }
87 }
88
89 private companion object {
90     const val CONCURRENT_REQUESTS = 5
91 }
92 }

```

الشرح

ProductsEvent : أحداث لمرة واحدة:

- **LoggedOut** : الانتقال لـ Login.
- **ConcurrencyTestFinished** : عرض رسالة نجاح اختبار التزامن.
- **ShowError** : عرض رسالة خطأ.

الحالة مقابل الأحداث:

النوع	السلوك
state	StateFlow
events	Channel

init { loadProducts() } : يبدأ التحميل فور إنشاء الـ ViewModel، مرة واحدة فقط وليس عند كل تدوير.

loadProducts() : Loading، ثم Success أو Error.

- الطلب يمر تلقائياً بالـ **AuthInterceptor** ، فلا يوجد أي ذكر للـ token هنا.
- **هذا هدف التصميم كله**: الـ ViewModel لا يعرف شيئاً عن الـ tokens.

★ **runConcurrencyTest()** : أداة لإثبات متطلب التزامن للمراجع

- **coroutineScope { ... }** : ينتظر انتهاء كل الطلبات بداخله.
- **async { repository.getProduct(id) }** : يطلق طلباً دون انتظار انتهائه، فتنتقل الطلبات الخمسة في نفس اللحظة.

اللحظة.

- **awaitAll()** : ينتظر الخمسة معاً.
- **طريقة العرض**: سجّل الدخول، وانتظر أكثر من 60 ثانية، ثم اضغط "Run concurrency test". في Logcat سيظهر طلب login واحد فقط ثم 5 طلبات منتجات.

logout() : يسمح الجلسة، ثم يرسل حدث **LoggedOut** لتنتقل الشاشة إلى Login.

app/src/main/res/layout/item_product.xml

57 lines

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <!-- One row of the RecyclerView. Rows are recycled: the same views show different products. -->
3  <com.google.android.material.card.MaterialCardView
xmlns:android="http://schemas.android.com/apk/res/android"
4      android:layout_width="match_parent"
5      android:layout_height="wrap_content"
6      android:layout_margin="6dp"
7      android:clickable="true"
8      android:focusable="true">
9
10     <LinearLayout
11         android:layout_width="match_parent"
12         android:layout_height="wrap_content"
13         android:gravity="center_vertical"
14         android:orientation="horizontal"
15         android:padding="12dp">
16
17         <ImageView
18             android:id="@+id/image"
19             android:layout_width="72dp"
20             android:layout_height="72dp"
21             android:background="@android:color/white"
22             android:contentDescription="@string/product_image"
23             android:padding="4dp"
24             android:scaleType="fitCenter" />
25
26         <LinearLayout
27             android:layout_width="0dp"
28             android:layout_height="wrap_content"
29             android:layout_marginStart="12dp"
30             android:layout_weight="1"
31             android:orientation="vertical">
32
33             <TextView
34                 android:id="@+id/title"
35                 android:layout_width="match_parent"
36                 android:layout_height="wrap_content"
37                 android:ellipsize="end"
38                 android:maxLines="2"
39                 android:textAppearance="?attr/textAppearanceTitleSmall" />
40
41             <TextView
42                 android:id="@+id/category"
43                 android:layout_width="match_parent"
44                 android:layout_height="wrap_content"
45                 android:layout_marginTop="2dp"
46                 android:textAppearance="?attr/textAppearanceBodySmall" />
47
48             <TextView
49                 android:id="@+id/price"
50                 android:layout_width="wrap_content"
51                 android:layout_height="wrap_content"
52                 android:layout_marginTop="4dp"

```

```

53         android:textAppearance="?attr/textAppearanceTitleMedium"
54         android:textColor="?attr/colorPrimary" />
55     </LinearLayout>
56 </LinearLayout>
57 </com.google.android.material.card.MaterialCardView>

```

الشرح

- `MaterialCardView` : بطاقة لها ظل وزوايا دائرية، وقابلة للضغط (`clickable`).
- **الصف أفقي**: صورة 72dp، ثم عمود نصوص (العنوان، والتصنيف، والسعر).
- `layout_weight="1"` + `layout_width="0dp"` : عمود النصوص يأخذ كل المساحة المتبقية بجانب الصورة.
- `ellipsize="end"` + `maxLines="2"` : العنوان الطويل يُقصر بـ "...".

★ 43. ui/products/ProductAdapter.kt

الدور: يربط قائمة المنتجات بال `RecyclerView`. هذا يتطلب `"RecyclerView"`.

app/src/main/java/com/example/fakestore/ui/products/ProductAdapter.kt

56 lines

```

1  package com.example.fakestore.ui.products
2
3  import android.view.LayoutInflater
4  import android.view.ViewGroup
5  import androidx.recyclerview.widget.DiffUtil
6  import androidx.recyclerview.widget.ListAdapter
7  import androidx.recyclerview.widget.RecyclerView
8  import coil.load
9  import com.example.fakestore.data.model.Product
10 import com.example.fakestore.databinding.ItemProductBinding
11 import com.example.fakestore.ui.common.formatPrice
12
13 /**
14  * RecyclerView adapter. [ListAdapter] + [DiffUtil] compute what changed between two lists
15  * on a background thread and update only those rows.
16  */
17 class ProductAdapter(
18     private val onProductClick: (Product) → Unit,
19 ) : ListAdapter<Product, ProductAdapter.ProductViewHolder>(ProductDiff) {
20
21     override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ProductViewHolder {
22         val binding = ItemProductBinding.inflate(LayoutInflater.from(parent.context), parent,
false)
23         return ProductViewHolder(binding, onProductClick)
24     }
25
26     override fun onBindViewHolder(holder: ProductViewHolder, position: Int) {
27         holder.bind(getItem(position))
28     }
29
30     /** Holds the views of one row so they can be recycled for other products while scrolling.
*/
31     class ProductViewHolder(
32         private val binding: ItemProductBinding,
33         onProductClick: (Product) → Unit,
34     ) : RecyclerView.ViewHolder(binding.root) {

```



```

35
36     private var product: Product? = null
37
38     init {
39         // Set the listener once per holder, not on every bind.
40         binding.root.setOnClickListener { product?.let(onProductClick) }
41     }
42
43     fun bind(product: Product) {
44         this.product = product
45         binding.title.text = product.title
46         binding.category.text = product.category
47         binding.price.text = product.price.formatPrice()
48         binding.image.load(product.imageUrl) { crossfade(true) }
49     }
50 }
51
52 private object ProductDiff : DiffUtil.ItemCallback<Product>() {
53     override fun areItemsTheSame(oldItem: Product, newItem: Product) = oldItem.id ==
54     newItem.id
55     override fun areContentsTheSame(oldItem: Product, newItem: Product) = oldItem == newItem
56 }

```

الشرح

كيف يعمل RecyclerView؟ القائمة فيها 20 منتجاً والشاشة تعرض 7 فقط. RecyclerView ينشئ **نحو 9 صفوف** فقط. عند التمرير، الصف الذي يخرج من الأعلى يُعاد استخدامه (**recycled**) لعرض منتج جديد في الأسفل. هذا يوفر الذاكرة ويجعل التمرير سلساً.

: onCreateViewHolder

- يُنشئ صفّاً جديداً من `item_product.xml`.
- يُستدعى **قليلاً**، بعدد الصفوف الظاهرة تقريباً.

: onBindViewHolder

- يضع بيانات منتج على صف موجود.
- يُستدعى **كثيراً**، مع كل تمرير.

ProductViewHolder : يمسك عناصر الصف (عبر binding) لإعادة استخدامها.

- **مستمع الضغط يُضبط مرة واحدة** في `init` ، وليس في كل `bind` ، فهذا أكفأ.
- يقرأ المنتج الحالي من المتغير `product` .
- `binding.image.load(url)` : مكتبة **Coil** تحمّل الصورة من الإنترنت في الخلفية، وتخزّنها مؤقتاً (**cache**)، وتعرضها بتأثير تلاشي (`crossfade`).

: DiffUtil + ListAdapter

- عند وصول قائمة جديدة عبر `submitList` ، يقارن `DiffUtil` القديمة بالجديدة **في الخلفية**، ويحدّث فقط الصفوف التي تغيّرت، مع حركة.
- `areItemsTheSame` : نفس المنتج؟ يقارن الـ `id` .
- `areContentsTheSame` : هل تغيّرت بياناته؟ يقارن كل الحقول عبر `equals` الخاصة بـ `data class` .

الدور: تصميم شاشة القائمة.

app/src/main/res/layout/fragment_products.xml

26 lines

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
3      xmlns:app="http://schemas.android.com/apk/res-auto"
4      android:layout_width="match_parent"
5      android:layout_height="match_parent">
6
7      <androidx.recyclerview.widget.RecyclerView
8          android:id="@+id/products_list"
9          android:layout_width="match_parent"
10         android:layout_height="match_parent"
11         android:clipToPadding="false"
12         android:padding="8dp"
13         app:layoutManager="androidx.recyclerview.widget.LinearLayoutManager" />
14
15     <com.google.android.material.progressindicator.CircularProgressIndicator
16         android:id="@+id/progress"
17         android:layout_width="wrap_content"
18         android:layout_height="wrap_content"
19         android:layout_gravity="center"
20         android:indeterminate="true" />
21
22     <include
23         android:id="@+id/error_view"
24         layout="@layout/view_error"
25         android:visibility="gone" />
26 </FrameLayout>

```

الشرح

- `FrameLayout` : العناصر فوق بعضها، ونُظهر واحداً حسب الحالة: القائمة، أو التحميل، أو الخطأ.
- `app:layoutManager="LinearLayoutManager"` : ترتيب الصفوف عمودياً.
- `clipToPadding="false"` : الصفوف تُرسم داخل الهامش أثناء التمرير، وهذا يعطي شكلاً أجمل.

الدور: القائمة العلوية (:) في شاشة المنتجات.

app/src/main/res/menu/menu_products.xml

19 lines

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <menu xmlns:android="http://schemas.android.com/apk/res/android"
3      xmlns:app="http://schemas.android.com/apk/res-auto">
4
5      <item
6          android:id="@+id/action_refresh"
7          android:title="@string/menu_refresh"
8          app:showAsAction="never" />
9

```

```

10     <item
11         android:id="@+id/action_concurrency_test"
12         android:title="@string/menu_concurrency_test"
13         app:showAsAction="never" />
14
15     <item
16         android:id="@+id/action_logout"
17         android:title="@string/menu_logout"
18         app:showAsAction="never" />
19 </menu>

```

الشرح

ثلاثة خيارات داخل القائمة المنسدلة (`showAsAction="never"`):

- **Refresh**: إعادة تحميل المنتجات.
- **Run concurrency test**: اختبار التزامن.
- **Log out**: تسجيل الخروج.

46. `ui/products/ProductsFragment.kt`

الدور: شاشة قائمة المنتجات.

app/src/main/java/com/example/fakestore/ui/products/ProductsFragment.kt

106 lines

```

1  package com.example.fakestore.ui.products
2
3  import android.os.Bundle
4  import android.view.Menu
5  import android.view.MenuInflater
6  import android.view.MenuItem
7  import android.view.View
8  import androidx.core.os.bundleOf
9  import androidx.core.view.MenuProvider
10 import androidx.core.view.isVisible
11 import androidx.fragment.app.Fragment
12 import androidx.fragment.app.viewModels
13 import androidx.lifecycle.Lifecycle
14 import androidx.navigation.fragment.findNavController
15 import com.example.fakestore.R
16 import com.example.fakestore.data.model.Product
17 import com.example.fakestore.databinding.FragmentProductsBinding
18 import com.example.fakestore.ui.common.UiState
19 import com.example.fakestore.ui.common.collectWhenStarted
20 import com.example.fakestore.ui.detail.ProductDetailViewModel
21 import com.google.android.material.snackbar.Snackbar
22 import dagger.hilt.android.AndroidEntryPoint
23
24 @AndroidEntryPoint
25 class ProductsFragment : Fragment(R.layout.fragment_products) {
26
27     private val viewModel: ProductsViewModel by viewModels()
28
29     private var _binding: FragmentProductsBinding? = null
30     private val binding get() = _binding!!
31

```

```

32     private val adapter = ProductAdapter { product →
33         findNavController().navigate(
34             R.id.action_products_to_detail,
35             bundleOf(ProductDetailViewModel.ARG_PRODUCT_ID to product.id),
36         )
37     }
38
39     override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
40         super.onViewCreated(view, savedInstanceState)
41         _binding = FragmentProductsBinding.bind(view)
42
43         binding.productsList.adapter = adapter
44         binding.errorView.retryButton.setOnClickListener { viewModel.loadProducts() }
45         setUpMenu()
46
47         collectWhenStarted(viewModel.state) { render(it) }
48         collectWhenStarted(viewModel.events) { handle(it) }
49     }
50
51     private fun render(state: UiState<List<Product>>) {
52         binding.progress.isVisible = state is UiState.Loading
53         binding.productsList.isVisible = state is UiState.Success
54         binding.errorView.root.isVisible = state is UiState.Error
55
56         when (state) {
57             is UiState.Success → adapter.submitList(state.data)
58             is UiState.Error → binding.errorView.errorMessage.setText(state.messageRes)
59             UiState.Loading → Unit
60         }
61     }
62
63     private fun handle(event: ProductsEvent) {
64         when (event) {
65             ProductsEvent.LoggedOut → findNavController().navigate(R.id.action_global_login)
66             is ProductsEvent.ConcurrencyTestFinished → Snackbar.make(
67                 binding.root,
68                 getString(R.string.concurrency_test_done, event.requestCount),
69                 Snackbar.LENGTH_LONG,
70             ).show()
71             is ProductsEvent.ShowError → Snackbar.make(
72                 binding.root, event.messageRes, Snackbar.LENGTH_LONG,
73             ).show()
74         }
75     }
76
77     private fun setUpMenu() {
78         requireActivity().addMenuProvider(object : MenuProvider {
79             override fun onCreateMenu(menu: Menu, menuInflater: MenuInflater) {
80                 menuInflater.inflate(R.menu.menu_products, menu)
81             }
82
83             override fun onMenuItemSelected(menuItem: MenuItem): Boolean = when
(menuItem.itemId) {
84                 R.id.action_refresh → {
85                     viewModel.loadProducts()
86                     true
87                 }
88                 R.id.action_concurrency_test → {
89                     viewModel.runConcurrencyTest()
90                     true
91                 }
92                 R.id.action_logout → {

```

```

93         viewModel.logout()
94         true
95     }
96     else → false
97 }
98 }, viewLifecycleOwner, Lifecycle.State.RESUMED)
99 }
100
101 override fun onDestroyView() {
102     super.onDestroyView()
103     binding.productsList.adapter = null // the adapter outlives the view: avoid leaking it
104     _binding = null
105 }
106 }

```

الشرح

adapter : يُنشأ مرة واحدة. عند الضغط على منتج نفتح التفاصيل مع رقمه:

```
bundleOf("productId" to product.id)
```

: **render(state)**

- **isVisible** : يُظهر عنصراً واحداً فقط حسب الحالة.
- **adapter.submitList(...)** : Success
- **Error** : نضع نص الخطأ.

: **handle(event)**

- **LoggedOut** : الانتقال لـ Login مع مسح كل الشاشات.
- **ShowError** / **ConcurrencyTestFinished** : رسالة **Snackbar** أسفل الشاشة.

: **setUpMenu()**

- **MenuProvider** هو الطريقة الحديثة لإضافة قائمة من **Fragment**.
- **viewLifecycleOwner, RESUMED** : القائمة تظهر فقط والشاشة نشطة، وتُزال تلقائياً عند مغادرتها.

: **onDestroyView**

- **productsList.adapter = null** : الـ adapter يعيش أطول من الواجهة، فنفصله عنها لتجنب تسريب الذاكرة.
- ثم **_binding = null**.

47. ui/detail/ProductDetailViewModel.kt

الدور: منطق شاشة التفاصيل.

```
app/src/main/java/com/example/fakestore/ui/detail/ProductDetailViewModel.kt
```

53 lines

```

1 package com.example.fakestore.ui.detail
2
3 import androidx.lifecycle.SavedStateHandle
4 import androidx.lifecycle.ViewModel
5 import androidx.lifecycle.viewModelScope

```

```

6 import com.example.fakestore.data.model.Product
7 import com.example.fakestore.data.repository.ProductRepository
8 import com.example.fakestore.ui.common.UiState
9 import com.example.fakestore.ui.common.toMessageRes
10 import dagger.hilt.android.lifecycle.HiltViewModel
11 import kotlinx.coroutines.CancellationException
12 import kotlinx.coroutines.flow.MutableStateFlow
13 import kotlinx.coroutines.flow.StateFlow
14 import kotlinx.coroutines.flow.asStateFlow
15 import kotlinx.coroutines.launch
16 import javax.inject.Inject
17
18 @HiltViewModel
19 class ProductDetailViewModel @Inject constructor(
20     savedStateHandle: SavedStateHandle,
21     private val repository: ProductRepository,
22 ) : ViewModel() {
23
24     // The navigation argument arrives through SavedStateHandle and survives process death.
25     private val productId: Int = checkNotNull(savedStateHandle[ARG_PRODUCT_ID]) {
26         "ProductDetailViewModel requires the '$ARG_PRODUCT_ID' argument"
27     }
28
29     private val _state = MutableStateFlow<UiState<Product>>(UiState.Loading)
30     val state: StateFlow<UiState<Product>> = _state.asStateFlow()
31
32     init {
33         loadProduct()
34     }
35
36     fun loadProduct() {
37         viewModelScope.launch {
38             _state.value = UiState.Loading
39             _state.value = try {
40                 UiState.Success(repository.getProduct(productId))
41             } catch (e: CancellationException) {
42                 throw e
43             } catch (e: Exception) {
44                 UiState.Error(e.toMessageRes())
45             }
46         }
47     }
48
49     companion object {
50         /** Must match the `<argument android:name>` in nav_graph.xml. */
51         const val ARG_PRODUCT_ID = "productId"
52     }
53 }

```

الشرح

- **SavedStateHandle** : يحمل الـ arguments القادمة من Navigation، أي `productId`.
- **ميزته**: يبقى حتى لو أغلق النظام التطبيق في الخلفية لتوفير الذاكرة (**process death**) ثم أعاده المستخدم.
- `checkNotNull(...)` : إذا لم يصل الرقم، فهذا خطأ برمجي، فنفسل فوراً برسالة واضحة.
- `ARG_PRODUCT_ID = "productId"` :
- يجب أن يطابق اسم الـ argument في `nav_graph.xml`.

- ثابت واحد يستخدمه الطرفان، فلا يحدث خطأ إملائي.

`loadProduct()` : نفس نمط القائمة. وطلب المنتج أيضاً يمر بال `Interceptor` تلقائياً.

48. `res/layout/fragment_product_detail.xml`

الدور: تصميم شاشة التفاصيل.

app/src/main/res/layout/fragment_product_detail.xml

77 lines

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
3      android:layout_width="match_parent"
4      android:layout_height="match_parent">
5
6      <androidx.core.widget.NestedScrollView
7          android:id="@+id/content"
8          android:layout_width="match_parent"
9          android:layout_height="match_parent"
10         android:visibility="gone">
11
12         <LinearLayout
13             android:layout_width="match_parent"
14             android:layout_height="wrap_content"
15             android:orientation="vertical"
16             android:padding="16dp">
17
18             <ImageView
19                 android:id="@+id/image"
20                 android:layout_width="match_parent"
21                 android:layout_height="260dp"
22                 android:background="@android:color/white"
23                 android:contentDescription="@string/product_image"
24                 android:padding="16dp"
25                 android:scaleType="fitCenter" />
26
27             <TextView
28                 android:id="@+id/title"
29                 android:layout_width="match_parent"
30                 android:layout_height="wrap_content"
31                 android:layout_marginTop="16dp"
32                 android:textAppearance="?attr/textAppearanceTitleLarge" />
33
34             <TextView
35                 android:id="@+id/price"
36                 android:layout_width="wrap_content"
37                 android:layout_height="wrap_content"
38                 android:layout_marginTop="8dp"
39                 android:textAppearance="?attr/textAppearanceHeadlineSmall"
40                 android:textColor="?attr/colorPrimary" />
41
42             <TextView
43                 android:id="@+id/category"
44                 android:layout_width="wrap_content"
45                 android:layout_height="wrap_content"
46                 android:layout_marginTop="8dp"
47                 android:textAppearance="?attr/textAppearanceLabelLarge" />
48
```

```

49         <TextView
50             android:id="@+id/rating"
51             android:layout_width="wrap_content"
52             android:layout_height="wrap_content"
53             android:layout_marginTop="4dp"
54             android:textAppearance="?attr/textAppearanceBodyMedium" />
55
56         <TextView
57             android:id="@+id/description"
58             android:layout_width="match_parent"
59             android:layout_height="wrap_content"
60             android:layout_marginTop="16dp"
61             android:lineSpacingMultiplier="1.2"
62             android:textAppearance="?attr/textAppearanceBodyLarge" />
63     </LinearLayout>
64 </androidx.core.widget.NestedScrollView>
65
66 <com.google.android.material.progressindicator.CircularProgressIndicator
67     android:id="@+id/progress"
68     android:layout_width="wrap_content"
69     android:layout_height="wrap_content"
70     android:layout_gravity="center"
71     android:indeterminate="true" />
72
73 <include
74     android:id="@+id/error_view"
75     layout="@layout/view_error"
76     android:visibility="gone" />
77 </FrameLayout>

```

الشرح

- `NestedScrollView` (معرفه `content`): المحتوى قابل للتمرير، لأن الوصف قد يكون طويلاً. مخفي حتى تصل البيانات.
- المحتوى بالترتيب: صورة كبيرة 260dp، ثم العنوان، والسعر (بلون الثيم الأساسي)، والتصنيف، والتقييم، والوصف.
- `lineSpacingMultiplier="1.2"`: مسافة أكبر بين الأسطر لتسهيل قراءة الوصف.

49. ui/detail/ProductDetailFragment.kt

الدور: شاشة التفاصيل.

app/src/main/java/com/example/fakestore/ui/detail/ProductDetailFragment.kt

58 lines

```

1 package com.example.fakestore.ui.detail
2
3 import android.os.Bundle
4 import android.view.View
5 import androidx.core.view.isVisible
6 import androidx.fragment.app.Fragment
7 import androidx.fragment.app.viewModels
8 import coil.load
9 import com.example.fakestore.R
10 import com.example.fakestore.data.model.Product
11 import com.example.fakestore.databinding.FragmentProductDetailBinding
12 import com.example.fakestore.ui.common.UiState
13 import com.example.fakestore.ui.common.collectWhenStarted
14 import com.example.fakestore.ui.common.formatPrice

```



```

15 import dagger.hilt.android.AndroidEntryPoint
16
17 @AndroidEntryPoint
18 class ProductDetailFragment : Fragment(R.layout.fragment_product_detail) {
19
20     private val viewModel: ProductDetailViewModel by viewModels()
21
22     private var _binding: FragmentProductDetailBinding? = null
23     private val binding get() = _binding!!
24
25     override fun onCreateView(view: View, savedInstanceState: Bundle?) {
26         super.onCreateView(view, savedInstanceState)
27         _binding = FragmentProductDetailBinding.bind(view)
28
29         binding.errorView.retryButton.setOnClickListener { viewModel.loadProduct() }
30         collectWhenStarted(viewModel.state) { render(it) }
31     }
32
33     private fun render(state: UiState<Product>) {
34         binding.progress.isVisible = state is UiState.Loading
35         binding.content.isVisible = state is UiState.Success
36         binding.errorView.root.isVisible = state is UiState.Error
37
38         when (state) {
39             is UiState.Success → showProduct(state.data)
40             is UiState.Error → binding.errorView.errorMessage.setText(state.messageRes)
41             UiState.Loading → Unit
42         }
43     }
44
45     private fun showProduct(product: Product) {
46         binding.image.load(product.imageUrl) { crossfade(true) }
47         binding.title.text = product.title
48         binding.price.text = product.price.formatPrice()
49         binding.category.text = product.category
50         binding.rating.text = getString(R.string.product_rating, product.rating,
product.ratingCount)
51         binding.description.text = product.description
52     }
53
54     override fun onDestroyView() {
55         super.onDestroyView()
56         _binding = null
57     }
58 }

```

الشرح

- نفس نمط شاشة القائمة: binding، ومراقبة الحالة، و render .
- showProduct() : يملأ العناصر. التقييم يُبنى من مورد نصي بقيمتين:

```
getString(R.string.product_rating, rating, count) → "★ 4.1 (259 reviews)"
```

- سهم الرجوع في الشريط العلوي يعمل تلقائياً، لأن التفاصيل ليست شاشة رئيسية في AppBarConfiguration

الجزء السابع: الاختبارات (Unit Tests)

مكان الاختبارات: المجلد `app/src/test/` . تعمل على الكمبيوتر مباشرة (JVM)، بلا هاتف أو محاكي. طريقة التشغيل: الأمر `./gradlew test` ، أو بزر ▶ بجانب أي اختبار في Android Studio.

النتيجة: 21 اختباراً، كلها ناجحة.

- شُغِّلَت 5 مرات متتالية للتأكد من أن اختبارات التزامن ثابتة وغير عشوائية.
- وأُثْبِت أنها تكشف الخطأ فعلاً: عند حذف التحقق الثاني داخل القفل، فشل اختباران:
 - 10 طلبات متزامنة أنتجت 10 طلبات login بدل 1.
 - 5 طلبات Retrofit متوازية أنتجت 5 طلبات login بدل 1.

50. `test/.../TestDoubles.kt`

الدور: بدائل وهمية (Test Doubles) للاختبار.

app/src/test/java/com/example/fakestore/data/auth/TestDoubles.kt

106 lines

```
1 package com.example.fakestore.data.auth
2
3 import com.example.fakestore.data.remote.AuthApi
4 import com.example.fakestore.data.remote.ProductApi
5 import okhttp3.OkHttpClient
6 import okhttp3.mockwebserver.Dispatcher
7 import okhttp3.mockwebserver.MockResponse
8 import okhttp3.mockwebserver.MockWebServer
9 import okhttp3.mockwebserver.RecordedRequest
10 import retrofit2.Retrofit
11 import retrofit2.converter.gson.GsonConverterFactory
12 import java.util.concurrent.atomic.AtomicInteger
13
14 /** Clock whose time only moves when the test says so. */
15 class FakeClock(var nowMillis: Long = 1_000_000L) : Clock {
16     override fun nowMillis(): Long = nowMillis
17     fun advanceBy(millis: Long) {
18         nowMillis += millis
19     }
20 }
21
22 /** Thread-safe in-memory replacement for the encrypted storage. */
23 class InMemoryTokenStorage(initial: AuthSession? = null) : TokenStorage {
24     @Volatile
25     var session: AuthSession? = initial
26
27     override fun read(): AuthSession? = session
28     override fun save(session: AuthSession) {
29         this.session = session
30     }
31     override fun clear() {
32         session = null
33     }
34 }
```

```

34 }
35
36 /**
37  * Local HTTP server that imitates FakeStore.
38  * - `POST /auth/login` returns `token-1`, `token-2`, ... (or [loginResponseCode] if not 200).
39  * - `GET /products/...` returns [productsResponseCode] with a small JSON body.
40  */
41 class FakeStoreServer {
42     val server = MockWebServer()
43     val loginCount = AtomicInteger()
44     val productRequests = AtomicInteger()
45
46     @Volatile var loginResponseCode = 200
47     @Volatile var loginDelayMillis = 0L
48     @Volatile var productsResponseCode = 200
49
50     private val seenAuthHeaders = java.util.concurrent.ConcurrentLinkedQueue<String?>()
51     val productAuthHeaders: List<String?> get() = seenAuthHeaders.toList()
52
53     init {
54         server.dispatcher = object : Dispatcher() {
55             override fun dispatch(request: RecordedRequest): MockResponse {
56                 val path = request.path.orEmpty()
57                 return when {
58                     path.startsWith("/auth/login") → {
59                         val n = loginCount.incrementAndGet()
60                         if (loginDelayMillis > 0) Thread.sleep(loginDelayMillis)
61                         if (loginResponseCode == 200) {
62                             MockResponse().setBody("""{"token":"token-$n"}""")
63                         } else {
64                             MockResponse().setResponseCode(loginResponseCode)
65                                 .setBody("username or password is incorrect")
66                         }
67                     }
68                     path.startsWith("/products") → {
69                         productRequests.incrementAndGet()
70                         seenAuthHeaders.add(request.getHeader(AuthHeaders.AUTHORIZATION))
71                         MockResponse().setResponseCode(productsResponseCode).setBody(
72                             if (path == "/products") "[]"
73                             else """
74 {"id":1,"title":"Bag","price":10.5,"description":"d","category":"c","image":"i","rating":
75 {"rate":4.1,"count":7}}"""
76
77                     )
78                 }
79             }
80             else → MockResponse().setResponseCode(404)
81         }
82         server.start()
83     }
84
85     private fun retrofit(client: OkHttpClient): Retrofit = Retrofit.Builder()
86         .baseUrl(server.url("/"))
87         .client(client)
88         .addConverterFactory(GsonConverterFactory.create())
89         .build()
90
91     /** Same wiring as NetworkModule: login on a plain client, products on an authenticated one.
92     */
93     fun authApi(): AuthApi = retrofit(OkHttpClient()).create(AuthApi::class.java)
94
95     fun productApi(authManager: AuthManager): ProductApi {

```

```

93         val client = OkHttpClient.Builder()
94             .addInterceptor(AuthInterceptor(authManager))
95             .authenticator(TokenAuthenticator(authManager))
96             .build()
97         return retrofit(client).create(ProductApi::class.java)
98     }
99
100     fun shutdown() = server.shutdown()
101 }
102
103 val TEST_CREDENTIALS = Credentials("mor_2314", "83r5^_")
104
105 fun sessionSavedAt(clock: FakeClock, token: String = "token-0") =
106     AuthSession(token, clock.nowMillis, TEST_CREDENTIALS)

```

الشرح

FakeClock: ساعة لا تتحرك إلا بأمر منا.

- `advanceBy(61_000)` = "مرت 61 ثانية" فوراً.

InMemoryTokenStorage: تخزين في الذاكرة بدل التشفير.

- `@Volatile`: يضمن أن كل الـ threads ترى آخر قيمة مكتوبة.

FakeStoreServer: يستخدم **MockWebServer**، وهو سيرفر HTTP حقيقي يعمل على الكمبيوتر محلياً ويقلد **FakeStore**:

- `/auth/login`: يرد `token-1` ثم `token-2` ... ويعدّ كم مرة استُدعي (`loginCount`).

- `/products`: يرد بمنتجات، ويسجّل الـ header الذي وصله (`productAuthHeaders`).

- `loginDelayMillis`: يؤخر رد `login`، ليبقى التجديد "جارياً" بينما تصل الطلبات الأخرى. **ضروري لاختبار التزامن بصدق.**

- `productsResponseCode` / `loginResponseCode`: لمحاكاة الرفض (401).

`productApi(authManager)`: ييني عميلاً محمياً بنفس طريقة `NetworkModule`، فنختبر التركيب الحقيقي نفسه.

51. test/.../AuthManagerTest.kt

الدور: اختبار منطق `AuthManager` مباشرة.

app/src/test/java/com/example/fakestore/data/auth/AuthManagerTest.kt

193 lines

```

1 package com.example.fakestore.data.auth
2
3 import kotlinx.coroutines.ExperimentalCoroutinesApi
4 import kotlinx.coroutines.flow.toList
5 import kotlinx.coroutines.launch
6 import kotlinx.coroutines.test.UnconfinedTestDispatcher
7 import kotlinx.coroutines.test.runTest
8 import org.junit.After
9 import org.junit.Assert.assertEquals
10 import org.junit.Assert.assertNotNull
11 import org.junit.Assert.assertNull
12 import org.junit.Assert.assertTrue
13 import org.junit.Assert.fail
14 import org.junit.Test
15 import java.io.IOException

```

```

16 import java.util.concurrent.CountDownLatch
17 import java.util.concurrent.Executors
18 import java.util.concurrent.TimeUnit
19
20 @OptIn(ExperimentalCoroutinesApi::class)
21 class AuthManagerTest {
22
23     private val server = FakeStoreServer()
24     private val clock = FakeClock()
25     private val storage = InMemoryTokenStorage()
26     private val authManager = AuthManager(server.authApi(), storage, clock)
27
28     @After
29     fun tearDown() = server.shutdown()
30
31     @Test
32     fun `login stores token and tokenSavedAt`() = runTest {
33         authManager.login("mor_2314", "83r5^_")
34
35         val session = storage.session!!
36         assertEquals("token-1", session.token)
37         assertEquals(clock.nowMillis, session.savedAtMillis)
38         assertTrue(authManager.isLoggedIn())
39     }
40
41     @Test
42     fun `login with wrong password throws InvalidCredentialsException`() = runTest {
43         server.loginResponseCode = 401
44         try {
45             authManager.login("mor_2314", "wrong")
46             fail("Expected InvalidCredentialsException")
47         } catch (e: InvalidCredentialsException) {
48             assertEquals(401, e.httpCode)
49         }
50         assertNull(storage.session)
51     }
52
53     @Test
54     fun `valid token is returned without any network call`() {
55         storage.session = sessionSavedAt(clock)
56         clock.advanceBy(59_000)
57
58         assertEquals("token-0", authManager.getValidToken())
59         assertEquals(0, server.loginCount.get())
60     }
61
62     @Test
63     fun `token is treated as expired after exactly 60 seconds and refreshed`() {
64         storage.session = sessionSavedAt(clock)
65         clock.advanceBy(AuthManager.TOKEN_LIFETIME_MS)
66
67         assertEquals("token-1", authManager.getValidToken())
68         assertEquals(1, server.loginCount.get())
69         assertEquals(clock.nowMillis, storage.session!!.savedAtMillis)
70     }
71
72     @Test
73     fun `clock moving backwards is treated as expired`() {
74         storage.session = sessionSavedAt(clock)
75         clock.advanceBy(-1)
76
77         assertEquals("token-1", authManager.getValidToken())

```

```

78     }
79
80     @Test
81     fun `concurrent callers with an expired token trigger exactly one refresh`() {
82         storage.session = sessionSavedAt(clock)
83         clock.advanceBy(61_000)
84         server.loginDelayMillis = 300 // keep the refresh in flight while others arrive
85
86         val threads = 10
87         val pool = Executors.newFixedThreadPool(threads)
88         val start = CountDownLatch(1)
89         val results = (1..threads).map {
90             pool.submit<String> {
91                 start.await()
92                 authManager.getValidToken()
93             }
94         }
95         start.countDown()
96         val tokens = results.map { it.get(10, TimeUnit.SECONDS) }
97         pool.shutdown()
98
99         assertEquals(1, server.loginCount.get())
100        assertTrue(tokens.all { it == "token-1" })
101    }
102
103    @Test
104    fun `rejected refresh forces logout and emits SessionExpired`() = runTest {
105        val events = mutableList0f<AuthEvent>()
106        backgroundScope.launch(UnconfinedTestDispatcher(testScheduler)) {
107            authManager.events.toList(events)
108        }
109        storage.session = sessionSavedAt(clock)
110        clock.advanceBy(61_000)
111        server.loginResponseCode = 401
112
113        try {
114            authManager.getValidToken()
115            fail("Expected SessionExpiredException")
116        } catch (expected: SessionExpiredException) {
117        }
118
119        assertNull(storage.session)
120        assertEquals(list0f<AuthEvent>(AuthEvent.SessionExpired), events)
121        assertEquals(1, server.loginCount.get()) // no retry loop
122    }
123
124    @Test
125    fun `network failure during refresh keeps the session`() {
126        storage.session = sessionSavedAt(clock)
127        clock.advanceBy(61_000)
128        server.shutdown()
129
130        try {
131            authManager.getValidToken()
132            fail("Expected IOException")
133        } catch (e: IOException) {
134            assertTrue(e !is SessionExpiredException)
135        }
136        assertNotNull(storage.session)
137    }
138
139    @Test

```

```

140 fun `server outage during login throws ServerUnavailableException`() = runTest {
141     server.loginResponseCode = 521 // Cloudflare: "web server is down"
142     try {
143         authManager.login("mor_2314", "83r5^_")
144         fail("Expected ServerUnavailableException")
145     } catch (e: ServerUnavailableException) {
146         assertEquals(521, e.httpCode)
147     }
148     assertNull(storage.session)
149 }
150
151 @Test
152 fun `server outage during refresh keeps the session and does not log out`() {
153     storage.session = sessionSavedAt(clock)
154     clock.advanceBy(61_000)
155     server.loginResponseCode = 503
156
157     try {
158         authManager.getValidToken()
159         fail("Expected ServerUnavailableException")
160     } catch (expected: ServerUnavailableException) {
161     }
162     assertNotNull(storage.session)
163     assertEquals(1, server.loginCount.get()) // no retry loop
164 }
165
166 @Test
167 fun `no session means SessionExpiredException without network`() {
168     try {
169         authManager.getValidToken()
170         fail("Expected SessionExpiredException")
171     } catch (expected: SessionExpiredException) {
172     }
173     assertEquals(0, server.loginCount.get())
174 }
175
176 @Test
177 fun `logout during a refresh does not bring the session back`() {
178     storage.session = sessionSavedAt(clock)
179     clock.advanceBy(61_000)
180     server.loginDelayMillis = 300
181
182     val pool = Executors.newSingleThreadExecutor()
183     val refresh = pool.submit<Result<String>> { runCatching { authManager.getValidToken() } }
184
185     Thread.sleep(100) // refresh is now waiting for the server
186     authManager.logout()
187
188     val result = refresh.get(5, TimeUnit.SECONDS)
189     pool.shutdown()
190
191     assertTrue(result.exceptionOrNull() is SessionExpiredException)
192     assertNull(storage.session)
193 }

```

الاختبار	ماذا يثبت
login stores token and tokenSavedAt	الدخول يحفظ ال token مع وقت الحفظ
login with wrong password...	كلمة مرور خاطئة تعطي InvalidCredentialsException ، ولا يُحفظ شيء
valid token is returned without any network call	عند 59 ثانية: لا يوجد أي طلب شبكة
token is treated as expired after exactly 60 seconds	عند 60 ثانية بالضبط: تجديد، و tokenSavedAt جديد
clock moving backwards...	تغيير ساعة الجهاز للخلف يعني اعتبار ال token منتهياً
concurrent callers ... exactly one ★ refresh	10 threads معاً تنتج login واحداً، وكلها تحصل على نفس ال token
rejected refresh forces logout...	رفض التجديد يعني مسح الجلسة، وإرسال الحدث، ومحاولة واحدة فقط
network failure during refresh keeps the session	انقطاع الإنترنت لا يُخرج المستخدم
server outage during login...	تعطّل السيرفر (521) أثناء الدخول يعطي ServerUnavailableException ، لا "كلمة مرور خاطئة"
server outage during refresh keeps the session...	تعطّل السيرفر (503) أثناء التجديد لا يُخرج المستخدم، ومحاولة واحدة فقط
no session means SessionExpiredException...	بلا جلسة: خطأ دون أي طلب
logout during a refresh...	Logout أثناء التجديد لا يعيد الجلسة

كيف يعمل اختبار التزامن؟

1. Executors.newFixedThreadPool(10) : عشرة threads حقيقية.
 2. CountDownLatch(1) : "خط البداية". كل ال threads تنتظر عنده.
 3. start.countDown() : تنطلق العشرة في نفس اللحظة.
 4. التحقق: loginCount = 1 ، وكل النتائج تساوي token-1 .
- runTest : يشغل كود coroutines في الاختبار. backgroundScope : يستمع للأحداث في الخلفية، ويُلغى تلقائياً في نهاية الاختبار.

الدور: اختبار متكامل (integration test): Retrofit + OkHttp + Interceptor + AuthManager معاً، أمام سيرفر محلي.

app/src/test/java/com/example/fakestore/data/auth/AuthInterceptorTest.kt

69 lines

```

1  package com.example.fakestore.data.auth
2
3  import com.example.fakestore.data.remote.ProductApi
4  import kotlinx.coroutines.async
5  import kotlinx.coroutines.awaitAll
6  import kotlinx.coroutines.runBlocking
7  import org.junit.After
8  import org.junit.Assert.assertEquals
9  import org.junit.Assert.assertTrue
10 import org.junit.Assert.fail
11 import org.junit.Test
12
13 /** End-to-end: Retrofit + OkHttp + AuthInterceptor + AuthManager against a local server. */
14 class AuthInterceptorTest {
15
16     private val server = FakeStoreServer()
17     private val clock = FakeClock()
18     private val storage = InMemoryTokenStorage()
19     private val authManager = AuthManager(server.authApi(), storage, clock)
20     private val productApi: ProductApi = server.productApi(authManager)
21
22     @After
23     fun tearDown() = server.shutdown()
24
25     @Test
26     fun `every product request carries the bearer token`() = runBlocking {
27         storage.session = sessionSavedAt(clock, token = "abc")
28
29         productApi.getProducts()
30         productApi.getProduct(1)
31
32         assertEquals(listOf("Bearer abc", "Bearer abc"), server.productAuthHeaders)
33         assertEquals(0, server.loginCount.get())
34     }
35
36     @Test
37     fun `expired token is refreshed before the request is sent`() = runBlocking {
38         storage.session = sessionSavedAt(clock, token = "old")
39         clock.advanceBy(61_000)
40
41         productApi.getProducts()
42
43         assertEquals(1, server.loginCount.get())
44         assertEquals(listOf("Bearer token-1"), server.productAuthHeaders) // never "Bearer old"
45     }
46
47     @Test
48     fun `parallel requests with an expired token share one refresh`() = runBlocking {
49         storage.session = sessionSavedAt(clock, token = "old")
50         clock.advanceBy(61_000)
51         server.loginDelayMillis = 300
52
53         (1..5).map { async(kotlinx.coroutines.Dispatchers.IO) { productApi.getProduct(it) } }

```

```

}.awaitAll()
54
55     assertEquals(1, server.loginCount.get())
56     assertEquals(5, server.productRequests.get())
57     assertTrue(server.productAuthHeaders.all { it == "Bearer token-1" })
58 }
59
60 @Test
61 fun `without a session the request fails and never reaches the server`() = runBlocking {
62     try {
63         productApi.getProducts()
64         fail("Expected SessionExpiredException")
65     } catch (expected: SessionExpiredException) {
66     }
67     assertEquals(0, server.productRequests.get())
68 }
69 }

```

الشرح

الاختبار	ماذا يثبت
every product request carries the bearer token	كل طلب منتجات وصل للسيرفر ومعه Bearer abc
expired token is refreshed before the request is sent	السيرفر لم يستقبل أبداً Bearer old المنتهي، بل Bearer token-1 الجديد مباشرة
parallel requests ... share one refresh ★	5 طلبات Retrofit متوازية تنتج login واحداً، و5 طلبات منتجات، كلها بال token الجديد
without a session the request fails...	بلا جلسة: الطلب لا يصل للسيرفر أصلاً

53. test/.../TokenAuthenticatorTest.kt

الدور: إثبات عدم وجود حلقة لا نهائية عند رد 401.

app/src/test/java/com/example/fakestore/data/auth/TokenAuthenticatorTest.kt

38 lines

```

1 package com.example.fakestore.data.auth
2
3 import kotlinx.coroutines.runBlocking
4 import org.junit.After
5 import org.junit.Assert.assertEquals
6 import org.junit.Assert.assertNull
7 import org.junit.Test
8 import retrofit2.HttpException
9
10 /** The 401 safety net: refresh + retry once, never loop. */
11 class TokenAuthenticatorTest {
12
13     private val server = FakeStoreServer()
14     private val clock = FakeClock()
15     private val storage = InMemoryTokenStorage()

```

```

16 private val authManager = AuthManager(server.authApi(), storage, clock)
17 private val productApi = server.productApi(authManager)
18
19 @After
20 fun tearDown() = server.shutdown()
21
22 @Test
23 fun `server keeps answering 401 - one retry then forced logout`() = runBlocking {
24     storage.session = sessionSavedAt(clock, token = "rejected")
25     server.productsResponseCode = 401
26
27     try {
28         productApi.getProducts()
29     } catch (e: HttpException) {
30         assertEquals(401, e.code())
31     }
32
33     assertEquals(2, server.productRequests.get()) // original + exactly one retry
34     assertEquals(1, server.loginCount.get())
35     assertEquals(listOf("Bearer rejected", "Bearer token-1"), server.productAuthHeaders)
36     assertNull(storage.session)
37 }
38 }

```

الشرح

السيناريو: السيرفر يرد 401 دائماً.

النتيجة المثبتة:

- طلبان فقط: الأصلي + إعادة واحدة.
- طلب login واحد.
- الجلسة مُسحت (خروج إجباري).
- ال headers: الأول Bearer rejected ، والثاني Bearer token-1 .

بدون فحص priorResponse كانت المحاولات ستتكرر حتى حد OkHttp الداخلي (20 محاولة).

الجزء الثامن: الصورة الكاملة

54. رحلة طلب كاملة عبر الملفات

السيناريو: المستخدم سجّل دخوله قبل 70 ثانية، ثم ضغط على منتج.

#	أين	ماذا يحدث
1	ProductAdapter	الضغط يستدعي <code>navigate(action_products_to_detail, productId=3)</code>
2	nav_graph.xml	يفتح <code>ProductDetailFragment</code>
3	ProductDetailViewModel	يقرأ <code>productId</code> من <code>SavedStateHandle</code> ، ثم <code>loadProduct()</code> ، والحالة <code>Loading</code>
4	ProductRepositoryImpl	<code>api.getProduct(3)</code>
5	ProductApi (Retrofit)	يبنى طلب <code>GET /products/3</code> على العميل <code>@AuthClient</code>
6	AuthInterceptor	<code>authManager.isValidToken()</code>
7	AuthManager	المسار السريع: عمر الـ 70 token ثانية، منتبه
8	AuthManager	يدخل <code>refreshLock</code> ويتحقق مرة ثانية: ما زال منتهياً
9	AuthManager.refresh	<code>requestNewSession</code> يرسل <code>AuthApi.login</code> على العميل <code>@PlainClient</code> ، بلا
Interceptor		
10	EncryptedTokenStorage	يحفظ الـ token الجديد مشفراً مع <code>الآن = tokenSavedAt</code>
11	AuthInterceptor	يضيف <code><جديد> Authorization: Bearer</code> ويرسل الطلب
12	Gson	يحوّل JSON إلى <code>ProductDto</code> ، ثم <code>toDomain()</code> إلى <code>Product</code>
13	ProductDetailViewModel	الحالة <code>Success(product)</code>
14	ProductDetailFragment	<code>render</code> ثم <code>showProduct</code> ، فتظهر التفاصيل

ولو رفض السيرفر بيانات الدخول في الخطوة 9:

- `forceLogout()`: يسمح الجلسة ويرسل `SessionExpired`.
- يُرمى `SessionExpiredException` (وهو `IOException`)، فتعرض الشاشة "انتهت الجلسة".
- `MainActivity` تستقبل الحدث، فتعرض `Toast` وتنتقل لـ `Login` مع مسح كل الشاشات.

55. كيف تعرض المشروع للمدير (Demo)

- شغل التطبيق، وافتح نافذة **Logcat** في Android Studio، واكتب في البحث: `okhttp`

2. سجّل الدخول بـ `mor_2314` وكلمة المرور `83r5^_`. سيظهر: `POST https://fakestoreapi.com/auth/login`
3. تصفّح المنتجات خلال أول دقيقة: تظهر طلبات `GET /products` فقط، بلا **login**.
4. انتظر أكثر من 60 ثانية، ثم من القائمة (:) اختر **Run concurrency test**. سيظهر: `POST /auth/login` مرة واحدة فقط. - ثم 5 طلبات `GET /products/1..5`.
5. اختر **Log out**: تعود لشاشة Login، وزر الرجوع يغلق التطبيق.
6. افصل الإنترنت ثم اختر **Refresh**: رسالة "No internet connection" مع زر Retry، ولا يحدث crash.

⚠ **هذه الخطوة لا تعمل في الوضع التجريبي:** السيرفر الوهمي داخل الهاتف، فلا يحتاج إنترنت، وتظهر المنتجات حتى مع فصل الشبكة (فقط الصور لا تظهر). انظر "حدود السيرفر الوهمي" في الجزء التاسع.

7. شغّل الاختبارات بـ `./gradlew test`، واعرض نتيجة الـ 21 اختباراً.

56. تقابل المتطلبات مع الكود

المتطلب في ملف المهمة	الملف	الكود
Username & password inputs, Login button	<code>fragment_login.xml</code>	<code>password_input</code> , <code>username_input</code> <code>login_button</code>
Loading & error states	<code>LoginFragment</code> / <code>LoginViewModel</code>	<code>render()</code> , <code>LoginUiState</code>
Securely store the token	<code>EncryptedTokenStorage</code>	AES-256 + Android Keystore
Authorization Interceptor	<code>AuthInterceptor</code>	<code>.header("Authorization", "Bearer ...")</code>
Token validity: 60 seconds	<code>AuthManager</code>	<code>TOKEN_LIFETIME_MS = 60_000L</code>
Store tokenSavedAt	<code>AuthSession</code>	<code>savedAtMillis</code>
Treat token as expired after 60 s	<code>AuthManager</code>	<code>isExpired()</code>
Do not send requests with an expired token	<code>AuthManager</code>	<code>getValidToken()</code> قبل كل طلب
Trigger refresh flow automatically	<code>AuthManager</code>	<code>refresh()</code>
Only one refresh request must run	<code>AuthManager</code>	<code>synchronized(refreshLock)</code> + التحقق الثاني

المتطلب في ملف المهمة	الملف	الكود
Other requests wait and resume	AuthManager	الانتظار على القفل ثم المتابعة بال token الجديد
Simulate refresh by re-calling login	AuthManager	AuthApi.login → requestNewSession()
Encapsulate inside AuthManager	AuthManager	كل المنطق في كلاس واحد
Avoid infinite retry loops	AuthManager / NetworkModule TokenAuthenticator	عمل منفصل + محاولة واحدة + priorResponse
If refresh fails → force logout	MainActivity / AuthManager	SessionExpired → forceLogout()
Products list (RecyclerView)	ProductAdapter fragment_products.xml	DiffUtil + ListAdapter
Product details screen	ProductDetailFragment	
Logout: clear token, go to Login	ProductsViewModel nav_graph.xml	action_global_login + logout()
Architecture: MVVM	كل الشاشات	Fragment + ViewModel + Repository
Networking: Retrofit + OkHttp	NetworkModule	
(إضافة) رسالة دقيقة عند تعطل السيرفر	ErrorMessages / AuthExceptions	The server" → ServerUnavailableException "is unavailable right now
(إضافة) تشغيل التطبيق رغم تعطل FakeStore	MockBackendInterceptor gradle.properties	fakestore.useMockBackend=true (نسخة التطوير فقط)

الجزء التاسع: الوضع التجريبي (Mock Backend)

لماذا أضفناه؟ أثناء العمل تعطل سيرفر FakeStore للجميع، وكانت Cloudflare تعرض الخطأ `Host Error`. بدون سيرفر لا يمكن اختبار تسجيل الدخول ولا المنتجات ولا منطق الـ Token على الهاتف.

الحل الاحترافي: سيرفر وهمي داخل التطبيق، لنسخة التطوير فقط. هذا ما تفعله الفرق عادةً حتى لا يتوقف عملها بسبب خدمة خارجية.

المبدأ الأهم: السيرفر الوهمي يستبدل الطرف البعيد فقط. كل شيء آخر يعمل كما هو في الإنتاج:

الجزء	مع السيرفر الوهمي
AuthInterceptor وإضافة Bearer	يعمل كما هو
Token بعد 60 ثانية	يعمل كما هو
القفل والتجديد الواحد	يعمل كما هو
الخروج الإجباري ورسائل الخطأ	تعمل كما هي
الرد على الطلب	من داخل الهاتف بدل fakestoreapi.com

التفعيل والإيقاف: سطر واحد في `gradle.properties`:

```
fakestore.useMockBackend=true
```

بعد تغييره اضغط **Sync Now**، ثم شغل التطبيق من جديد.

57. data/mock/MockProducts.kt

الدور: بيانات المنتجات التي يردها السيرفر الوهمي.

app/src/main/java/com/example/fakestore/data/mock/MockProducts.kt

55 lines

```
1 package com.example.fakestore.data.mock
2
3 /**
4  * Sample catalogue returned by [MockBackendInterceptor], in the exact JSON shape of
5  * `GET https://fakestoreapi.com/products`. Images come from picsum.photos because the
6  * FakeStore image host is down together with its API.
7  */
8 internal object MockProducts {
9
10     private data class Item(
11         val id: Int,
12         val title: String,
13         val price: Double,
14         val category: String,
15         val description: String,
16         val rate: Double,
```

```

17         val count: Int,
18     )
19
20     private val items = listOf(
21         Item(1, "Fjallraven Foldsack No. 1 Backpack", 109.95, "men's clothing",
22             "Your perfect pack for everyday use and walks in the forest. Stash your laptop (up
to 15 inches) in the padded sleeve.", 3.9, 120),
23         Item(2, "Mens Casual Premium Slim Fit T-Shirts", 22.30, "men's clothing",
24             "Slim-fitting style, contrast raglan long sleeve, three-button henley placket, light
weight and soft fabric.", 4.1, 259),
25         Item(3, "Mens Cotton Jacket", 55.99, "men's clothing",
26             "Great outerwear jacket for spring, autumn and winter, suitable for working, hiking,
camping and climbing.", 4.7, 500),
27         Item(4, "John Hardy Women's Legends Naga Bracelet", 695.00, "jewelery",
28             "From the Legends Collection, the Naga was inspired by the mythical water dragon
that protects the ocean's pearl.", 4.6, 400),
29         Item(5, "Solid Gold Petite Micropave", 168.00, "jewelery",
30             "Satisfaction guaranteed. Return or exchange any order within 30 days.", 3.9, 70),
31         Item(6, "WD 2TB Elements Portable External Hard Drive", 64.00, "electronics",
32             "USB 3.0 and USB 2.0 compatibility, fast data transfers, improved PC performance and
high capacity.", 3.3, 203),
33         Item(7, "SanDisk SSD PLUS 1TB Internal SSD", 109.00, "electronics",
34             "Easy upgrade for faster boot up, shutdown, application load and response.", 2.9,
470),
35         Item(8, "Samsung 49-Inch CHG90 Curved Gaming Monitor", 999.99, "electronics",
36             "49 inch super ultrawide 32:9 curved gaming monitor with dual 27 inch screen side by
side.", 2.2, 140),
37         Item(9, "BIYLACLESEN Women's 3-in-1 Snowboard Jacket", 56.99, "women's clothing",
38             "Detachable liner fabric: warm fleece. Materials: 100% polyester. Detachable
functional liner.", 2.6, 235),
39         Item(10, "DANVOUY Womens T Shirt Casual Cotton Short", 12.99, "women's clothing",
40             "95% cotton, 5% spandex. Features: casual, short sleeve, letter print, v-neck,
fashion tees.", 3.6, 145),
41     )
42
43     val ids: Set<Int> = items.map { it.id }.toSet()
44
45     fun listJson(): String = items.joinToString(prefix = "[", postfix = "]", separator = ",") {
it.toJson() }
46
47     fun productJson(id: Int): String? = items.firstOrNull { it.id == id }?.toJson()
48
49     private fun Item.toJson(): String =
50         """
51         {"id":$id,"title":${quote(title)}, "price":$price, "description":${quote(description)}, "" +
52         """"category":${quote(category)}, "image": "https://picsum.photos/seed/fakestore$id/400", "" +
53         """"rating":{"rate":$rate, "count":$count}}""""
54
55     private fun quote(text: String) = "\"" + text.replace("\\", "\\\\").replace("\"", "\\\"") +
56     "\""
57 }

```

الشرح

- `internal object` : كائن وحيد، ومرئي داخل الوحدة `app` فقط.
- `items` : عشرة منتجات بنفس أسماء وأسعار وتصنيفات FakeStore الحقيقية تقريباً.
- `listJson()` و `productJson(id)` : تبني نص JSON بنفس شكل رد FakeStore بالضبط، فيحوّله Gson إلى `ProductDto` دون أي تغيير في باقي الكود.

- الصور من `picsum.photos` : لأن صور FakeStore موجودة على نفس السيرفر المتعطّل.
- `quote()` : تضع النص بين علامتي تنصيص، وتهرب الرموز الخاصة، حتى يبقى JSON صحيحاً.

★ 58. `data/mock/MockBackendInterceptor.kt`

الدور: السيرفر الوهمي نفسه.

`app/src/main/java/com/example/fakestore/data/mock/MockBackendInterceptor.kt`

85 lines

```

1  package com.example.fakestore.data.mock
2
3  import com.example.fakestore.data.auth.AuthHeaders
4  import com.example.fakestore.data.remote.dto.LoginRequest
5  import com.google.gson.Gson
6  import okhttp3.Interceptor
7  import okhttp3.MediaType.Companion.toMediaType
8  import okhttp3.Protocol
9  import okhttp3.Response
10 import okhttp3.ResponseBody.Companion.toResponseBody
11 import okio.Buffer
12 import java.util.concurrent.atomic.AtomicInteger
13
14 /**
15  * Development-only stand-in for fakestoreapi.com, used while the real server is down.
16  *
17  * It is added as the LAST interceptor, so everything before it runs exactly as in
18  * production: AuthInterceptor, token expiry, the refresh lock, logging. Only the
19  * remote end is replaced: the request never leaves the device.
20  *
21  * Enabled with `fakestore.useMockBackend=true` in gradle.properties (debug builds only).
22  */
23 class MockBackendInterceptor(
24     private val loginDelayMillis: Long = 800,
25     private val productsDelayMillis: Long = 300,
26 ) : Interceptor {
27
28     /** Number of login calls received: lets tests prove that a refresh happened only once. */
29     val loginCount = AtomicInteger()
30
31     private val gson = Gson()
32
33     override fun intercept(chain: Interceptor.Chain): Response {
34         val request = chain.request()
35         val segments = request.url.pathSegments.filter { it.isNotEmpty() }
36
37         val (code, body) = when {
38             request.method == "POST" && segments == listOf("auth", "login") → login(chain)
39             request.method == "GET" && segments.firstOrNull() == "products" → products(chain,
segments)
40         } else → 404 to "Not found"
41     }
42
43     return Response.Builder()
44         .request(request)
45         .protocol(Protocol.HTTP_1_1)
46         .code(code)
47         .message(if (code in 200..299) "OK" else "Error")

```

```

48         .body(body.toResponseBody(JSON))
49         .build()
50     }
51
52     private fun login(chain: Interceptor.Chain): Pair<Int, String> {
53         val number = loginCount.incrementAndGet()
54         Thread.sleep(loginDelayMillis) // keeps a refresh "in flight" so concurrent requests
must wait
55
56         val buffer = Buffer()
57         chain.request().body?.writeTo(buffer)
58         val credentials = runCatching { gson.fromJson(buffer.readUtf8(),
LoginRequest::class.java) }.getOrNull()
59
60         return if (credentials?.username == USERNAME && credentials.password == PASSWORD) {
61             200 to ""{"token":"mock-token-$number"}""
62         } else {
63             401 to "username or password is incorrect"
64         }
65     }
66
67     private fun products(chain: Interceptor.Chain, segments: List<String>): Pair<Int, String> {
68         // Unlike the real FakeStore, the mock really enforces authentication.
69         val token = AuthHeaders.tokenFrom(chain.request().header(AuthHeaders.AUTHORIZATION))
70         if (token.isNullOrEmpty()) return 401 to "Missing bearer token"
71
72         Thread.sleep(productsDelayMillis)
73         if (segments.size == 1) return 200 to MockProducts.listJson()
74
75         val id = segments[1].toIntOrNull()
76         val product = id?.let { MockProducts.productJson(it) } ?: return 404 to "Product not
found"
77         return 200 to product
78     }
79
80     companion object {
81         const val USERNAME = "mor_2314"
82         const val PASSWORD = "83r5^_"
83         private val JSON = "application/json; charset=utf-8".toMediaType()
84     }
85 }

```

الشرح

كيف "يردّ" Interceptor بدل السيرفر؟ أي Interceptor عادي يستدعي `chain.proceed(request)` ليمرّ الطلب إلى الأمام. هذا الكلاس **لا يستدعيها أبداً**، بل يبني بنفسه `Response` كاملاً، بكود وجسم JSON، ويعيده. فيظن Retrofit و OkHttp أن الرد جاء من الإنترنت.

`intercept()` : يقرأ الطريقة (POST / GET) والمسار، ويوجّه الطلب:

الطلب	الدالة
POST /auth/login	login()
GET /products أو GET /products/{id}	products()
أي شيء آخر	404

: login()

- loginCount.incrementAndGet() : يعدّ طلبات الدخول. تستخدمه الاختبارات لإثبات حدوث تجديد واحد فقط.
- Thread.sleep(loginDelayMillis) : تأخير مقصود (800 ميلي ثانية). بدونه يكون الرد فورياً، فلا تجتمع الطلبات المتزامنة عند القفل، ويصبح اختبار التزامن بلا معنى.
- يقرأ جسم الطلب، ويحوّله بـ GSON إلى LoginRequest :
 - بيانات صحيحة: 200 مع {"token": "mock-token-N"} ، فكل تجديد يعطي token برقم جديد.
 - بيانات خاطئة: 401 ، تماماً مثل FakeStore.

: products()

- يتحقق من وجود Bearer token فعلاً. إذا غاب يرد 401. هذا أكثر صرامة من FakeStore الحقيقي، الذي لا يتحقق أصلاً.
- /products : يرجع القائمة كاملة.
- /products/{id} : يرجع المنتج، أو 404 إذا لم يوجد.
- Response.Builder() : يبني الرد: الطلب الأصلي، والبروتوكول، والكود، والرسالة، والجسم بنوع application/json .

59. test/.../MockBackendInterceptorTest.kt

الدور: إثبات أن التطبيق كله يعمل فوق السيرفر الوهمي.

app/src/test/java/com/example/fakestore/data/mock/MockBackendInterceptorTest.kt

93 lines

```
1 package com.example.fakestore.data.mock
2
3 import com.example.fakestore.data.auth.AuthInterceptor
4 import com.example.fakestore.data.auth.AuthManager
5 import com.example.fakestore.data.auth.FakeClock
6 import com.example.fakestore.data.auth.InMemoryTokenStorage
7 import com.example.fakestore.data.auth.InvalidCredentialsException
8 import com.example.fakestore.data.auth.TokenAuthenticator
9 import com.example.fakestore.data.remote.ApiConfig
10 import com.example.fakestore.data.remote.AuthApi
11 import com.example.fakestore.data.remote.ProductApi
12 import kotlinx.coroutines.Dispatchers
13 import kotlinx.coroutines.async
14 import kotlinx.coroutines.awaitAll
15 import kotlinx.coroutines.runBlocking
16 import okhttp3.OkHttpClient
17 import org.junit.Assert.assertEquals
18 import org.junit.Assert.assertTrue
19 import org.junit.Assert.fail
20 import org.junit.Test
21 import retrofit2.HttpException
22 import retrofit2.Retrofit
23 import retrofit2.converter.gson.GsonConverterFactory
24
25 /** The app wired exactly like NetworkModule with the mock enabled: no network at all. */
26 class MockBackendInterceptorTest {
27
28     private val mock = MockBackendInterceptor(loginDelayMillis = 200, productsDelayMillis = 0)
29     private val clock = FakeClock()
```

```

30 private val storage = InMemoryTokenStorage()
31
32 private fun retrofit(client: OkHttpClient) = Retrofit.Builder()
33     .baseUrl(ApiConfig.BASE_URL)
34     .client(client)
35     .addConverterFactory(GsonConverterFactory.create())
36     .build()
37
38 private val authApi = retrofit(OkHttpClient.Builder().addInterceptor(mock).build())
39     .create(AuthApi::class.java)
40 private val authManager = AuthManager(authApi, storage, clock)
41 private val productApi = retrofit(
42     OkHttpClient.Builder()
43         .addInterceptor(AuthInterceptor(authManager))
44         .addInterceptor(mock)
45         .authenticator(TokenAuthenticator(authManager))
46         .build(),
47 ).create(ProductApi::class.java)
48
49 @Test
50 fun `login and products work end to end`() = runBlocking {
51     authManager.login(MockBackendInterceptor.USERNAME, MockBackendInterceptor.PASSWORD)
52
53     val products = productApi.getProducts()
54     val first = productApi.getProduct(1)
55
56     assertEquals(10, products.size)
57     assertEquals(1, first.id)
58     assertEquals("mock-token-1", storage.session!!.token)
59 }
60
61 @Test
62 fun `wrong password is rejected`() = runBlocking {
63     try {
64         authManager.login(MockBackendInterceptor.USERNAME, "wrong")
65         fail("Expected InvalidCredentialsException")
66     } catch (e: InvalidCredentialsException) {
67         assertEquals(401, e.httpCode)
68     }
69 }
70
71 @Test
72 fun `expired token with parallel requests triggers a single mock login`() = runBlocking {
73     authManager.login(MockBackendInterceptor.USERNAME, MockBackendInterceptor.PASSWORD)
74     clock.advanceBy(61_000)
75
76     (1..5).map { async(Dispatchers.IO) { productApi.getProduct(it) } }.awaitAll()
77
78     assertEquals(2, mock.loginCount.get()) // the initial login + exactly one refresh
79     assertEquals("mock-token-2", storage.session!!.token)
80 }
81
82 @Test
83 fun `unknown product returns 404`() = runBlocking {
84     authManager.login(MockBackendInterceptor.USERNAME, MockBackendInterceptor.PASSWORD)
85     try {
86         productApi.getProduct(999)
87         fail("Expected HttpException")
88     } catch (e: HttpException) {
89         assertEquals(404, e.code())
90     }
91     assertTrue(MockProducts.ids.contains(1))

```

```
92 }
93 }
```

الشرح

يبيّن الاختبار العميلين بنفس ترتيب `NetworkModule` : `AuthInterceptor` ، ثم السيرفر الوهمي في الآخر.

الاختبار	ماذا يثبت
<code>login and products work end to end</code>	الدخول ثم القائمة (10 منتجات) ثم منتج واحد
<code>wrong password is rejected</code>	كلمة مرور خاطئة تعطي 401 و <code>InvalidCredentialsException</code>
<code>expired token with parallel requests...</code> ★	بعد 61 ثانية: 5 طلبات متزامنة تنتج login واحداً إضافياً فقط (<code>loginCount = 2</code>) : الدخول الأول + تجديد واحد
<code>unknown product returns 404</code>	منتج غير موجود يعطي 404

حدود السيرفر الوهمي: ما الذي لا يمكن رؤيته على الهاتف؟

السيرفر الوهمي يرد دائماً بنجاح على البيانات الصحيحة، لذلك بعض الحالات **لا تظهر على الهاتف** في الوضع التجريبي:

الحالة	لماذا لا تظهر	أين أثبتناها
الخروج الإجباري عند رفض التجديد	السيرفر الوهمي لا يرفض أبداً بيانات دخول محفوظة وصحيحة	<code>rejected refresh forces logout...</code> في القسم 51
رسالة "لا يوجد إنترنت"	الرد يأتي من داخل الهاتف، فالشبكة غير مستخدمة	<code>network failure during refresh...</code> في القسم 51
رسالة "السيرفر غير متاح"	السيرفر الوهمي لا يتعطل	<code>server outage...</code> في القسم 51
<code>TokenAuthenticator</code> عند رد 401	انظر الشرح التقني أدناه	القسم 53

الشرح التقني لسطر `TokenAuthenticator` :

- `OkHttp` يستدعي الـ `Authenticator` من حلقة داخلية اسمها `RetryAndFollowUpInterceptor` ، وهي تعمل **بعد** كل الـ `Interceptors` التي نضيفها بـ `addInterceptor` .
- السيرفر الوهمي واحد من هذه الـ `Interceptors` ، ويرد **قبل** الوصول إلى تلك الحلقة.
- لذلك لو رد السيرفر الوهمي بـ 401 ، لا يرى `OkHttp` ذلك كرد من الشبكة، ولا يستدعي `TokenAuthenticator` .
- عملياً لا يحدث هذا أبداً، لأن `AuthInterceptor` يضيف الـ `token` دائماً قبل السيرفر الوهمي.

الخلاصة للمقابلة: "الهاتف يعرض المسار الطبيعي والتزامن. أما مسارات الفشل فمثبتة بالاختبارات، لأنها أصعب في الإنتاج يدوياً على الهاتف، والاختبار يكررها بدقة في كل مرة."

60. كيف تعرض الوضع التجريبي للمدير

1. افتح التطبيق: سيظهر الشريط الملون أعلى شاشة الدخول: `Demo mode: using a built-in mock server instead of fakestoreapi.com`
2. قل للمدير بوضوح: "سيرفر FakeStore تعطل للجميع أثناء العمل، فأضفت سيرفراً وهمياً لنسخة التطوير فقط، يستبدل الطرف البعيد ويُبقي منطق الـ Token حقيقياً. نسخة الإنتاج تستخدم FakeStore دائماً."
3. اعرض الخطوات نفسها من القسم 55. في Logcat ستظهر الطلبات كأنها ذهبت إلى `fakestoreapi.com` ، لكن الرد يأتي من داخل الهاتف.
4. عندما يعود FakeStore: غيّر المفتاح إلى `false` ، واضغط **Sync Now**، فيختفي الشريط ويعود التطبيق للسيرفر الحقيقي.

الجزء العاشر ★: فهم المنطق بالسيناريوهات

هذا الجزء يشرح **ماذا يحدث** في التطبيق كقصة، خطوة بخطوة، دون الدخول في الكود. كل خطوة تذكر الملف المسؤول عنها، فإذا أردت رؤية الكود ارجع لقسمه.

61. المنطق كله في سبع جمل

1. عند الدخول نحفظ **ثلاثة أشياء** مشفرة: الـ token، ووقت حفظه (tokenSavedAt)، وبيانات الدخول.
2. كل طلب منتجات يمر بـ AuthInterceptor، وهو يسأل AuthManager: "أعطني token صالحاً".
3. AuthManager يحسب العمر: tokenSavedAt - الآن. أقل من 60 ثانية يعني صالح، فيعيده فوراً.
4. إذا كان منتجياً، يجدد بإرسال login من جديد بالبيانات المحفوظة، **قبل** إرسال الطلب.
5. التجديد داخل **قفل**: أول طلب يجدد، والبقية ينتظرون ثم يأخذون الـ token الجديد **دون** تجديد آخر.
6. طلب login نفسه يمر بعميل منفصل بلا AuthInterceptor، فلا يمكن أن يطلب تجديداً، **فلا حلقة لا نهائية**.
7. إذا رفض السيرفر التجديد: خروج إجباري. وإذا كانت المشكلة في الشبكة أو السيرفر: تبقى الجلسة، وتظهر رسالة.

62. السيناريوهات خطوة بخطوة

السيناريو 1: فتح التطبيق

#	الملف	ماذا يحدث
1	MainActivity	تسأل authManager.isLoggedIn(): هل توجد جلسة محفوظة؟
2	EncryptedTokenStorage	يفك تشفير الملف ويقرأ الحقول
3	MainActivity	توجد جلسة: تبدأ بشاشة المنتجات. لا توجد: تبدأ بشاشة Login

ملاحظة: لا نتحقق هنا من انتهاء الـ token. حتى لو كان منتجياً منذ ساعات، سيُجدد تلقائياً مع أول طلب (السيناريو 4).

السيناريو 2: تسجيل الدخول

#	الملف	ماذا يحدث
1	LoginFragment	المستخدم يضغط الزر، فيُرسل النصين للـ ViewModel
2	LoginViewModel	الحقول فارغة؟ رسالة خطأ دون إرسال. غير ذلك: الحالة isloading = true
3	AuthManager.login	ينتقل إلى thread خلفي (Dispatchers.IO)
4	requestNewSession	يرسل POST /auth/login على العميل العادي

#	الملف	ماذا يحدث
5	requestNewSession	200 مع token: ينشئ AuthSession بوقت الآن. 4xx: InvalidCredentialsException 5xx: ServerUnavailableException
6	saveIfStillCurrent	يحفظ الجلسة مشفرة
7	LoginViewModel	isLoggedIn = true ، أو رسالة الخطأ المناسبة
8	LoginFragment	ينتقل للمنتجات، ويحذف Login من سجل الرجوع

السيناريو 3: طلب عادي وال token صالح (عمره 20 ثانية)

#	الملف	ماذا يحدث
1	ProductsViewModel	repository.getProducts()
2	AuthInterceptor	getValidToken()
3	AuthManager	المسار السريع: العمر 20 ثانية، أقل من 60، فيعيد ال token دون أي قفل
4	AuthInterceptor	يضيف Authorization: Bearer <token> ويرسل

هذه هي الحالة الأكثر حدوثاً، ولهذا جعلناها بلا قفل ولا انتظار.

السيناريو 4: طلب واحد وال token منتهٍ (عمره 75 ثانية)

#	الملف	ماذا يحدث
1	AuthManager	المسار السريع: $60 \leq 75$ ، منتهٍ
2	AuthManager	يدخل القفل refreshLock
3	AuthManager	التحقق الثاني: ما زال منتهياً، فيجدد
4	refresh	login جديد بالبيانات المحفوظة، ثم حفظ ال token الجديد بوقت الآن
5	AuthInterceptor	يرسل الطلب بال token الجديد

النقطة المهمة: السيرفر لم يستقبل أبداً ال token المنتهي. التجديد حدث قبل الإرسال، لا بعد الفشل.

السيناريو 5: خمسة طلبات معاً وال token منتهٍ (اختبار التزامن)

الزمن	الطلب 1	الطلبات 2 إلى 5
ms 0	منتهٍ، يدخل القفل	منتهٍ، تنتظر عند القفل
ms 0	التحقق الثاني: منتهٍ، يرسل login	تنتظر
800 ms	يصل الرد، يحفظ ال token الجديد، يخرج	تنتظر

الزمن	الطلب 1	الطلبات 2 إلى 5
800 ms	يرسل طلبه 	تدخل واحداً تلو الآخر، والتحقق الثاني يجد token صالحاً، فتخرج فوراً وترسل 

النتيجة في Logcat: سطر login واحد، ثم 5 طلبات منتجات.

لماذا التحقق الثاني هو السر؟ بدون، كل طلب ينتظر دوره ثم يجد بنفسه: 5 طلبات login متتالية. جربنا حذفه في الاختبارات، فأنتجت 10 طلبات متزامنة 10 طلبات login.

السيناريو 6: السيرفر يرفض التجديد (مثلاً تغيّرت كلمة المرور)

#	الملف	ماذا يحدث
1	requestNewSession	السيرفر يرد 401، فيُرمى InvalidCredentialsException
2	refresh	يلتقطه، ويستدعي forceLogout()
3	forceLogout	يُسمح الجلسة، ثم يرسل الحدث SessionExpired
4	refresh	يرمي SessionExpiredException ، دون إعادة محاولة
5	ProductsViewModel	يعرض "Your session has expired"
6	MainActivity	تستقبل الحدث: Toast، ثم انتقال إلى Login مع مسح كل الشاشات

لماذا لا نعيد المحاولة؟ لأن البيانات نفسها سترفض مرة أخرى، وإعادة المحاولة هنا هي الحلقة اللانهائية التي تحدّر منها المهمة.

السيناريو 7: انقطاع الإنترنت أثناء التجديد

#	الملف	ماذا يحدث
1	requestNewSession	OkHttp يرمي IOException (لا يوجد اتصال)
2	refresh	لا يلتقطه: يمر للطلب كما هو. الجلسة لا تُمسح
3	ErrorMessages	يحوّله إلى "No internet connection"
4	المستخدم	يضغط Retry بعد عودة الإنترنت، فيحدث التجديد بنجاح

القرار: المستخدم لم يخطئ، والمشكلة مؤقتة، فلا نجبره على تسجيل الدخول من جديد. نفس الشيء عند تعطل السيرفر (5xx).

السيناريو 8: المستخدم يضغط Logout أثناء تجديد جارٍ

#	من	ماذا يحدث
1	Thread التجديد	يحفظ رقم الجيل الحالي (مثلاً 4) ثم يرسل login
2	المستخدم	يضغط Logout: الجيل يصبح 5، والجلسة تُمسح
3	Thread التجديد	يصل الرد بـ token جديد

#	من	ماذا يحدث
4	<code>saveIfStillCurrent</code>	يقارن: 4 ≠ 5، إذن حدث خروج. لا يحفظ . ويرمي <code>SessionExpiredException</code>

بدون رقم الجيل: يُحفظ ال token الجديد، فيعود المستخدم مسجلاً للدخول بعد أن خرج.

السيناريو 9: السيرفر يرد 401 على طلب منتجات (سيرفر حقيقي صارم)

#	الملف	ماذا يحدث
1	<code>OkHttp</code>	يرى 401، فيستدعي <code>TokenAuthenticator</code>
2	<code>TokenAuthenticator</code>	هل هذه إعادة محاولة سابقة (<code>priorResponse</code>)؟ لا
3	<code>refreshAfterUnauthorized</code>	هل جدّد طلب آخر قبلنا؟ إذا نعم نأخذ ال token الموجود. إذا لا نجدّد
4	<code>OkHttp</code>	يعيد الطلب مرة واحدة بال token الجديد
5	إذا رد 401 مرة ثانية	<code>priorResponse</code> موجود الآن: خروج إجباري، ولا محاولة ثالثة

FakeStore لا يرد بـ 401 أبداً، فهذا السيناريو "شبكة أمان" للسيرفرات الحقيقية.

السيناريو 10: إغلاق التطبيق دون Logout ثم فتحه

#	الملف	ماذا يحدث
1	النظام	التطبيق يُغلق. الجلسة باقية في الملف المشفّر على القرص
2	<code>MainActivity</code>	عند الفتح: <code>isLoggedIn()</code> يجد جلسة، فيبدأ بالمنتجات مباشرة
3	<code>AuthManager</code>	أول طلب: إذا مرت أكثر من 60 ثانية، تجديد واحد تلقائي (السيناريو 4)

لماذا `currentTimeMillis` وليس `elapsedRealtime` ؟ لأن الثاني يعود للصفر عند إعادة تشغيل الهاتف، فيصبح `tokenSavedAt` المحفوظ بلا معنى.

63. اختبر فهمك (أسئلة متوقعة في المقابلة)

س: لماذا عميلاً `OkHttp` وليس عميلاً واحداً؟

ج: لو مر طلب login بال `AuthInterceptor` ، لطلب token صالحاً، وال token منتهٍ، فيطلب تجديداً، والتجديد هو login، فيمر بال `Interceptor` مرة أخرى... حلقة لا نهائية. العميل المنفصل يقطعها من الأساس.

س: لماذا `synchronized` وليس `Mutex` ؟

ج: `intercept()` دالة عادية تعمل على thread حقيقي من `OkHttp`، وليست `suspend`. `Mutex` يحتاج `suspend`، فكنا سنحتاج `runBlocking` داخل ال `Interceptor`.

س: لماذا المسار السريع خارج القفل؟

ج: لأن 99% من الطلبات تجد token صالحاً. لو دخلت كلها القفل لانتظرت بعضها بلا سبب.

س: ماذا لو حذفنا التحقق الثاني داخل القفل؟

ج: كل طلب منتظر سيجدد بعد دوره، فتصبح 5 طلبات = 5 login. الاختبار `concurrent callers ... exactly one refresh` يكشف ذلك فوراً.

س: لماذا تترث الأخطاء من `IOException` ؟

ج: لأنها تُرث من داخل `OkHttp`. `OkHttp` يمرر `IOException` كفشل عادي للطلب، أما أي نوع آخر فيُسقط التطبيق.

س: كيف تختبر انتهاء الصلاحية دون انتظار 60 ثانية؟

ج: الوقت يأتي من interface اسمه `clock`. في الاختبارات نستخدم `FakeClock` ونقدمه 61 ثانية فوراً.

س: لماذا تحفظ كلمة المرور؟ أليس هذا خطراً؟

ج: `FakeStore` لا يوفر `refresh token`، والمهمة تطلب التجديد بإعادة `login`. لذلك نحفظها **مشفرة** بمفتاح في `Android Keystore`. في تطبيق حقيقي نحفظ `refresh token` من السيرفر، ولا نحفظ كلمة المرور أبداً.

س: ما الفرق بين `AuthInterceptor` و `TokenAuthenticator` ؟

ج: الأول **استباقي**: يمنع إرسال `token` منتهٍ أصلاً. الثاني **تفاعلي**: يعالج رد 401 إذا رفض السيرفر `token` كنا نظنه صالحاً.

64. دليل اختبار التطبيق على الهاتف

التحضير (مرة واحدة):

- افتح Logcat في Android Studio، واكتب في البحث: `okhttp`
- تأكد أن الهاتف متصل عبر Wireless debugging، وأن اسمه ظاهر بجانب زر .
- اضغط  Run.

أشكال الأسطر في Logcat:

```
→ POST https://fakestoreapi.com/auth/login (43-byte body)
← 200 OK https://fakestoreapi.com/auth/login (805ms, 24-byte body)
→ GET https://fakestoreapi.com/products
← 200 OK https://fakestoreapi.com/products (302ms, ...)
```

السهم `>` هو الطلب، والسهم `<` هو الرد. العنوان يبقى `fakestoreapi.com` حتى في الوضع التجريبي، لكن الرد يأتي من داخل الهاتف.

قائمة الاختبارات:

#	الخطوات	النتيجة المتوقعة
1	افتح التطبيق	شاشة Login، وفي أعلاها شريط <code>Demo mode</code>
2	اضغط Log in والحقول فارغة	<code>Please enter your username and password.</code> ولا يظهر أي سطر في Logcat
3	اكتب اسم المستخدم <code>mor_2314</code>	الحرف الأول يبقى صغيراً، ولا تظهر اقتراحات (تعديل المراجعة)

#	الخطوات	النتيجة المتوقعة
4	كلمة مرور خاطئة، ثم Log in	Invalid username or password. دائرة تحميل، ثم Logcat وفي سطر 401 ←
5	كلمة المرور الصحيحة 83r5^_	قائمة بـ 10 منتجات. Logcat: login واحد، ثم GET /products
6	اضغط منتجاً	شاشة التفاصيل: صورة، وعنوان، وسعر، وتقييم، ووصف
7	ارجع، ثم : ← Refresh خلال أول 60 ثانية	Logcat: GET /products فقط، بلا login
8	انتظر أكثر من 60 ثانية، ثم : ← Run concurrency test	Logcat: login واحد ثم 5 طلبات GET /products/1 إلى 5. رسالة أسفل الشاشة: 5 parallel requests finished...
9	أعد الاختبار 8 مباشرة (قبل مرور 60 ثانية)	5 طلبات منتجات بلا login، لأن الـ token جديد
10	★ أغلق التطبيق دون Logout (اسحبه من قائمة التطبيقات الأخيرة)، ثم افتحه	يفتح على المنتجات مباشرة دون شاشة Login
11	بعد الاختبار 10: إذا مرت أكثر من 60 ثانية على آخر login	Logcat: login واحد تلقائي قبل GET /products
12	في شاشة المنتجات اضغط زر الرجوع في الهاتف	يُغلق التطبيق، ولا يعود لشاشة Login
13	: ← Log out زر الرجوع الآن يُغلق التطبيق	شاشة Login. زر الرجوع الآن يُغلق التطبيق
14	أغلق التطبيق وافتحه بعد Logout	يفتح على Login، لأن الجلسة مُسحت
15	على الكمبيوتر، في Terminal الخاص بـ Android Studio: .\gradlew.bat test	BUILD SUCCESSFUL ، والـ 21 اختباراً ناجحة

إذا فشل أي اختبار: صوّر الشاشة، وانسخ أسطر Logcat التي تبدأ بـ < و -- و ← ، وأرسلها.

65. نتائج المراجعة الشاملة

ما الذي رُوجع:

- كل ملفات المشروع (56 ملفاً): الكود، وملفات XML، وGradle، والاختبارات.
- الاختبارات الـ 21 أعيد تشغيلها بعد التعديلات: **كلها ناجحة**.
- كل متطلب في ملف المهمة له كود يقابله (القسم 56).

ما الذي عُدّل:

التعديل	الملف	السبب
@Synchronized على read و save و clear	EncryptedTokenStorage	منع قراءة جلسة نصف مكتوبة (القسم 16)

التعديل	الملف	السبب
حقل اسم المستخدم بلا تكبير ولا تصحيح	fragment_login.xml	المشكلة التي واجهناها مع لوحة مفاتيح Infinix (القسم 38)
تنبيه أن فصل الإنترنت لا يظهر خطأ في الوضع التجريبي	هذا الملف، القسم 55	كان الشرح يعد بنتيجة لن تظهر
جدول "حدود السيرفر الوهمي"	هذا الملف، الجزء التاسع	توضيح ما يُرى على الهاتف وما يُثبت بالاختبارات

حدود معروفة ومقصودة (قلها بصراحة إذا سُئلت):

- 1. كلمة المرور محفوظة (مشفرة): ضرورة لمحاكاة التجديد، ولا تُستخدم في تطبيق حقيقي.
- 2. EncryptedSharedPreferences أوقفت Google تطويرها: خلف interface، ويمكن استبدالها بسطر واحد.
- 3. قراءة الجلسة عند فتح التطبيق تحدث على الـ Main thread: تستغرق أجزاء من الثانية. في تطبيق كبير نقرأها في الخلفية مع شاشة بداية (Splash screen).
- 4. مسارات الفشل لا تُعرض على الهاتف في الوضع التجريبي: مثبتة بالاختبارات (الجزء التاسع).